

成人人口普查收入

第三組

111029208	吳文秀
112029012	翁鈺淇
112029030	魏麒恩
112029040	劉涵雲
112029043	朱芯僊

組員介紹

111029208	吳文秀
112029012	翁鈺淇
112029030	魏麒恩
112029040	劉涵雲
112029043	朱芯僊



目錄

01

研究背景

02

資料描述

03

資料前處理

04

視覺化分析

05

問題假說

06

實驗設計
& 模型訓練

07

參考文獻
& 分工表

08

Q&A與建議

研究背景



研究背景

我們選擇這個主題，是因為「收入」與「教育、性別、年齡」等因素可能密切相關，這些因素其實都反映了社會階層與資源分配的問題。透過成人人口普查資料，可以幫助政策制定者了解不同族群在經濟上的落差，進一步作為制定社會政策與教育政策的重要參考依據。

資料描述



原始資料說明

- ① 該數據由 RONNY KOHAVI 和 BARRY BECKER(數據挖掘和可視化,SILICON GRAPHICS)從1994 年人口普查局數據庫中提取。
共有32,561筆資料，15個資料欄位。
- ② 其中包括類別型資料 9 欄
以及連續型資料 6欄。

一 類別型資料

欄位名稱	說明	範例值
workclass	工作類別 (如政府、私人企業、自營等)	Private, Self-emp-not-inc, ?
education	教育程度名稱	HS-grad, Bachelors
marital.status	婚姻狀態	Married-civ-spouse, Divorced,Widowed, Separated,Never-married
occupation	職業類別	Prof-specialty, Sales

— 類別型資料

欄位名稱	說明	範例值
relationship	家庭關係	Husband, Not-in-family
race	種族	White, Black, Asian-Pac- Islander
sex	性別	Male, Female
native.country	原生國家	United-States, Mexico
income	收入分類 (是否超過 50,000 美元)	<=50K, >50K

— 連續型資料

欄位名稱	說明	範例值
age	年齡（以歲為單位）	17-90
fnlwgt	最終權重， 代表該筆樣本在母體中的代表性	12285-1,484,705
education.num	教育程度數值化 （年級數）	1-16
capital.gain	資本收益	0-99,999
capital.loss	資本損失	0-4,356
hours.per.week	每週工作時數	1-99

資料前處理



缺失值刪除

使用Excel將資料全部選取→點選資料點擊篩選問號
→成為新的資料→把新的資料拉進SPSS

刪除缺失值後
剩下**30162**筆資料

A	B	C	D	E	F	G	H
age	workclass	fnlwgt	educatio	educatio	marital	occupat	relati
63	?	29859	HS-grad	9	Married-ci?		Husb
76	?	224680	Prof-schoc	15	Married-ci?		Husb
67	?	184506	11th	7	Married-ci?		Husb
27	?	501172	Some-coll	10	Married-ci?		Husb
24	?	151153	Some-coll	10	Never-mai?		Not-i
61	?	139391	Some-coll	10	Married-ci?		Husb
69	?	323016	Bachelors	13	Married-ci?		Husb
58	?	266792	Some-coll	10	Married-ci?		Husb
17	?	304873	10th	6	Never-mai?		Own
20	?	273701	Some-coll	10	Never-mai?		Other
58	?	353244	Bachelors	13	Widowed?		Unm
75	?	111177	Bachelors	13	Widowed?		Not-i
65	?	224472	Prof-schoc	15	Never-mai?		Not-i
68	?	146645	Doctorate	16	Married-ci?		Husb
67	?	129188	Doctorate	16	Married-ci?		Husb
79	?	76641	Masters	14	Married-ci?		Husb
38	?	70282	Masters	14	Married-ci?		Wife
52	?	92968	Masters	14	Married-ci?		Wife

A	B	C	D
1	age	workclass	fnlwgt
19	61	Private	29059

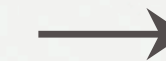
1	age	workclass	fnlwgt	educatio	educatio	marital	occupat	relation	race	sex	capital.gai	capital.	hours.p	native.c	income
2	82	Private	132870	HS-grad	9	Widowed	Exec-man	Not-in-fan	White	Female	0	4356	18	United-Sta	<=50K
3	54	Private	140359	7th-8th	4	Divorced	Machine-c	Unmarrie	White	Female	0	3900	40	United-Sta	<=50K
4	41	Private	264663	Some-coll	10	Separated	Prof-speci	Own-child	White	Female	0	3900	40	United-Sta	<=50K
5	34	Private	216864	HS-grad	9	Divorced	Other-serv	Unmarrie	White	Female	0	3770	45	United-Sta	<=50K
6	38	Private	150601	10th	6	Separated	Adm-cleri	Unmarrie	White	Male	0	3770	40	United-Sta	<=50K
7	74	State-gov	88638	Doctorate	16	Never-mai	Prof-speci	Other-rela	White	Female	0	3683	20	United-Sta	>50K
8	68	Federal-gc	422013	HS-grad	9	Divorced	Prof-speci	Not-in-fan	White	Female	0	3683	40	United-Sta	<=50K
9	45	Private	172274	Doctorate	16	Divorced	Prof-speci	Unmarrie	Black	Female	0	3004	35	United-Sta	>50K
10	38	Self-emp-i	164526	Prof-schoc	15	Never-mai	Prof-speci	Not-in-fan	White	Male	0	2824	45	United-Sta	>50K
11	52	Private	129177	Bachelors	13	Widowed	Other-serv	Not-in-fan	White	Female	0	2824	20	United-Sta	>50K
12	32	Private	136204	Masters	14	Separated	Exec-man	Not-in-fan	White	Male	0	2824	55	United-Sta	>50K
13	46	Private	45363	Prof-schoc	15	Divorced	Prof-speci	Not-in-fan	White	Male	0	2824	40	United-Sta	>50K
14	45	Private	172822	11th	7	Divorced	Transport-	Not-in-fan	White	Male	0	2824	76	United-Sta	>50K
15	57	Private	317847	Masters	14	Divorced	Exec-man	Not-in-fan	White	Male	0	2824	50	United-Sta	>50K
16	34	Private	203034	Bachelors	13	Separated	Sales	Not-in-fan	White	Male	0	2824	50	United-Sta	>50K
17	37	Private	188774	Bachelors	13	Never-mai	Exec-man	Not-in-fan	White	Male	0	2824	40	United-Sta	>50K
18	29	Private	77009	11th	7	Separated	Sales	Not-in-fan	White	Female	0	2754	42	United-Sta	<=50K
19	61	Private	29059	HS-grad	9	Divorced	Sales	Unmarrie	White	Female	0	2754	25	United-Sta	<=50K
20	51	Private	153870	Some-coll	10	Married-ci	Transport-	Husband	White	Male	0	2603	40	United-Sta	<=50K
21	21	Private	34310	Assoc-voc	11	Married-ci	Craft-repa	Husband	White	Male	0	2603	40	United-Sta	<=50K
22	33	Private	228696	1st-4th	2	Married-ci	Craft-repa	Not-in-fan	White	Male	0	2603	32	Mexico	<=50K
23	49	Private	122066	5th-6th	3	Married-ci	Other-serv	Husband	White	Male	0	2603	40	Greece	<=50K
24	37	Self-emp-i	107164	10th	6	Never-mai	Transport-	Not-in-fan	White	Male	0	2559	50	United-Sta	>50K
25	38	Private	175360	10th	6	Never-mai	Prof-speci	Not-in-fan	White	Male	0	2559	90	United-Sta	>50K
26	23	Private	44064	Some-coll	10	Separated	Other-serv	Not-in-fan	White	Male	0	2559	40	United-Sta	>50K

缺失值檢查

確認資料內無缺失值

各欄位缺漏值統計：

	缺漏值數量
age	0
workclass	1836
fnlwgt	0
education	0
education.num	0
marital.status	0
occupation	1843
relationship	0
race	0
sex	0
capital.gain	0
capital.loss	0
hours.per.week	0
native.country	583
income	0



各欄位缺漏值統計：

	缺漏值數量
age	0
workclass	0
fnlwgt	0
education	0
education.num	0
marital.status	0
occupation	0
relationship	0
race	0
sex	0
capital.gain	0
capital.loss	0
hours.per.week	0
native.country	0
income	0
收入	0
教育程度	0
性別	0
種族	0

沒有發現缺漏值！

異常值刪除

age =>18 and age <=65

18歲以下未有工作能力

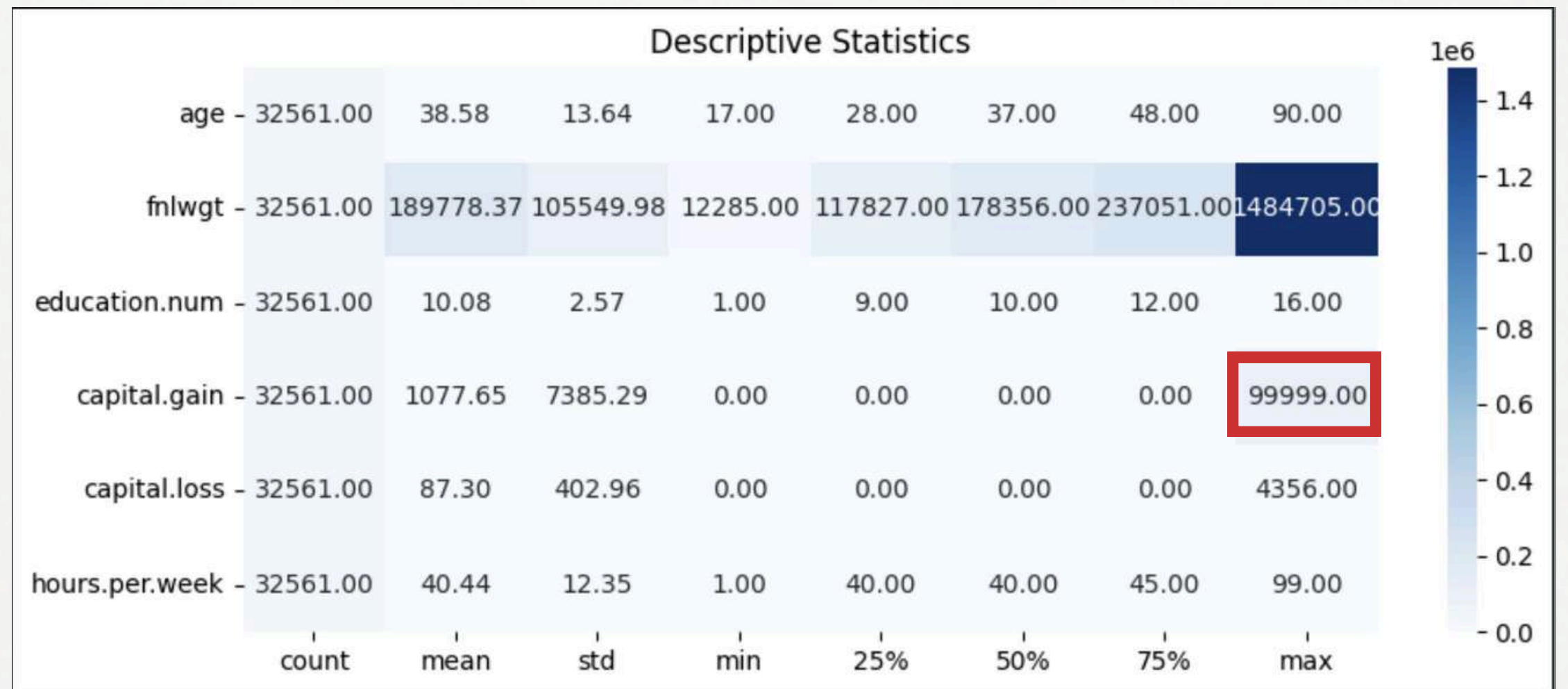
65歲以上已退休

capital_gain<=99999

為極端值

模型可能會被影響

刪除異常值後
剩下**28852**筆資料



資料編碼

income_bin :

"0" 表示 " $\leq 50k$ "

"1" 表示 " $> 50k$ "

sex_cat :

"1" 表示 "男"

"2" 表示 "女"

occ_cat :

"0" 為低技能

"1" 為技術中等

"2" 為高技能

mar_cat :

"0" 表示未婚

"1" 表示已婚

"2" 表示其他(分居/喪偶)

nat_cat :

"0" 為發展落後地區

"1" 為新興經濟體

"2" 為已開發國家

rel_cat :

"0" 為無家庭關係

"1" 為旁系

"2" 為直系

資料編碼

work_cat :

"0"為無薪或其他工作
"1"為私人或自營企業
"2"為政府相關企業

edu_cat :

"0"為低學歷(國小以下)
"1"為中學歷(國高中)
"2"為高學歷(大學以上)

edun_cat :

"1"為0-5級 表示"教育程度低"
"2"為6-10級 表示"教育程度中"
"3"為11-15級 表示"教育程度中高"
"4"為16-20級 表示"教育程度高"

age :

"1"為18-25歲 表示"青少年"
"2"為26-35歲 表示"青壯年"
"3"為36-65歲 表示"壯年"

race_cat :

"1"為白人
"2"為黑人
"3"為亞裔美國人
"4"為愛斯基摩人
"5"為其他

資料平衡



模型架構前處理

```
# =====
# 4 8:2 切分資料
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("訓練集原始大小:", len(y_train))
print("各類別數量:\n", y_train.value_counts())
```

```
# =====
# 6 儲存訓練集與測試集
# =====
train_data = pd.concat([X_train_res.reset_index(drop=True), y_train_res.reset_index(drop=True)])
test_data = pd.concat([X_test.reset_index(drop=True), y_test.reset_index(drop=True)], axis=1)
pyreadstat.write_sav(train_data, "train_data.sav")
pyreadstat.write_sav(test_data, "test_data.sav")
print("檔案已成功產生: train_data.sav & test_data.sav")
```

```
# =====
# 5 訓練集套用 SMOTE 平衡 (維持總數)
# =====
N_train = len(y_train)
target_per_class = N_train // 2 # 每類筆數目標

# 少數類用 SMOTE 補足
minority_class = y_train.value_counts().idxmin()
smote = SMOTE(sampling_strategy={minority_class: target_per_class}, random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 多數類超過 target → 隨機抽樣
majority_class = y_train.value_counts().idxmax()
X_major = X_train_res[y_train_res==majority_class]
y_major = y_train_res[y_train_res==majority_class]
if len(y_major) > target_per_class:
    X_major_down, y_major_down = resample(
        X_major, y_major, replace=False, n_samples=target_per_class, random_state=42
    )
X_minor = X_train_res[y_train_res!=majority_class]
y_minor = y_train_res[y_train_res!=majority_class]
# 合併
X_train_res = pd.concat([X_major_down, X_minor], axis=0).reset_index(drop=True)
y_train_res = pd.concat([y_major_down, y_minor], axis=0).reset_index(drop=True)

print("平衡後訓練集大小:", len(y_train_res))
print("平衡後各類別數量:\n", y_train_res.value_counts())
```

模型架構前處理

===== 原始資料總覽 =====

原始資料量: (28852, 26)

原始收入分布:

income_bin

0.0 21676

1.0 7176

Name: count, dtype: int64

訓練集大小: (23081, 13)

訓練集收入分布:

income_bin

0.0 17340

1.0 5741

Name: count, dtype: int64

測試集大小: (5771, 13)

測試集收入分布:

income_bin

0.0 4336

1.0 1435

Name: count, dtype: int64

===== 平衡後訓練集總覽 =====

平衡後訓練集大小: (23080, 13)

平衡後訓練集分布:

income_bin

0.0 11540

1.0 11540

Name: count, dtype: int64

視覺化分析



熱力圖程式碼

```
# Step 1. 上傳 .sav 檔案
from google.colab import files
upload = files.upload()

# Step 2. 安裝並載入套件
!pip install pyreadstat

import pyreadstat
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 3. 讀取 .sav 檔案
file_name = list(upload.keys())[0]
df, meta = pyreadstat.read_sav(file_name)

print("資料筆數與欄位形狀:", df.shape)
display(df.head())
```

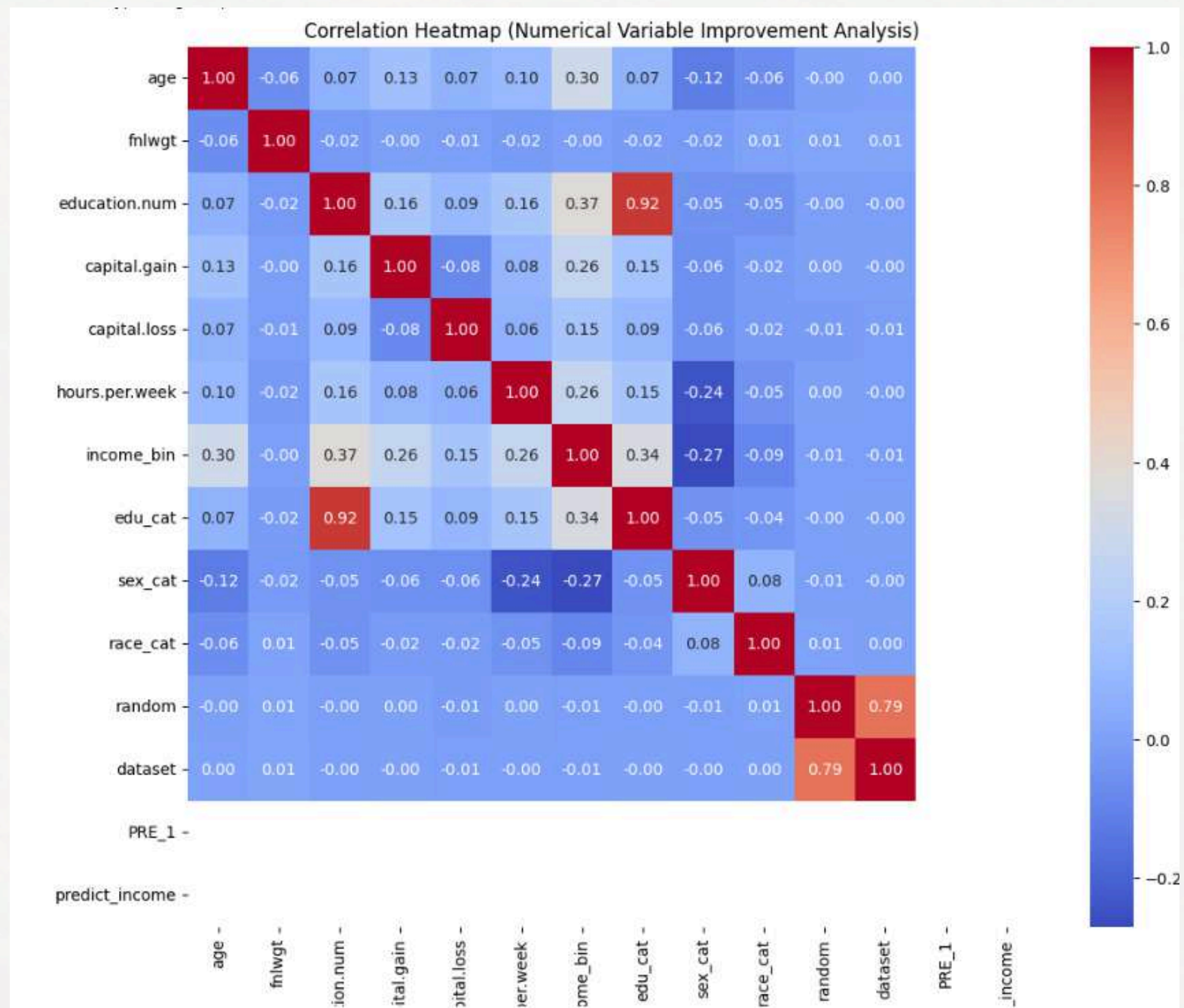
```
# Step 4. 選取原始數值型欄位（適合相關係數）
numeric_cols = [
    'age',
    'fnlwgt',
    'education.num',
    'capital.gain',
    'capital.loss',
    'hours.per.week',
    'income_bin'
]

print("可用於相關係數的欄位:")
print(numeric_cols)

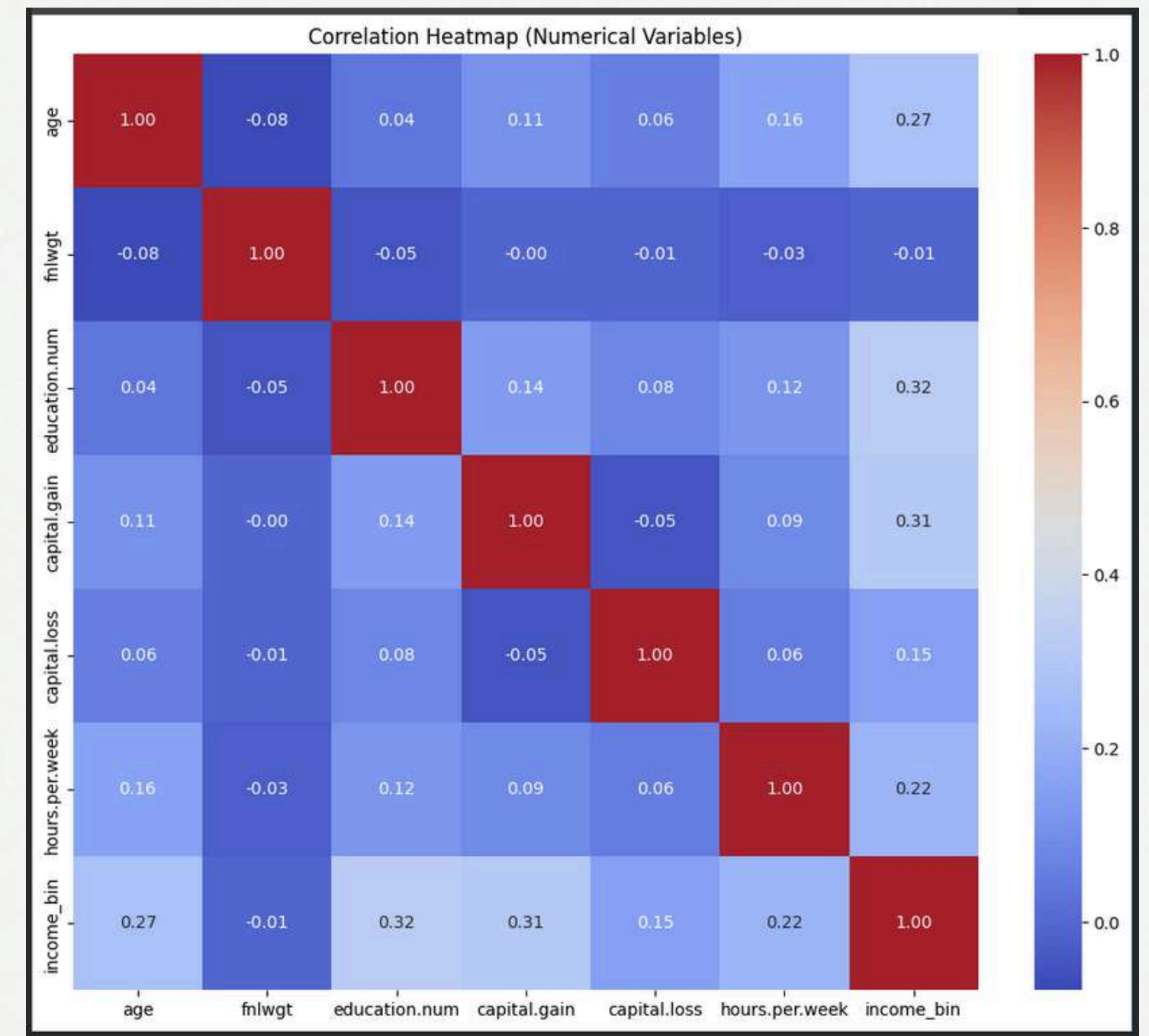
# Step 5. 計算相關矩陣
corr = df[numeric_cols].corr()
display(corr)

# Step 6. 繪製熱力圖
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Numerical Variables)")
plt.show()
```


熱力圖



含編碼後類別型資料熱力圖



只有連續型資料與收入比較

視覺化分析程式碼

```
#0. 上傳檔案 (一定要先跑這段)
from google.colab import files
uploaded = files.upload()

# 1. 載入套件
import pandas as pd
import plotly.express as px
import os
!pip install pyreadstat # 安裝讀取 SPSS .sav 檔案的套件
import pyreadstat # 讀取 SPSS .sav 檔案

# 2. 自動抓剛上傳的 SAV 檔名
sav_files = [f for f in os.listdir() if f.endswith(".sav")]

if len(sav_files) == 0:
    raise FileNotFoundError("目前資料夾沒有任何 SAV 檔案")
elif len(sav_files) > 1:
    print("找到多個 SAV, 使用第一個:", sav_files[0])

file_name = sav_files[0]
print("使用檔案:", file_name)

# 3. 讀取 SPSS (.sav) 檔案
df, meta = pyreadstat.read_sav(file_name)

# 4. 檢視資料
df.head()

# 3. 讀取 SPSS (.sav) 資料
import pyreadstat
import plotly.express as px

df, meta = pyreadstat.read_sav(file_name)
```

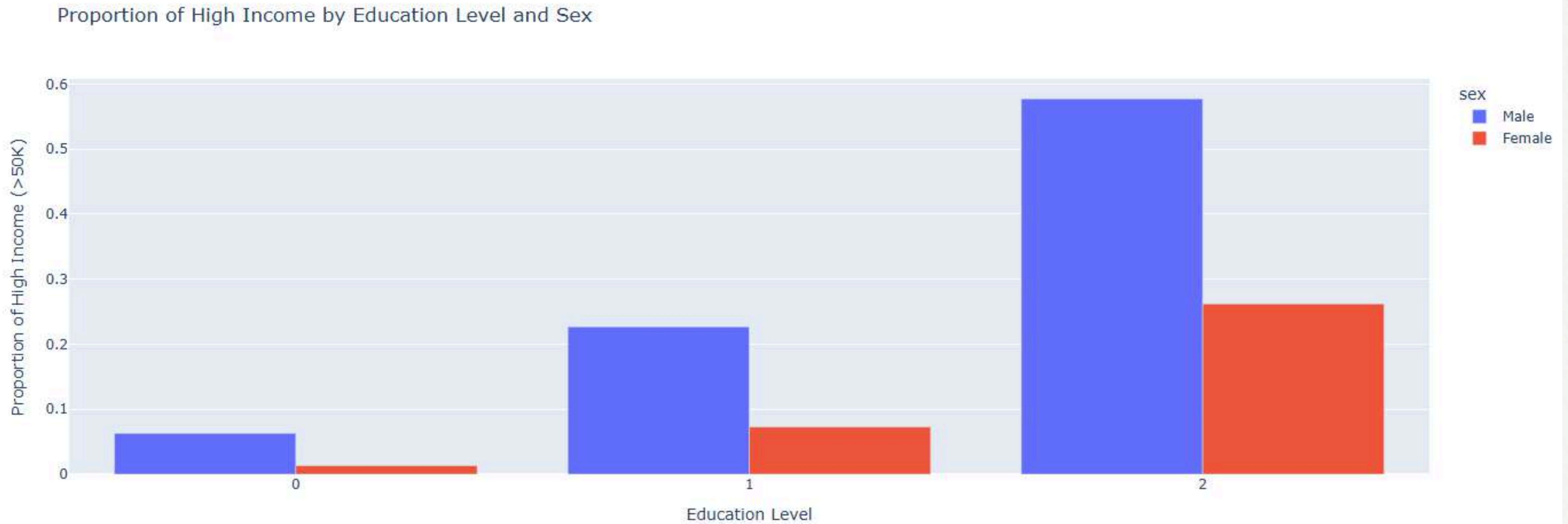
```
# 4. 建立二元收入變數
# 假設 income_new: 0 = <=50K, 1 = >50K
# df["income_bin"] = (df["income_new"] == 2).astype(int) # 此行因為 income_new 不存在而導致錯誤, 且 income_bin 已存在於 df 中, 故移除

# 5. 分組計算高收入比例
summary = (
    df
    .groupby(["edu_cat", "sex_cat"], dropna=False)["income_bin"]
    .mean()
    .reset_index()
)

# 6. 性別轉成文字 (SPSS 常見 1/2 編碼)
summary["sex"] = summary["sex_cat"].map({
    1: "Male",
    2: "Female"
})

# 7. 畫群組長條圖
fig = px.bar(
    summary,
    x="edu_cat",
    y="income_bin",
    color="sex",
    barmode="group",
    labels={
        "edu_cat": "Education Level",
        "income_bin": "Proportion of High Income (>50K)"
    },
    title="Proportion of High Income by Education Level and Sex"
)

fig.show()
```



視覺化分析程式碼

```
#0. 上傳檔案 (一定要先跑這段)
from google.colab import files
uploaded = files.upload()

# 1. 載入套件
import pandas as pd
import plotly.express as px
import os
!pip install pyreadstat # 安裝讀取 SPSS .sav 檔案的套件
import pyreadstat # 讀取 SPSS .sav 檔案

# 2. 自動抓剛上傳的 SAV 檔名
sav_files = [f for f in os.listdir() if f.endswith(".sav")]

if len(sav_files) == 0:
    raise FileNotFoundError("目前資料夾沒有任何 SAV 檔案")
elif len(sav_files) > 1:
    print("找到多個 SAV, 使用第一個:", sav_files[0])

file_name = sav_files[0]
print("使用檔案:", file_name)

# 3. 讀取 SPSS (.sav) 檔案
df, meta = pyreadstat.read_sav(file_name)

# 4. 檢視資料
df.head()

# 3. 讀取 SPSS (.sav) 資料
import pyreadstat
import plotly.express as px

df, meta = pyreadstat.read_sav(file_name)
```

```
# 3. 讀取 SPSS (.sav) 資料
import pyreadstat
import plotly.express as px
import pandas as pd

df, meta = pyreadstat.read_sav(file_name)

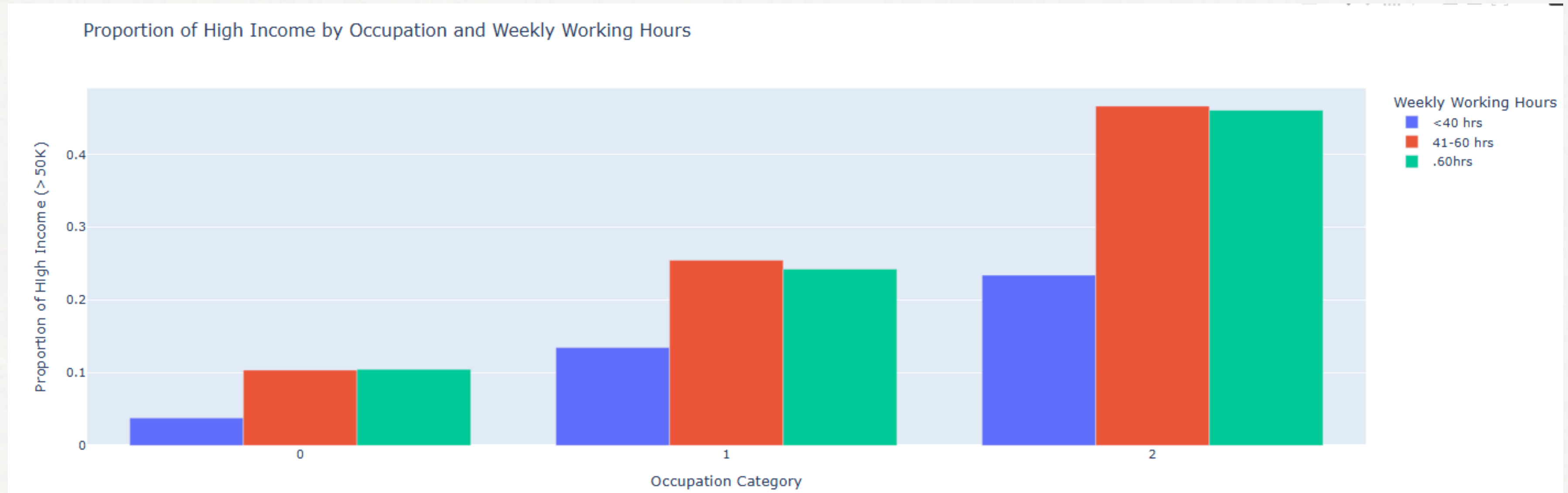
# 4. 確認 income_bin 已存在 (1 = 高收入, 0 = 低收入)
# 不重新建立, 直接使用

# 5. 將每週工時分組 (避免每個數值都變成一根 bar)
df["hours_group"] = pd.cut(
    df["hours_per_week"],
    bins=[0, 40, 60, 100],
    labels=["≤40 hrs", "41-60 hrs", ">60 hrs"]
)

# 6. 分組計算高收入比例
summary = (
    df
    .groupby(["occ_cat", "hours_group"], dropna=False)["income_bin"]
    .mean()
    .reset_index()
)

# 7. 畫群組長條圖
fig = px.bar(
    summary,
    x="occ_cat",
    y="income_bin",
    color="hours_group",
    barmode="group",
    labels={
        "occ_cat": "Occupation Category",
        "income_bin": "Proportion of High Income (>50K)",
        "hours_group": "Weekly Working Hours"
    },
    title="Proportion of High Income by Occupation and Weekly Working Hours"
)

fig.show()
```



問題假說





探討成人人口高收入 預測與關鍵特徵分析

實驗設計



切分資料集

統計量

income_bin

N	有效	28852
	遺漏	0
平均數		.2487
中位數		.0000

income_bin

		次數分配表	百分比	有效百分比	累積百分比
有效	<=50K	21676	75.1	75.1	75.1
	>50K	7176	24.9	24.9	100.0
	總計	28852	100.0	100.0	

一 預測模型

隨機森林

決策樹

羅吉斯回歸

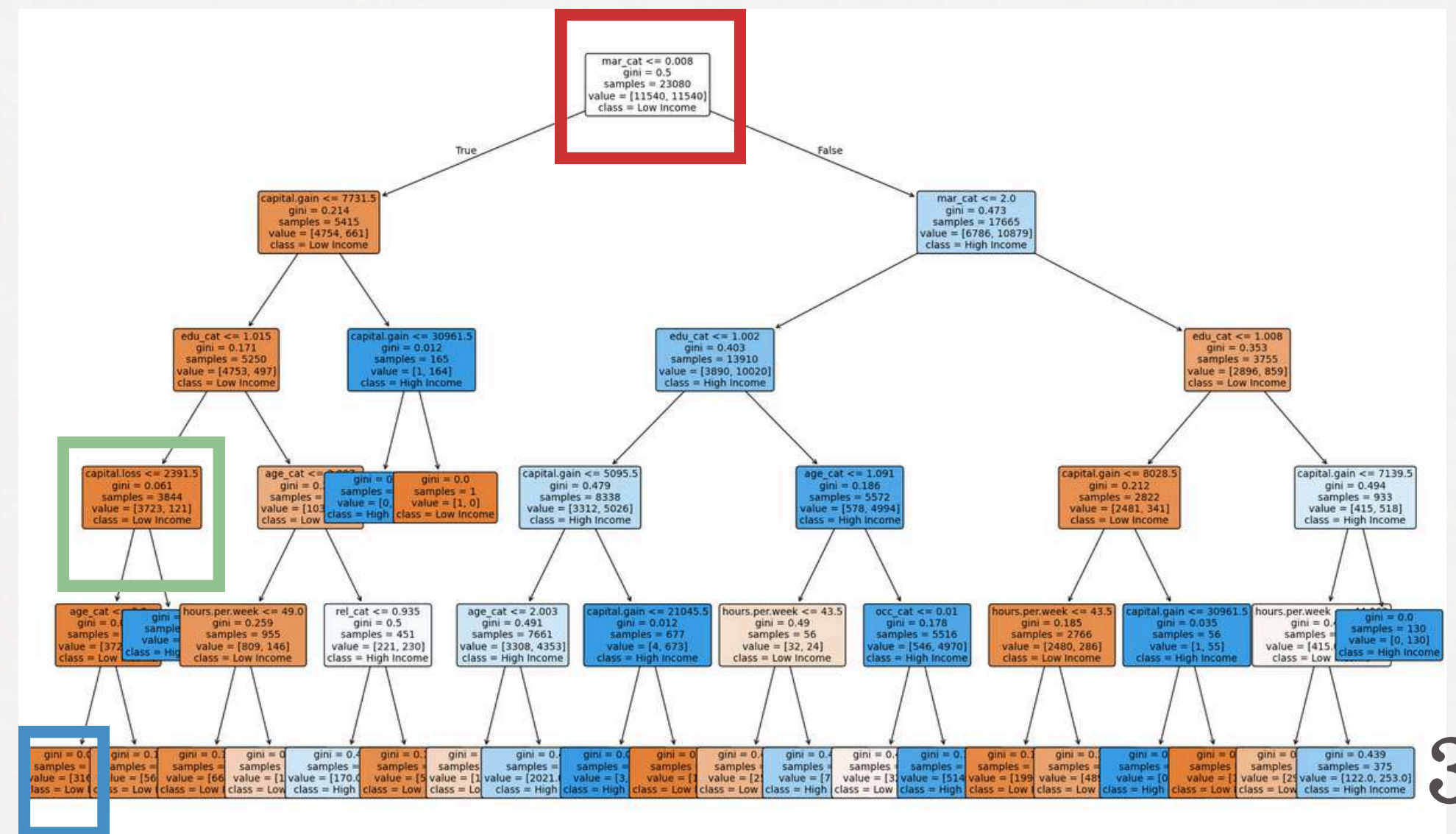
交叉驗證方式 → 五折驗證

模型訓練



決策樹-模型介紹

- 根節點：代表模型進行的第一個判斷條件，通常是最能區分資料的變數
- 內部節點：每個內部節點都是根據一個特徵對資料進行分割的判斷條件
- 葉節點：代表最終的分類結果



一 決策樹-超參數

抽資料方式	80/20法則
最大樹深	4
樹葉節點 (每一個分支的最少樣本數)	20
亂數種子	42

決策樹-模型建構程式碼

```
# =====
# 1 套件
# =====
!pip install -q pyreadstat shap
import pandas as pd
import numpy as np
import pyreadstat
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    roc_curve, roc_auc_score,
    classification_report
)

# =====
# 2 讀取資料
# =====
train_df, _ = pyreadstat.read_sav("train_balanced.sav")
test_df, _ = pyreadstat.read_sav("test.sav")

feature_cols = [
    "work_cat", "edu_cat", "mar_cat", "occ_cat", "rel_cat",
    "race_cat", "sex_cat", "nat_cat", "edun_cat", "age_cat",
    "capital.gain", "capital.loss", "hours.per.week"
]

X_train = train_df[feature_cols]
y_train = train_df["income_bin"]

X_test = test_df[feature_cols]
y_test = test_df["income_bin"]
```

```
# =====
# 3 模型 & CV
# =====
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_leaf=20,
    random_state=42
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# =====
# 4 儲存容器 (五折)
# =====
cm_train_list, cm_test_list = [], []
acc_train_list, acc_test_list = [], []

roc_train_list, roc_test_list = [], []
auc_train_list, auc_test_list = [], []

importance_list = []
report_list_train, report_list_test = [], []
```

決策樹-模型建構程式碼

```
# =====
# 5 五折交叉驗證 (每折: Train/Test CM + Train/Test ROC)
# =====
for fold, (train_idx, val_idx) in enumerate(cv.split(X_train, y_train), start=1):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    model.fit(X_tr, y_tr)

    # ===== 預測: Train / Test
    y_tr_pred = model.predict(X_tr)
    y_test_pred = model.predict(X_test)

    # ===== Train/Test accuracy (存起來)
    acc_train = accuracy_score(y_tr, y_tr_pred)
    acc_test = accuracy_score(y_test, y_test_pred)
    acc_train_list.append(acc_train)
    acc_test_list.append(acc_test)

    # =====
    # (A) Confusion Matrix (每折: Train + Test 同一張)
    # =====
    cm_tr = confusion_matrix(y_tr, y_tr_pred)
    cm_te = confusion_matrix(y_test, y_test_pred)

    cm_train_list.append(cm_tr)
    cm_test_list.append(cm_te)

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    sns.heatmap(cm_tr, annot=True, fmt="d", cmap="Oranges")
    plt.title(f"Fold {fold} Train CM")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.subplot(1, 2, 2)
    sns.heatmap(cm_te, annot=True, fmt="d", cmap="Blues")
    plt.title(f"Fold {fold} Test CM")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.tight_layout()
    plt.show()
```

```
# =====
# (B) ROC (每折: Train + Test 同一張)
# =====
y_tr_prob = model.predict_proba(X_tr)[:, 1]
y_test_prob = model.predict_proba(X_test)[:, 1]

fpr_tr, tpr_tr, _ = roc_curve(y_tr, y_tr_prob)
fpr_te, tpr_te, _ = roc_curve(y_test, y_test_prob)

roc_train_list.append((fpr_tr, tpr_tr))
roc_test_list.append((fpr_te, tpr_te))

auc_tr = roc_auc_score(y_tr, y_tr_prob)
auc_te = roc_auc_score(y_test, y_test_prob)
auc_train_list.append(auc_tr)
auc_test_list.append(auc_te)

plt.figure(figsize=(7, 6))
plt.plot(fpr_tr, tpr_tr, label=f"Train AUC={auc_tr:.3f}")
plt.plot(fpr_te, tpr_te, label=f"Test AUC={auc_te:.3f}")
plt.plot([0, 1], [0, 1], '--', color='gray')
plt.title(f"Fold {fold} ROC (Train vs Test)")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()
plt.tight_layout()
plt.show()

# ===== 特徵重要性 (折內存起來做平均 ± std)
importance_list.append(model.feature_importances_)

# ===== 分類報告 (Train 用 X_tr, Test 用同一份 X_test)
report_list_train.append(classification_report(y_tr, y_tr_pred, output_dict=True))
report_list_test.append(classification_report(y_test, y_test_pred, output_dict=True))
```


決策樹-模型建構程式碼

```
# =====
# 6 平均混淆矩陣 (Train / Test)
# =====

cm_train_mean = np.mean(np.array(cm_train_list), axis=0)
cm_test_mean  = np.mean(np.array(cm_test_list), axis=0)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
sns.heatmap(cm_train_mean, annot=True, fmt=".1f", cmap="Oranges")
plt.title("Mean Train Confusion Matrix (5-Fold)")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.subplot(1,2,2)
sns.heatmap(cm_test_mean, annot=True, fmt=".1f", cmap="Blues")
plt.title("Mean Test Confusion Matrix (5-Fold)")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.tight_layout()
plt.show()
```

```
# =====
# 7 Mean ROC (Train + Test 同一張, 含 AUC 平均±標準差)
# =====
mean_fpr = np.linspace(0, 1, 200)

tpr_tr_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_train_list])
tpr_te_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_test_list])

mean_tpr_tr = tpr_tr_interp.mean(axis=0)
std_tpr_tr  = tpr_tr_interp.std(axis=0)

mean_tpr_te = tpr_te_interp.mean(axis=0)
std_tpr_te  = tpr_te_interp.std(axis=0)

mean_auc_tr = np.mean(auc_train_list)
std_auc_tr  = np.std(auc_train_list)

mean_auc_te = np.mean(auc_test_list)
std_auc_te  = np.std(auc_test_list)

plt.figure(figsize=(8,6))
plt.plot(mean_fpr, mean_tpr_tr, label=f"Mean Train ROC (AUC={mean_auc_tr:.3f} ± {std_auc_tr:.3f})")
plt.fill_between(mean_fpr, mean_tpr_tr-std_tpr_tr, mean_tpr_tr+std_tpr_tr, alpha=0.2)

plt.plot(mean_fpr, mean_tpr_te, label=f"Mean Test ROC (AUC={mean_auc_te:.3f} ± {std_auc_te:.3f})")
plt.fill_between(mean_fpr, mean_tpr_te-std_tpr_te, mean_tpr_te+std_tpr_te, alpha=0.2)

plt.plot([0,1],[0,1], '--', color='gray')
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("Mean ROC over 5 Folds (Train vs Test)")
plt.legend()
plt.tight_layout()
plt.show()
```

決策樹-模型建構程式碼

```
# =====
# 8 平均特徵重要性 (五折平均 ± 標準差) + 圖
# =====
importance_arr = np.array(importance_list)
feat_df = pd.DataFrame({
    "feature": feature_cols,
    "mean_importance": importance_arr.mean(axis=0),
    "std_importance": importance_arr.std(axis=0)
}).sort_values("mean_importance", ascending=True)

plt.figure(figsize=(10,6))
plt.barh(
    y=feat_df["feature"],
    width=feat_df["mean_importance"],
    xerr=feat_df["std_importance"].to_numpy(),
    capsize=5
)
plt.xlabel("Mean Feature Importance ± Std")
plt.ylabel("Feature")
plt.title("Feature Importance over 5 folds")
plt.tight_layout()
plt.show()
```

```
# =====
# 9 五折分類報告 (mean / std / mean±std) - 取 positive 類
# =====
def five_fold_classification_stats(reports):
    metrics = ["precision", "recall", "f1-score"]
    sample_keys = reports[0].keys()
    class_keys = [k for k in sample_keys if k not in ["accuracy", "macro avg", "weighted avg"]]
    positive_key = sorted(class_keys)[-1] # 預設取較大的那類當 positive

    mean_dict, std_dict, mean_std_dict = {}, {}, {}
    for m in metrics:
        values = np.array([r[positive_key][m] for r in reports])
        mean_dict[m] = values.mean()
        std_dict[m] = values.std()
        mean_std_dict[m] = f"{values.mean():.6f} ± {values.std():.6f}"

    support = np.mean([r[positive_key]["support"] for r in reports])

    mean_df = pd.DataFrame(mean_dict, index=["positive"])
    std_df = pd.DataFrame(std_dict, index=["positive"])
    mean_std_df = pd.DataFrame(mean_std_dict, index=["positive"])

    mean_df["support"] = support
    std_df["support"] = support
    mean_std_df["support"] = support
    return mean_df, std_df, mean_std_df

mean_report, std_report, mean_std_report = five_fold_classification_stats(report_list_test)
```

決策樹-模型建構程式碼

```
# =====
# 10 平均分類報告 (Train / Test)
# =====
def average_classification_report(reports):
    keys = list(reports[0].keys())
    avg_report = {}
    for k in keys:
        if k not in ["accuracy", "macro avg", "weighted avg"]:
            avg_report[k] = {
                "precision": np.mean([r[k]["precision"] for r in reports]),
                "recall": np.mean([r[k]["recall"] for r in reports]),
                "f1-score": np.mean([r[k]["f1-score"] for r in reports]),
                "support": np.mean([r[k]["support"] for r in reports]),
            }
    return avg_report

avg_report_train = average_classification_report(report_list_train)
avg_report_test = average_classification_report(report_list_test)

# =====
# 11 訓練整個訓練集並畫決策樹
# =====
model.fit(X_train, y_train)
plt.figure(figsize=(20, 12))
plot_tree(
    model,
    feature_names=feature_cols,
    class_names=["False", "True"],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.tight_layout()
plt.show()
```

```
# =====
# 12 SHAP 特徵重要性分析 (bar + beeswarm)
# =====
import shap
model.fit(X_train, y_train)

sample_size = min(2000, len(X_test))
X_shap = X_test.sample(n=sample_size, random_state=42)

explainer = shap.Explainer(model, X_train)
sv = explainer(X_shap)

vals = sv.values
shap_pos = vals[:, :, 1] if vals.ndim == 3 else vals

mean_abs_shap = np.abs(shap_pos).mean(axis=0)
shap_importance_df = pd.DataFrame({
    "feature": X_shap.columns,
    "mean_abs_shap": mean_abs_shap
}).sort_values("mean_abs_shap", ascending=False)

plt.figure(figsize=(10, 6))
shap.summary_plot(shap_pos, X_shap, plot_type="bar", show=False)
plt.title("SHAP Feature Importance (bar)")
plt.show()

plt.figure(figsize=(10, 6))
shap.summary_plot(shap_pos, X_shap, show=False)
plt.title("SHAP Summary (beeswarm)")
plt.show()
```


決策樹-模型建構程式碼

```
# =====
# 1 3 文字輸出
# =====
acc_train_arr = np.array(acc_train_list)
acc_test_arr = np.array(acc_test_list)

mean_train_acc = acc_train_arr.mean()
std_train_acc = acc_train_arr.std()

mean_test_acc = acc_test_arr.mean()
std_test_acc = acc_test_arr.std()

def _df_to_block(df, round_n=6):
    """把 DataFrame 轉成像截圖那樣的純文字區塊"""
    return df.round(round_n).to_string()

print(f"Train Accuracy: Mean={mean_train_acc:.4f}, Std={std_train_acc:.4f}")
print(f"Test Accuracy: Mean={mean_test_acc:.4f}, Std={std_test_acc:.4f}\n")

print("==== Average Classification Report - Train ====")
print(_df_to_block(pd.DataFrame(avg_report_train).T))
print()

print("==== Average Classification Report - Test ====")
print(_df_to_block(pd.DataFrame(avg_report_test).T))
print()

print("==== 五折交叉驗證準確率 ====")
for i, acc in enumerate(acc_test_arr, 1):
    print(f"Fold {i}: {acc:.6f}")
print()

print("==== 平均準確率 ====")
print(f"Mean Accuracy: {mean_test_acc:.6f}")
print(f"Std Accuracy : {std_test_acc:.6f}")
print()
```

```
print("==== 五折偏差 (每折 - 平均) ====")
for i, acc in enumerate(acc_test_arr, 1):
    print(f"Fold {i}: {acc - mean_test_acc:+.6f}")
print()
```

```
print("==== 五折交叉驗證「平均分類報告」 ====")
print(_df_to_block(mean_report))
print()
```

```
print("==== 五折交叉驗證「標準差分類報告」 ====")
print(_df_to_block(std_report))
print()
```

```
print("==== 五折交叉驗證「平均 ± 標準差 分類報告」 ====")
print(_df_to_block(mean_std_report))
print()
```

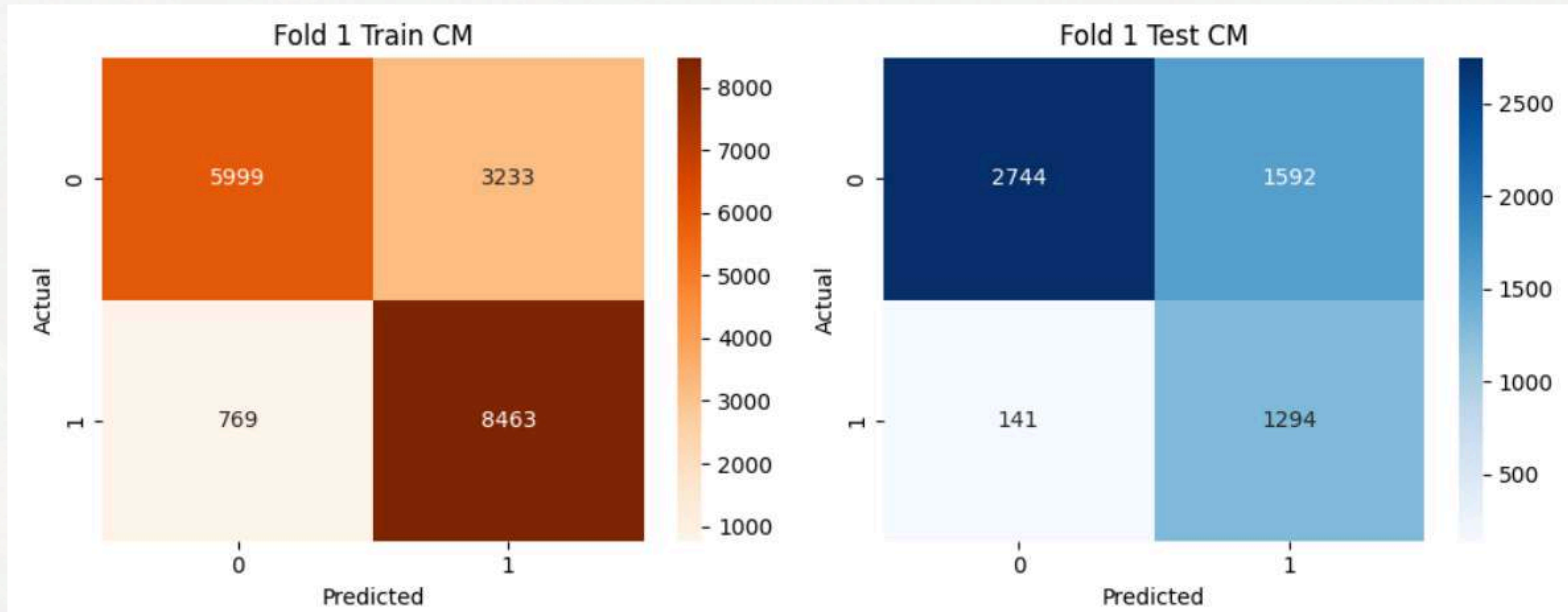
```
print("==== Mean Feature Importance ====")
print(_df_to_block(feats_df, round_n=6))
print()
```

```
print("==== SHAP Feature Importance (mean |SHAP|) ====")
print(_df_to_block(shap_importance_df, round_n=6))
```

決策樹-混淆矩陣-Fold 1

Train Accuracy : 0.7832

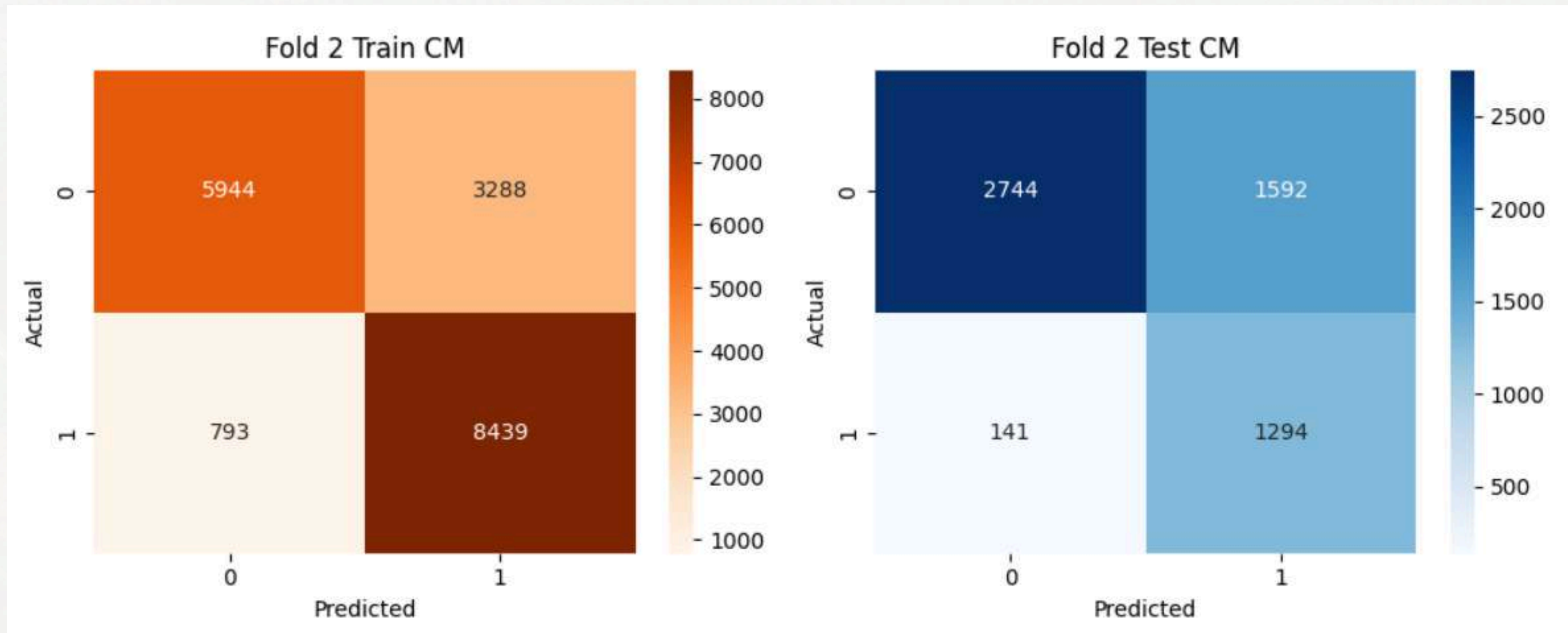
Test Accuracy : 0.6997



決策樹-混淆矩陣-Fold 2

Train Accuracy : 0.7789

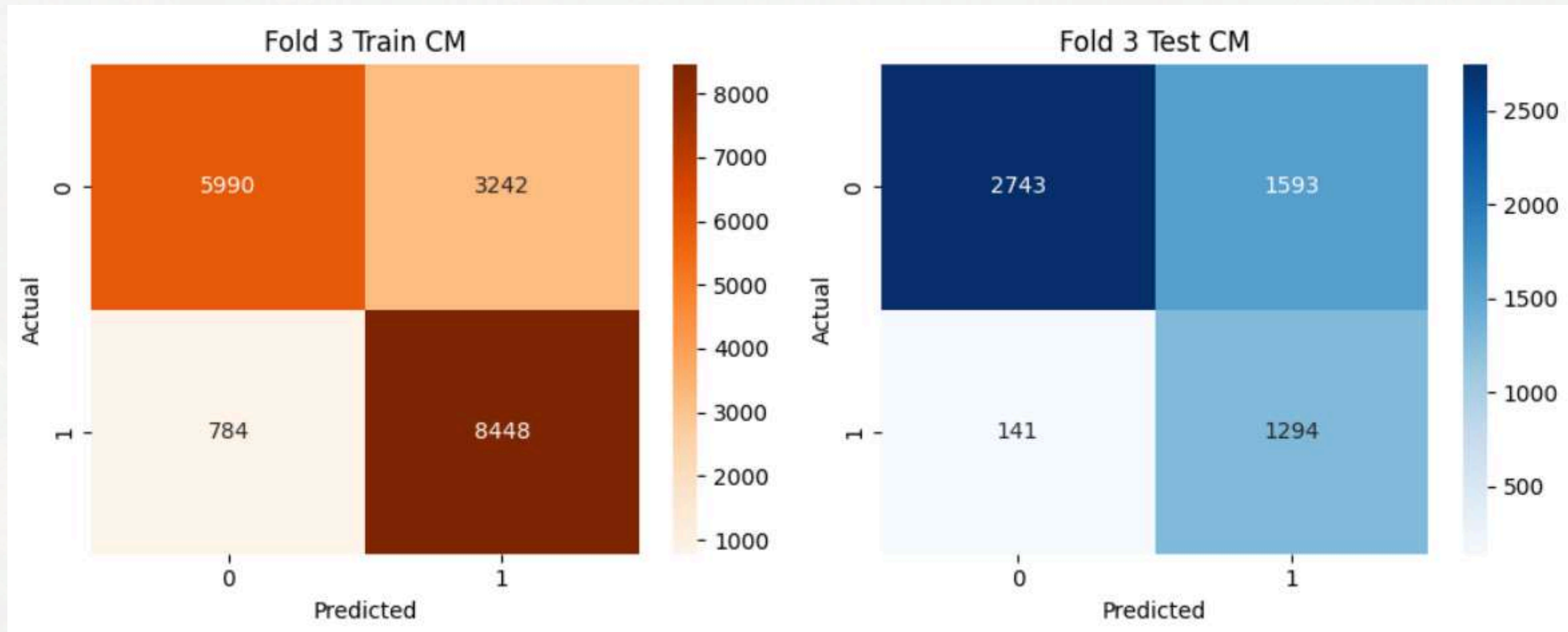
Test Accuracy : 0.6997



決策樹-混淆矩陣-Fold 3

Train Accuracy : 0.7819

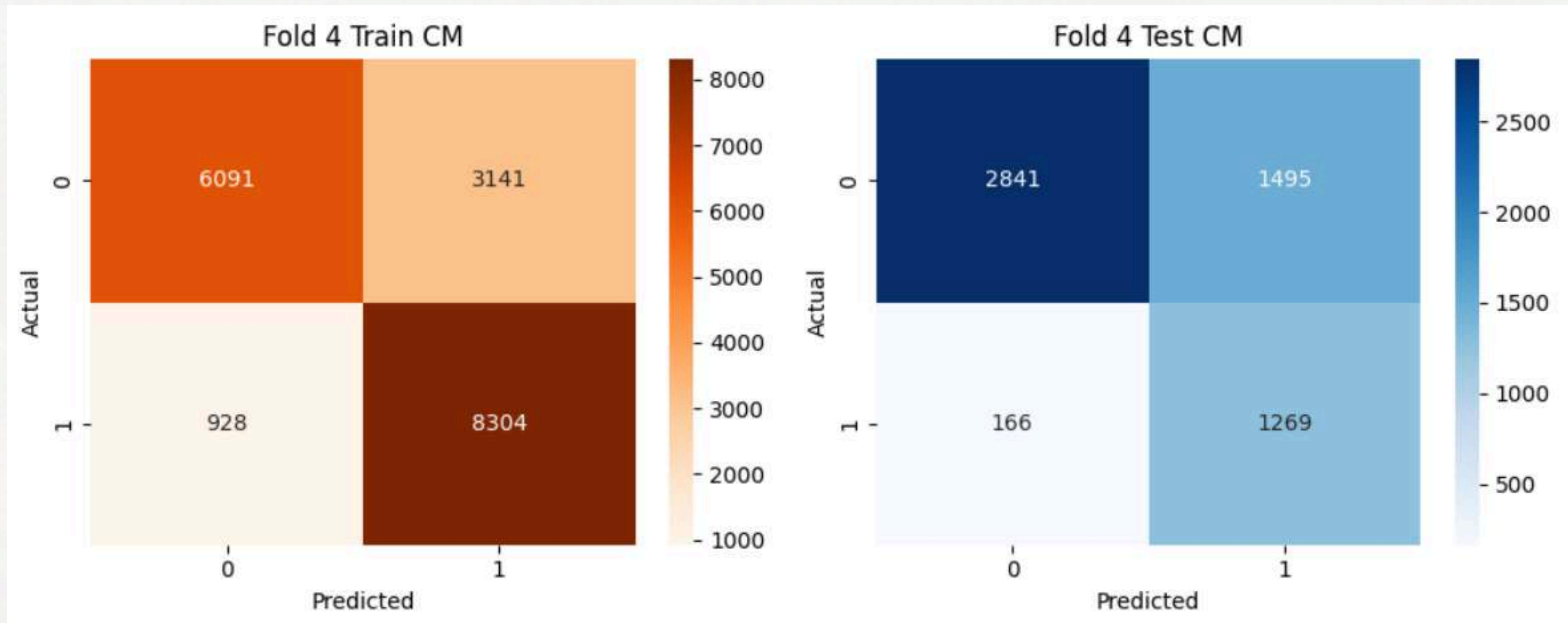
Test Accuracy : 0.6995



決策樹-混淆矩陣-Fold 4

Train Accuracy : 0.7796

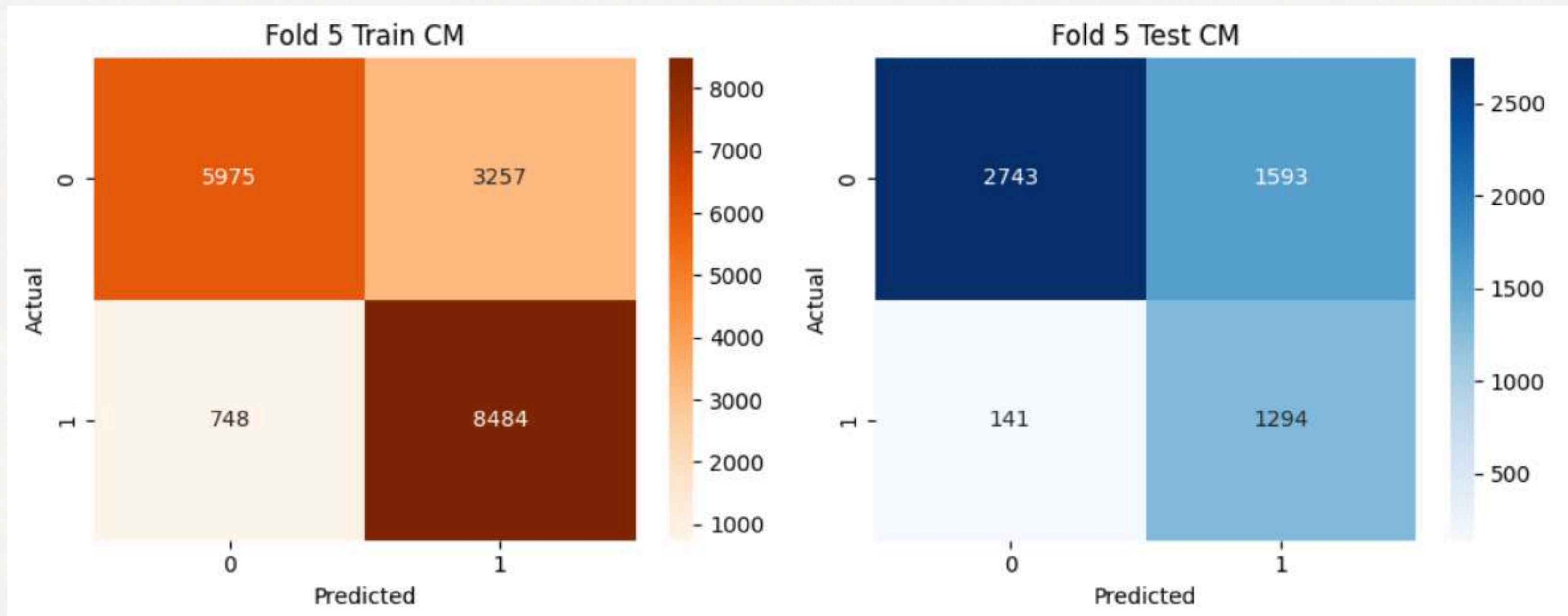
Test Accuracy : 0.7121



決策樹-混淆矩陣-Fold 5

Train Accuracy : 0.7830

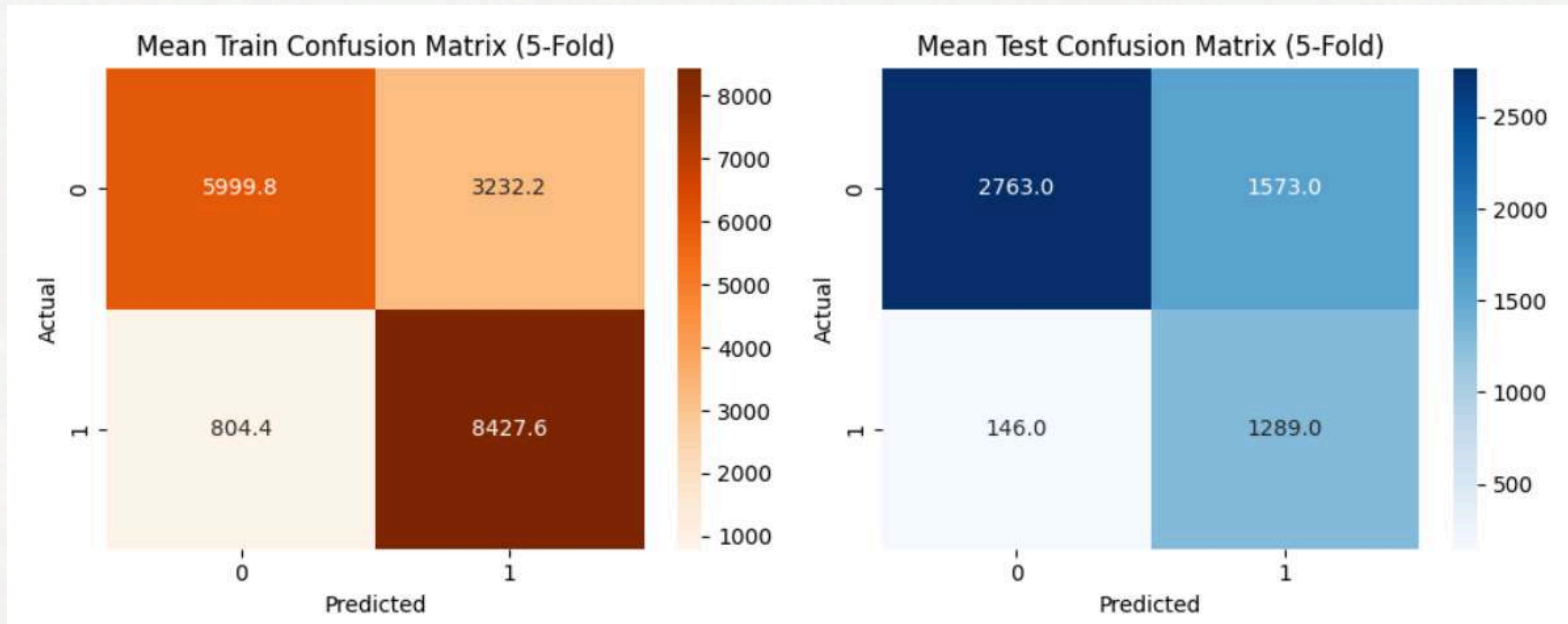
Test Accuracy : 0.6995



決策樹-混淆矩陣-五折平均

Train Accuracy : 0.7814

Test Accuracy : 0.7021



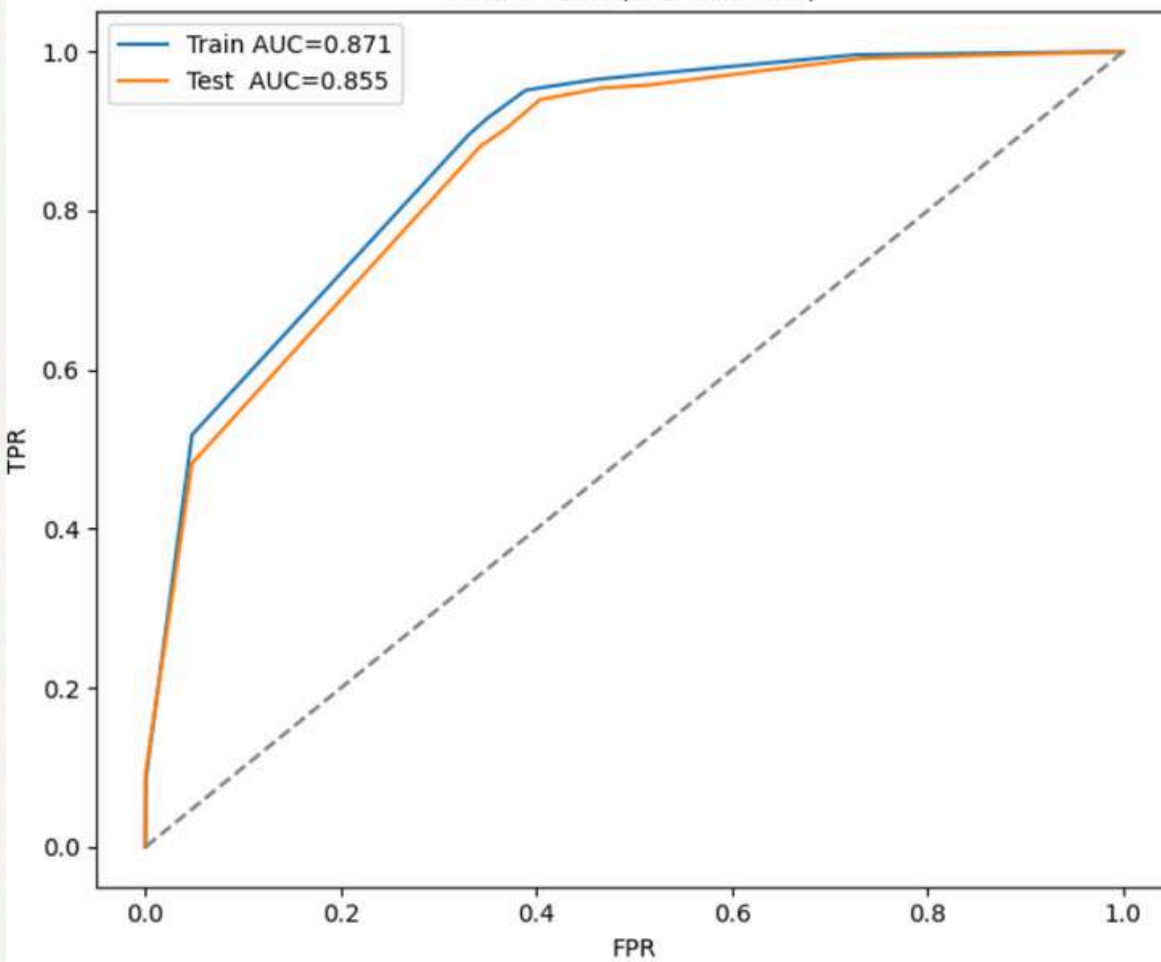
決策樹-ROC曲線

- 定義：AUC=ROC曲線下的面積
- 是一種用來評估分類模型表現的圖形，透過比較真陽性率(TPR)與假陽性率(FPR)，來判斷模型區分正負類的能力。

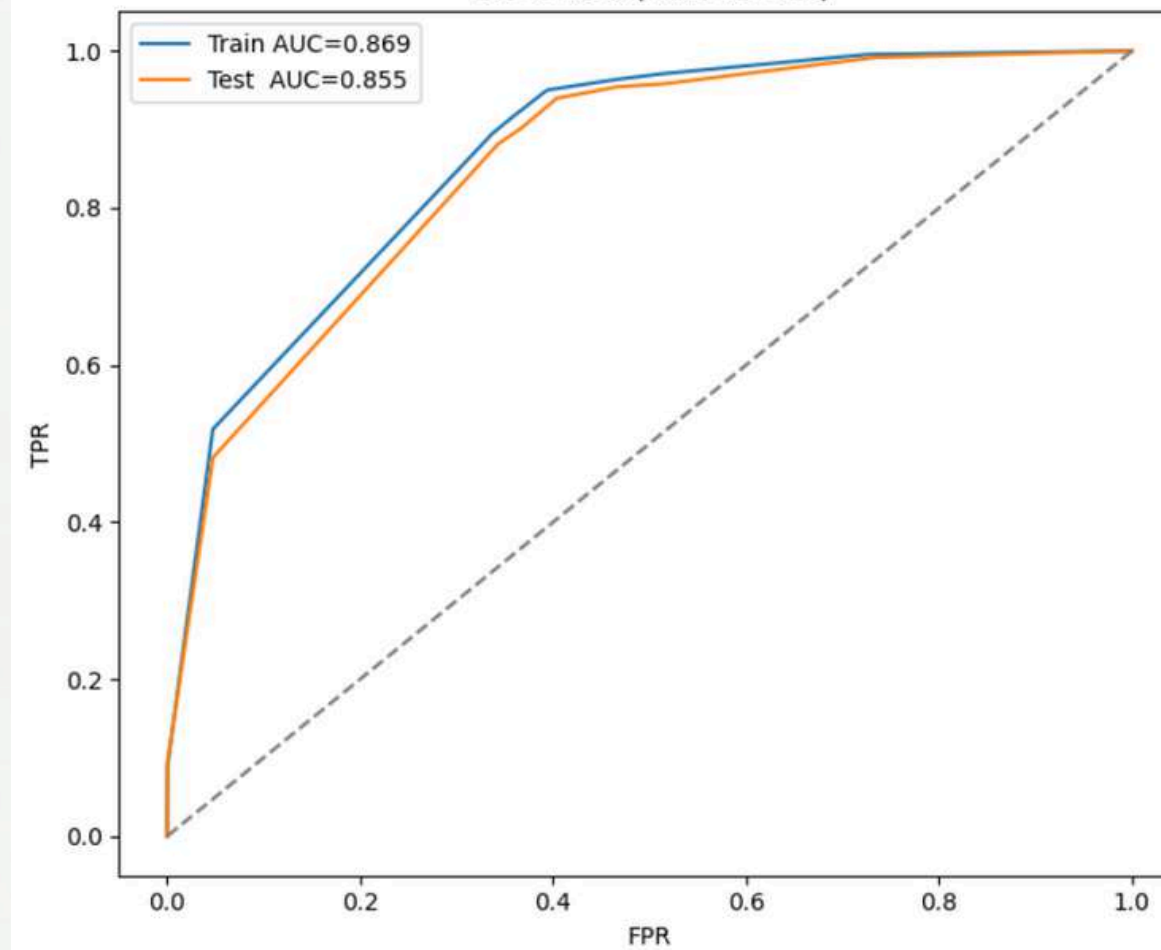
AUC數值	模型程度
< 0.7	模型弱，需改進
0.7 ~ 0.8	一般
0.8 ~ 0.9	很好
0.9以上	非常優秀

決策樹-ROC曲線

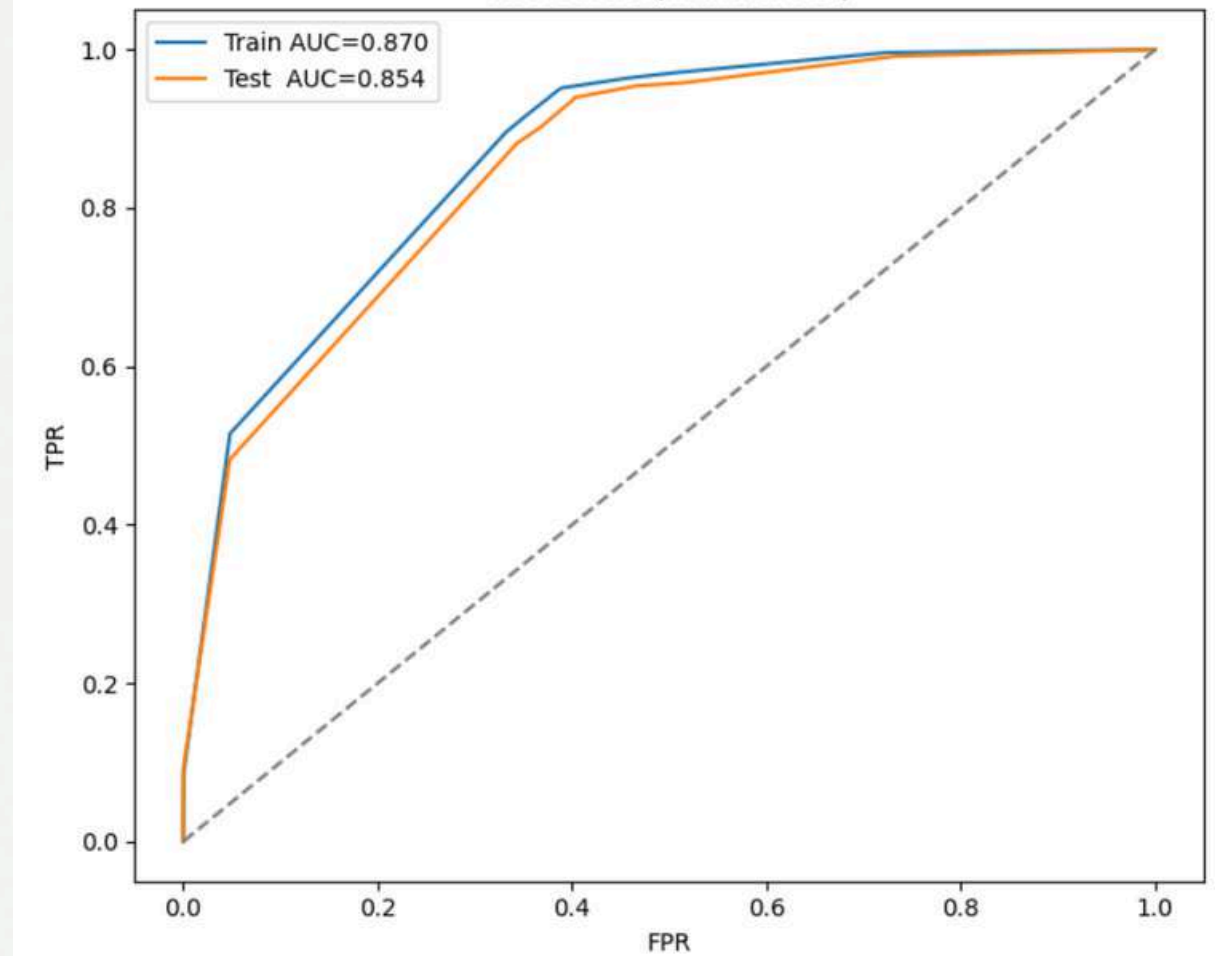
Fold 1 ROC (Train vs Test)



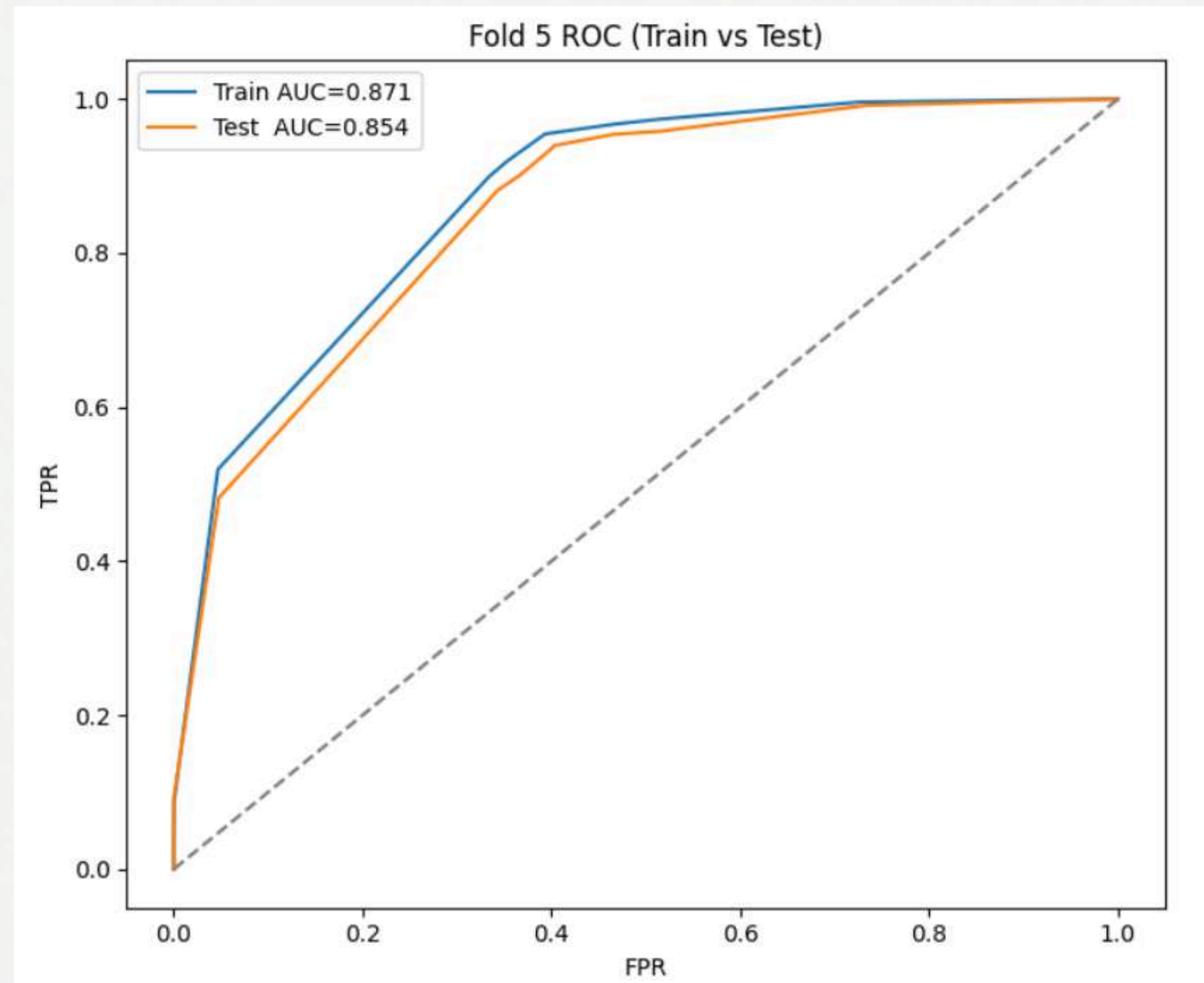
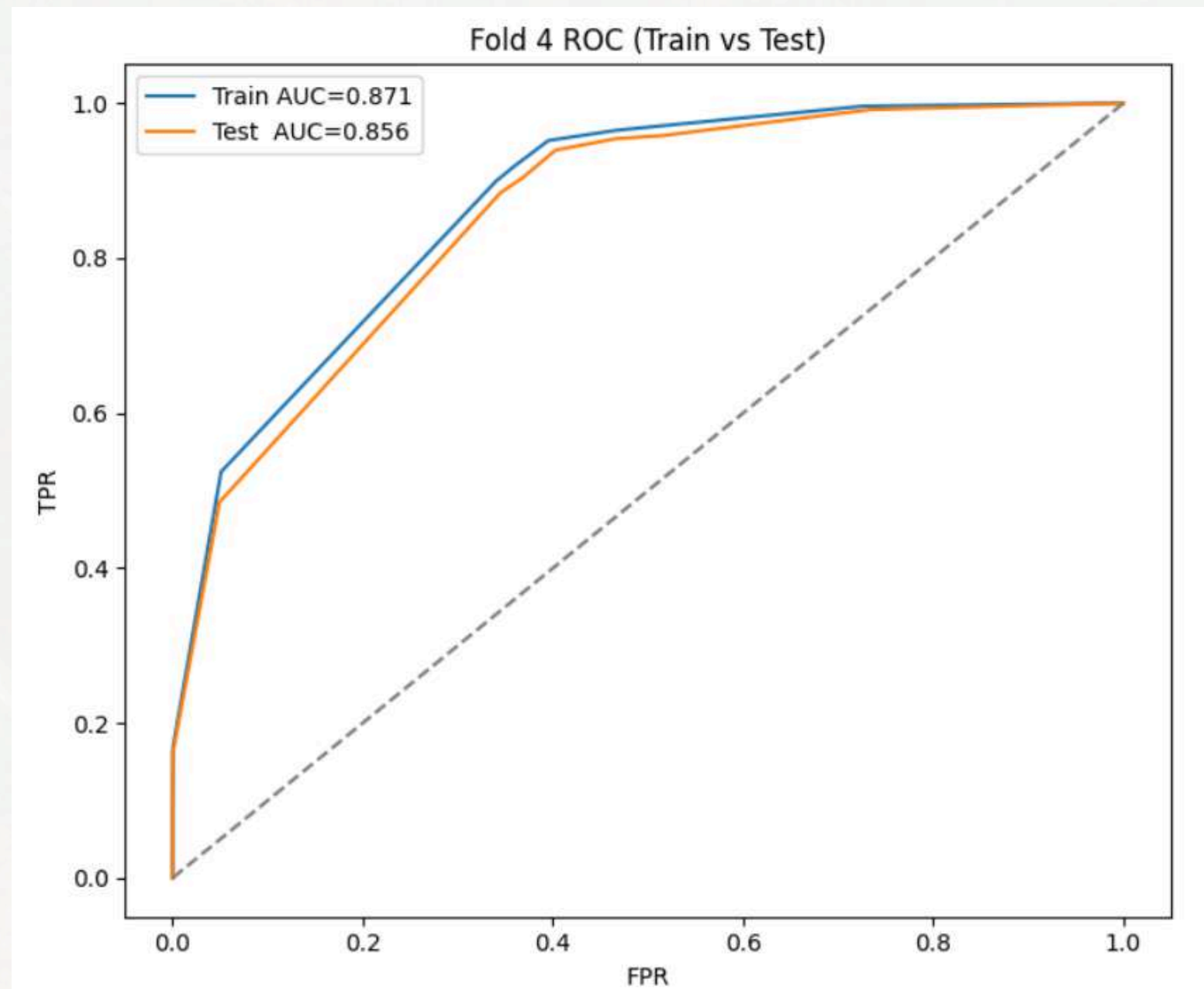
Fold 2 ROC (Train vs Test)



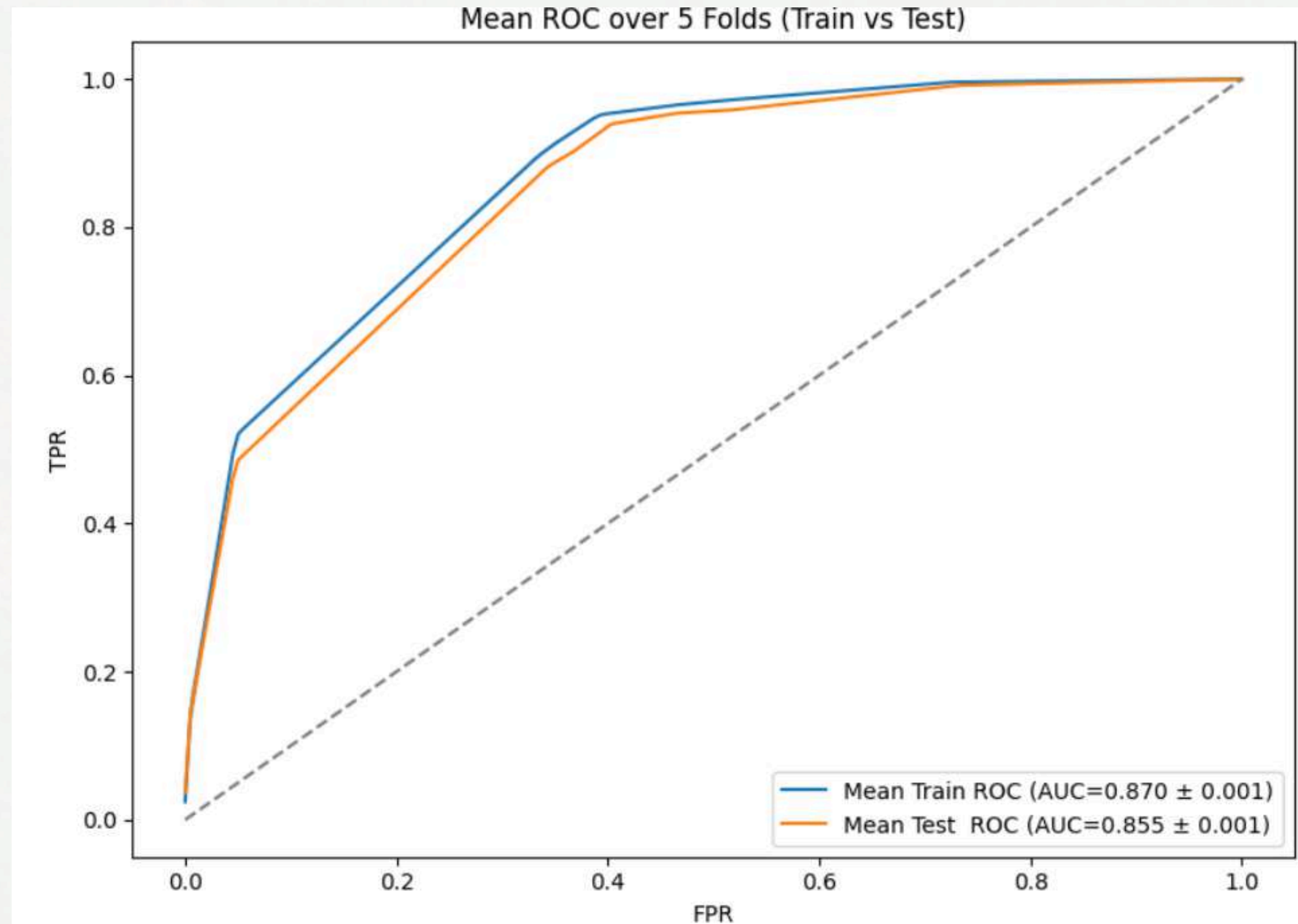
Fold 3 ROC (Train vs Test)



決策樹-ROC曲線



決策樹-ROC曲線-五折平均



決策樹-模型整體效能評估摘要 (五折交叉驗證)

Train Accuracy: Mean=0.7814, Std=0.0018
 Test Accuracy: Mean=0.7021, Std=0.0050

==== Average Classification Report - Train ====

	precision	recall	f1-score	support
0.0	0.881892	0.649892	0.748277	9232.0
1.0	0.722805	0.912868	0.806776	9232.0

==== Average Classification Report - Test ====

	precision	recall	f1-score	support
0.0	0.949854	0.637223	0.762695	4336.0
1.0	0.450458	0.898258	0.599979	1435.0

==== 五折交叉驗證準確率 ====

Fold 1: 0.699705
 Fold 2: 0.699705
 Fold 3: 0.699532
 Fold 4: 0.712182
 Fold 5: 0.699532

==== 平均準確率 ====

Mean Accuracy: 0.702131
 Std Accuracy : 0.005026

==== 五折偏差 (每折 - 平均) ====

Fold 1: -0.002426
 Fold 2: -0.002426
 Fold 3: -0.002599
 Fold 4: +0.010050
 Fold 5: -0.002599

==== 五折交叉驗證「平均分類報告」 ====

	precision	recall	f1-score	support
positive	0.450458	0.898258	0.599979	1435.0

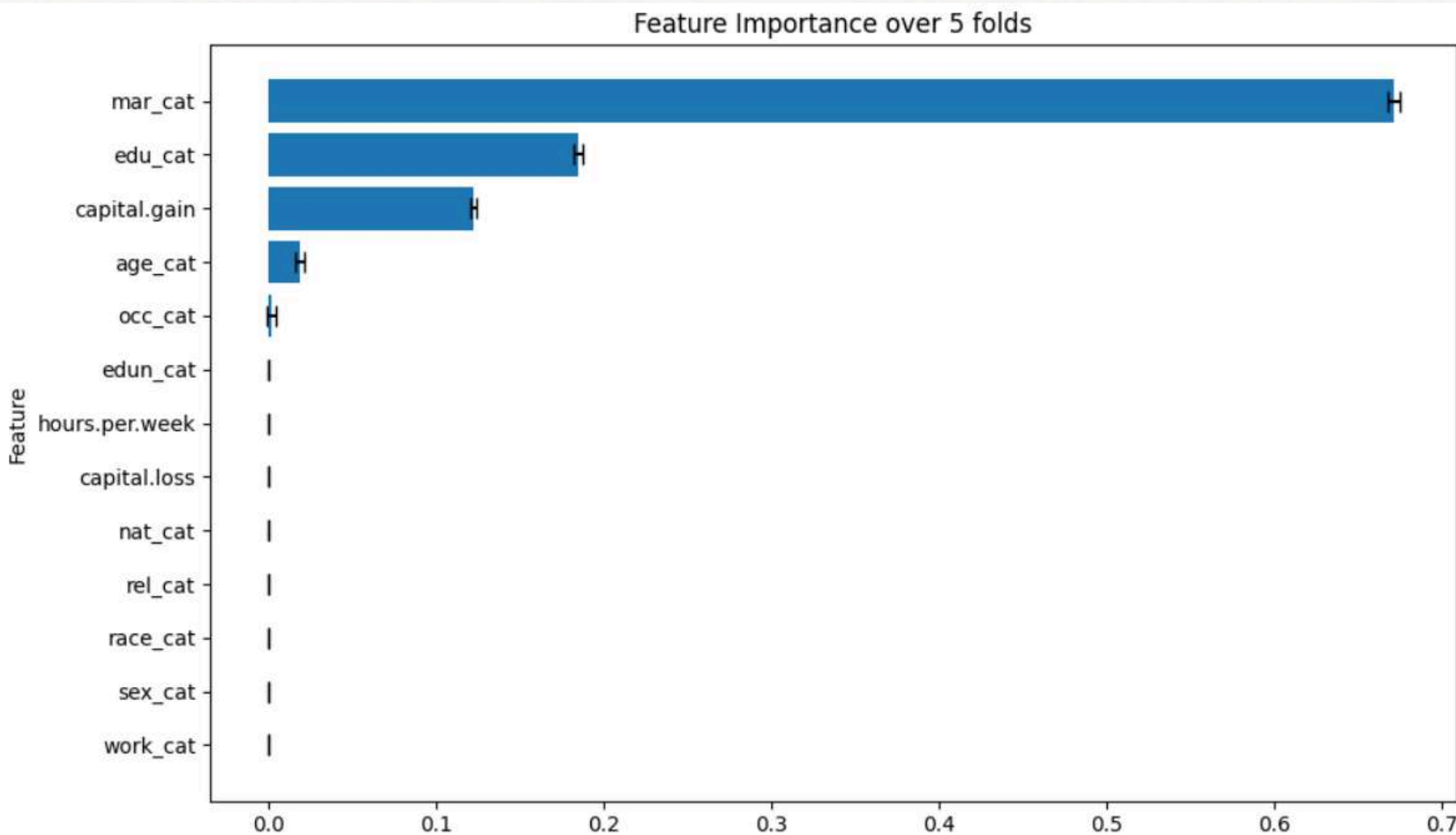
==== 五折交叉驗證「標準差分類報告」 ====

	precision	recall	f1-score	support
positive	0.00433	0.006969	0.002226	1435.0

==== 五折交叉驗證「平均 ± 標準差 分類報告」 ====

	precision	recall	f1-score	support
positive	0.450458 ± 0.004330	0.898258 ± 0.006969	0.599979 ± 0.002226	1435.0

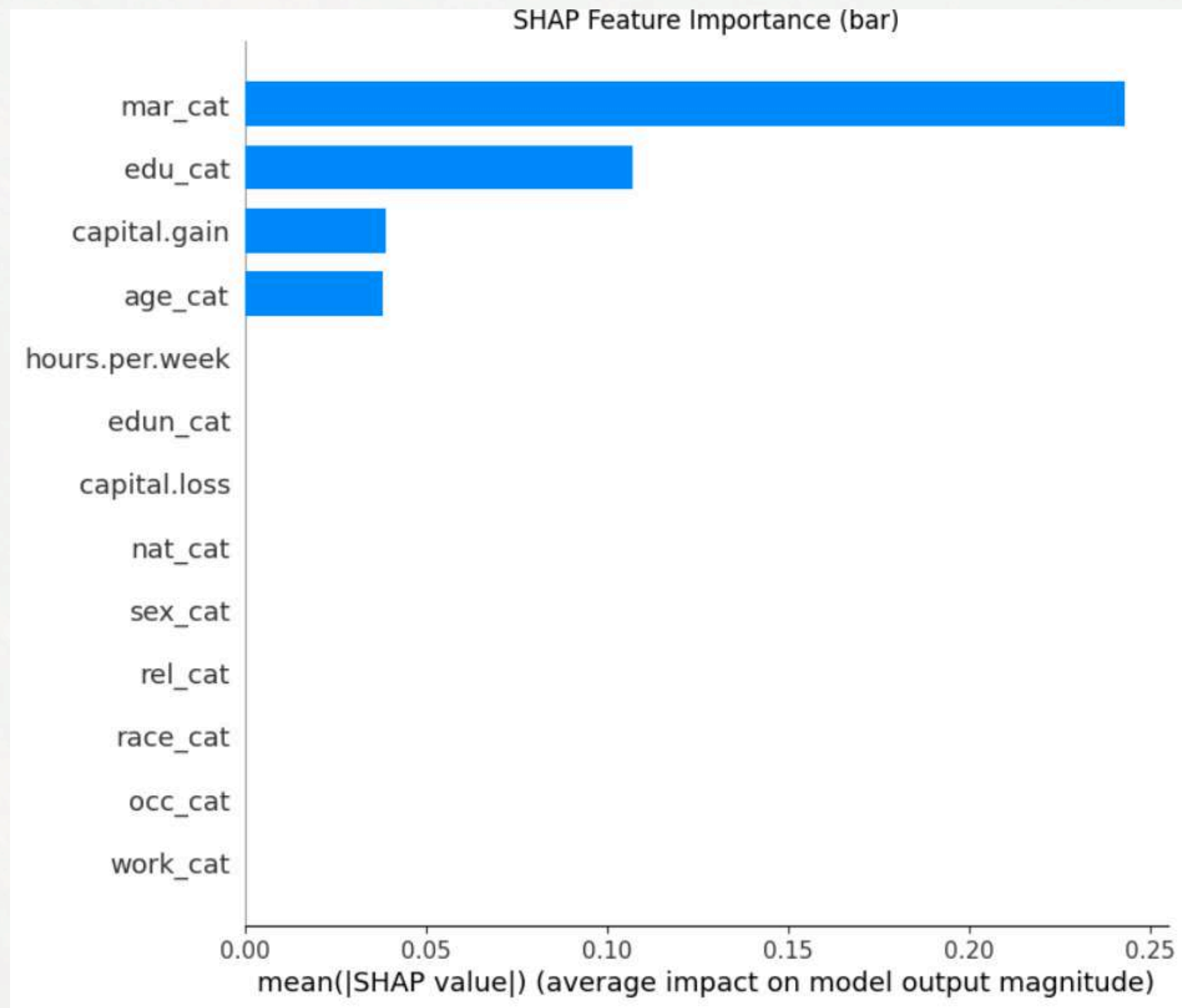
決策樹-特徵重要性



==== Average Feature Importance ====

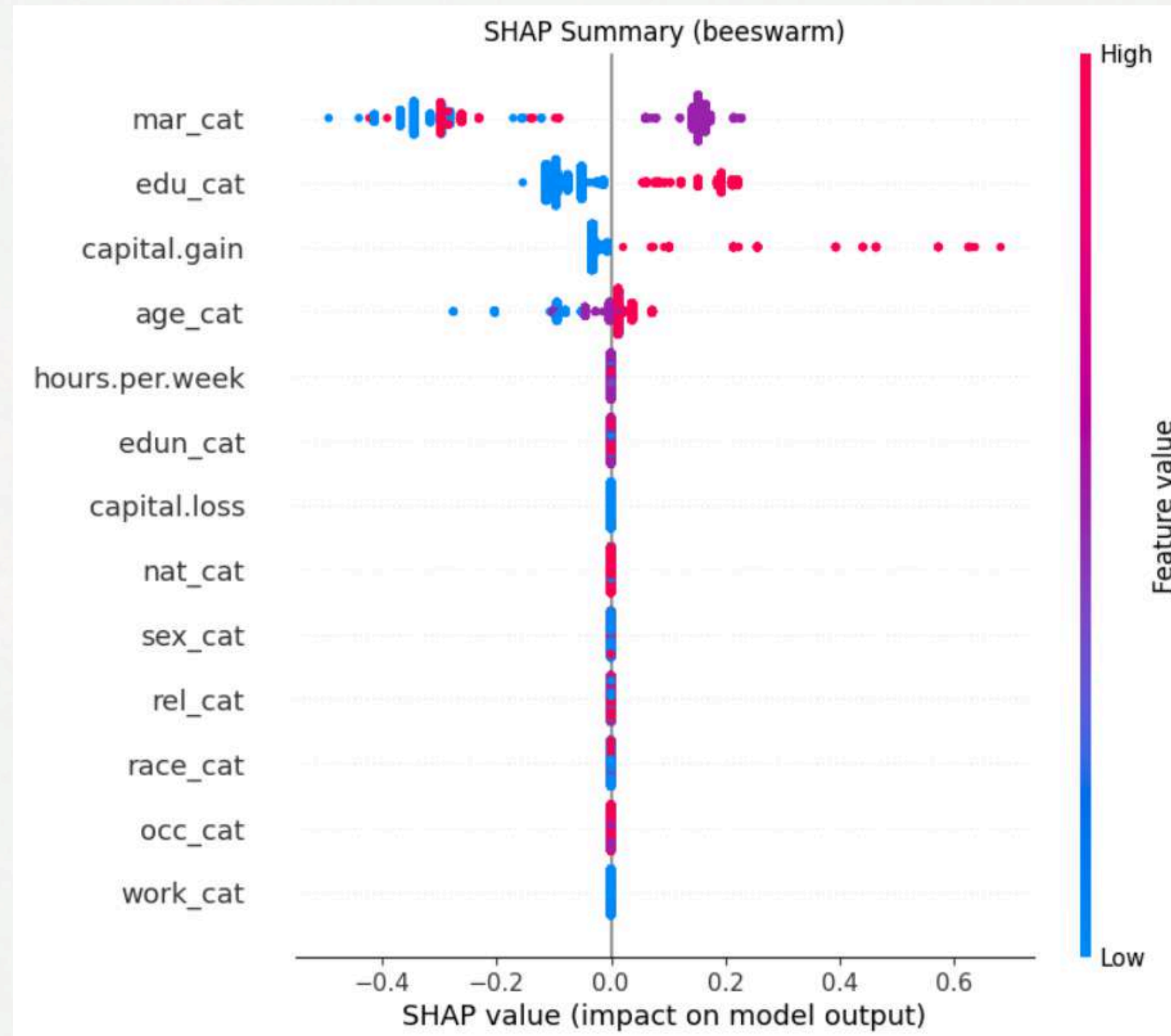
	feature	mean_importance	std_importance
0	work_cat	0.000000	0.000000
6	sex_cat	0.000000	0.000000
5	race_cat	0.000000	0.000000
4	rel_cat	0.000000	0.000000
7	nat_cat	0.000000	0.000000
11	capital.loss	0.000000	0.000000
12	hours.per.week	0.000000	0.000000
8	edun_cat	0.000010	0.000008
3	occ_cat	0.002011	0.002467
9	age_cat	0.019231	0.002947
10	capital.gain	0.122853	0.001721
1	edu_cat	0.184933	0.002818
2	mar_cat	0.670962	0.003398

決策樹-SHAP全域特徵重要性



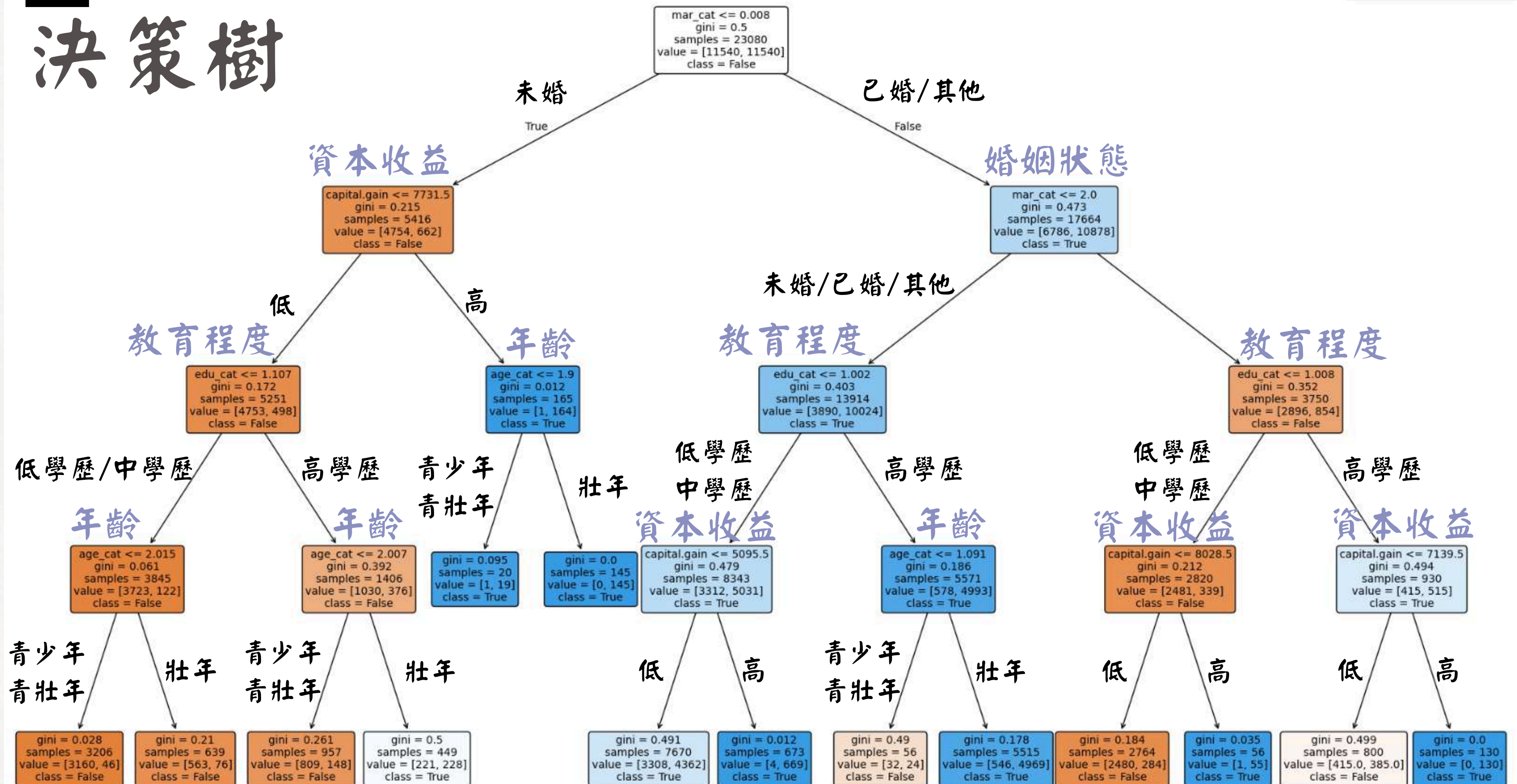
```
===== SHAP Feature Importance (mean |SHAP|) =====  
feature    mean_abs_shap  
2          mar_cat    0.242884  
1          edu_cat    0.107016  
10         capital.gain 0.039008  
9          age_cat    0.038123  
0          work_cat    0.000000  
4          rel_cat    0.000000  
3          occ_cat    0.000000  
5          race_cat    0.000000  
6          sex_cat    0.000000  
8          edun_cat    0.000000  
7          nat_cat    0.000000  
11         capital.loss 0.000000  
12         hours.per.week 0.000000
```

決策樹-SHAP值分佈摘要圖



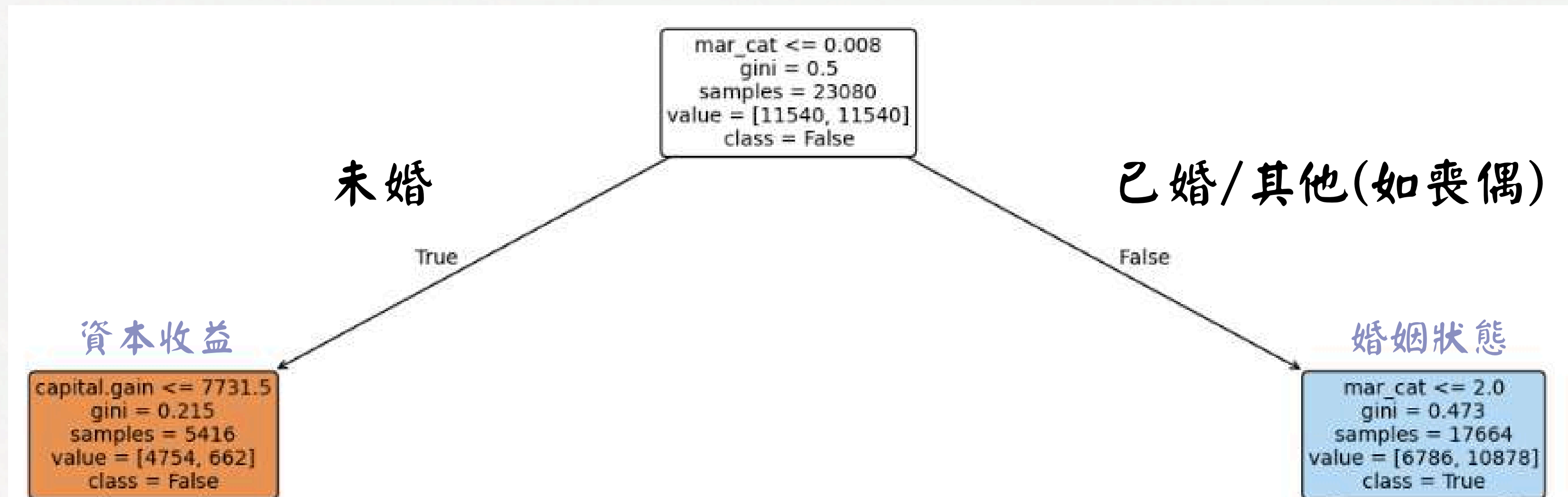
決策樹

婚姻狀態

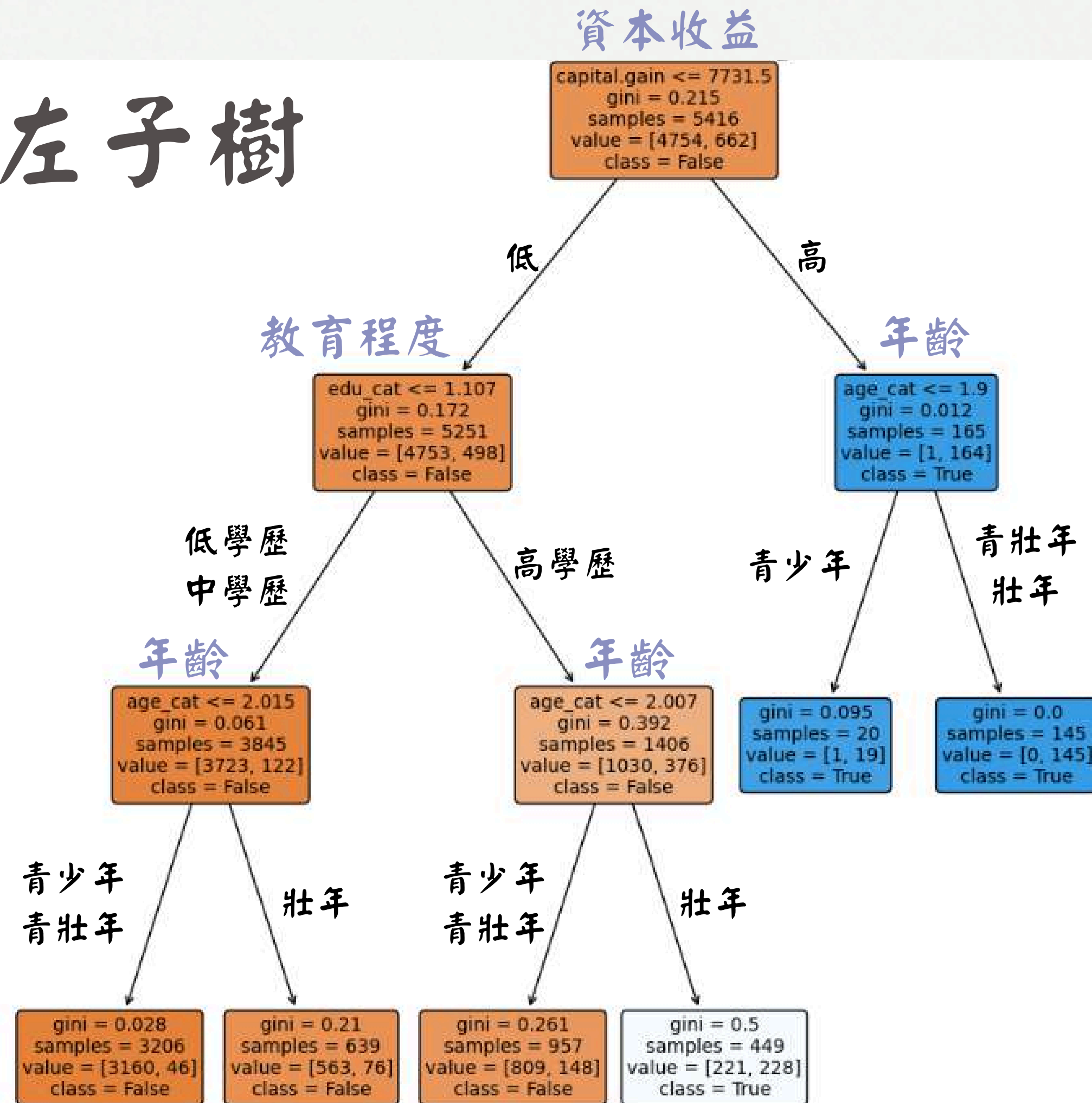


決策樹-根節點

婚姻狀態

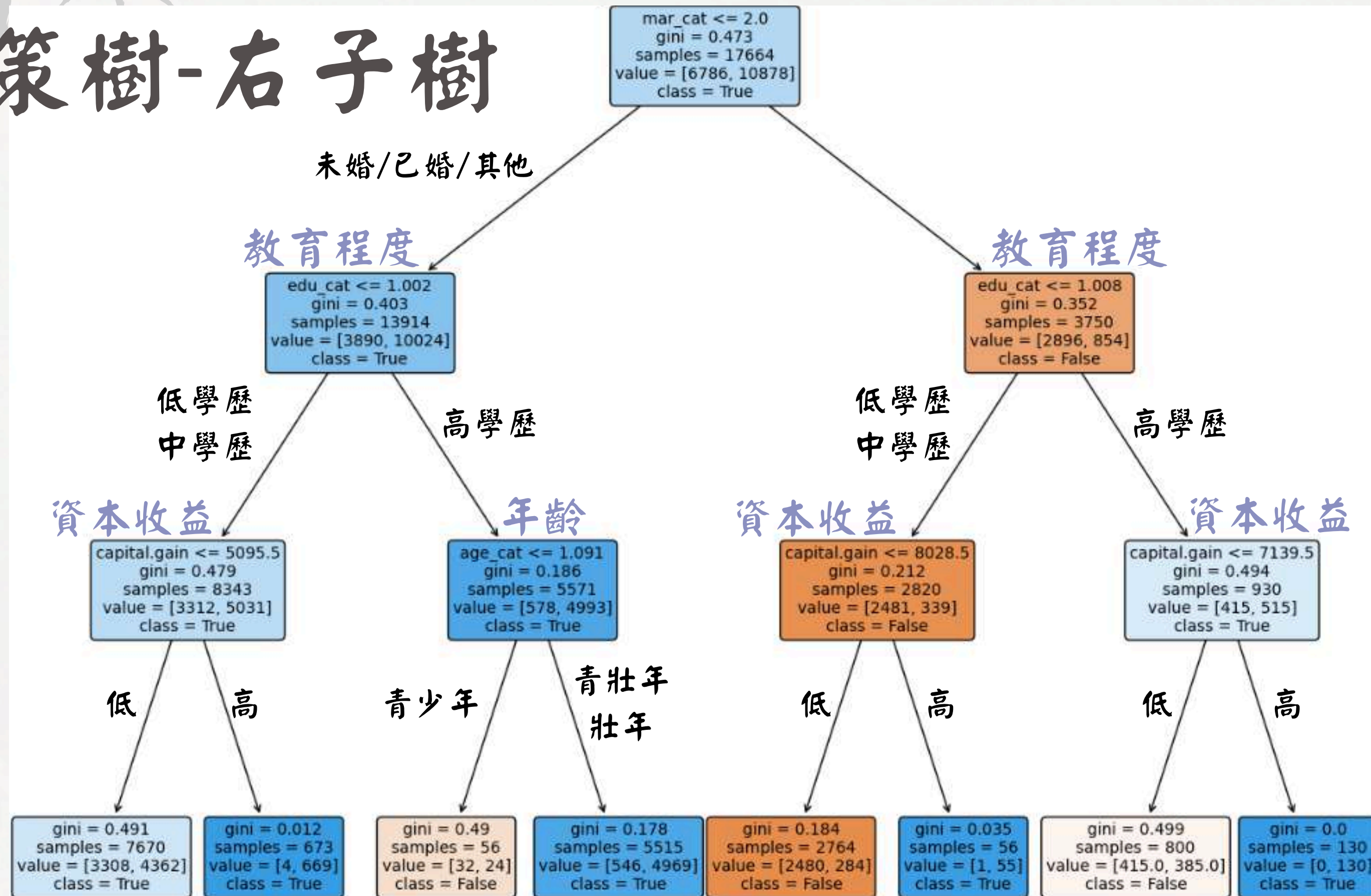


決策樹-左子樹



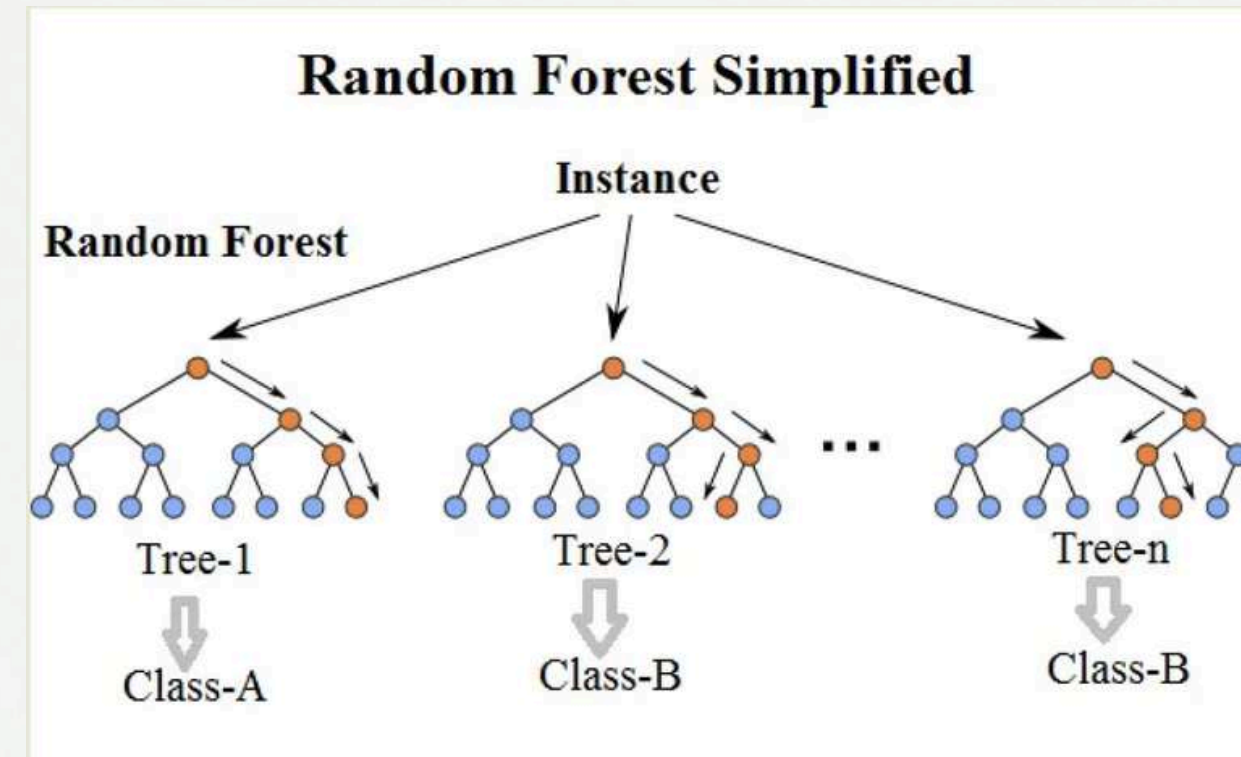
婚姻狀態

決策樹-右子樹



隨機森林

• 模型介紹



1. Bootstarp是指重新抽取原本的資料在產生新的資料，這過程保證是隨機且均勻可重複的
2. 隨機森林是把很多棵決策樹組在一起，但每棵樹都有自己的資料和特徵，讓結果比較準確

隨機森林-超參數

樹的數量	100
最大樹深	4
抽資料方式	80/20法則
樹葉節點 (每一個分支的最少樣本數)	20 (資料筆數位於1000~5000之間， 故設置葉節點為20合理)
亂數種子	42

隨機森林-模型建構程式碼

```
# =====
# 0. 安裝套件 (Colab 才需要)
# =====
!pip install pyreadstat seaborn
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score
```

```
# =====
# 1. 匯入套件
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    classification_report,
    roc_curve,
    auc
)
```

```
# =====
# 2. 載入訓練 / 測試資料
# =====
train_path = '/content/train_data.sav'
test_path = '/content/test_data.sav'

try:
    train_data = pd.read_spss(train_path)
    test_data = pd.read_spss(test_path)
    print("訓練與測試資料載入成功!")
except Exception as e:
    print(f"讀取失敗: {e}")
    exit()
```

```
# =====
# 3. 指定目標與特徵
# =====
TARGET_COLUMN = 'income_bin'

FEATURE_COLUMNS = [
    'mar_cat',
    'edu_cat',
    'hours.per.week',
    'capital.gain',
    'age_cat',
    'occ_cat'
]
```

```
# =====
# 4. 檢查欄位是否存在
# =====
missing_cols = [c for c in FEATURE_COLUMNS + [TARGET_COLUMN]
                 if c not in train_data.columns]

if missing_cols:
    print("找不到以下欄位:", missing_cols)
    print("目前資料欄位:", train_data.columns.tolist())
    exit()
```

```
# =====
# 5. 切分 X / y (已分好, 不再 split)
# =====
X_train = train_data.drop(TARGET_COLUMN, axis=1)
y_train = train_data[TARGET_COLUMN]

X_test = test_data.drop(TARGET_COLUMN, axis=1)
y_test = test_data[TARGET_COLUMN]
```

```
# =====
# 6. 目標變數編碼
# =====
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)
```

選取婚姻狀態、學歷、工
時、資本收益、年齡與職業
作為特徵因素

隨機森林-模型建構程式碼

```
# =====
# 7. 缺失值處理 (數值)
# =====
for col in X_train.columns:
    if X_train[col].dtype in [np.float64, np.int64]:
        median_val = X_train[col].median()
        X_train[col] = X_train[col].fillna(median_val)
        X_test[col] = X_test[col].fillna(median_val)

# =====
# 8. 類別變數 One-hot 編碼
# =====
cat_cols = X_train.select_dtypes(include=['object', 'category']).columns

X_train = pd.get_dummies(X_train, columns=cat_cols, drop_first=True)
X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True)

# 對齊欄位
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

print("資料前處理完成")
```

資料前處理完成

```
# =====
# 9. 訓練 Random Forest (保守設定)
# =====
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=1,
    min_samples_leaf=20,
    random_state=42,
    n_jobs=-1
)

print("開始訓練模型...")
model.fit(X_train, y_train)
```

```
# =====
# 10. 五折交叉驗證
# =====
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
import numpy as np
import pandas as pd

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 五折交叉驗證 accuracy
cv_scores = cross_val_score(
    model,
    X_train_res,
    y_train_res,
    cv=cv,
    scoring='accuracy'
)

# =====
# 只累積「正樣本」報告
# =====
report_accum = []

for train_idx, val_idx in cv.split(X_train_res, y_train_res):
    X_tr = X_train_res.iloc[train_idx]
    X_val = X_train_res.iloc[val_idx]
    y_tr = y_train_res[train_idx]
    y_val = y_train_res[val_idx]

    model.fit(X_tr, y_tr)
    y_val_pred = model.predict(X_val)

    report = classification_report(
        y_val,
        y_val_pred,
        output_dict=True,
        zero_division=0
    )

    # 只存正樣本 (label = 1)
    report_accum.append(report['1'])
```

```
# =====
# 平均 precision / recall / f1 (正樣本)
# =====
avg_report = {}
std_report = {}

for metric in report_accum[0].keys():
    values = [r[metric] for r in report_accum]
    avg_report[metric] = np.mean(values)
    std_report[metric] = np.std(values)

# =====
# 最終模型
# =====
model.fit(X_train_res, y_train_res)

y_train_pred = model.predict(X_train_res)
y_test_pred = model.predict(X_test)

print("\n訓練集準確率:", accuracy_score(y_train_res, y_train_pred))
print("測試集準確率:", accuracy_score(y_test, y_test_pred))
print("\n")

std_acc = np.std(cv_scores)
mean_acc = np.mean(cv_scores)

acc_deviation = cv_scores - mean_acc

print("五折交叉驗證準確率:", cv_scores)
print("平均準確率:", mean_acc)

print("\n五折交叉驗證平均分類報告:")
print(pd.DataFrame(avg_report, index=['positive']).T)

print("\n五折偏差 (每折-平均):", acc_deviation)
print("五折標準差:", std_acc)

print("\n五次交叉驗證標準差分類報告:")
print(pd.DataFrame(std_report, index=['positive']).T)

print("\n")
print(f"平均 ± 標準差: {mean_acc} ± {std_acc}")
```


隨機森林-模型建構程式碼

```
# =====
# 11. 混淆矩陣對比 - 五折交叉驗證 (含平均)
# =====

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix, accuracy_score

# =====
# 合併原本的 train / test 資料
# =====
X_combined = np.vstack((X_train, X_test))
y_combined = np.hstack((y_train, y_test))

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# =====
# 用來儲存每一折的結果
# =====
cm_train_list = []
cm_test_list = []
acc_train_list = []
acc_test_list = []

# =====
# 建立 6 x 2 子圖 (最後一列放平均)
# =====
fig, axes = plt.subplots(6, 2, figsize=(16, 36))

# =====
# 五折交叉驗證
# =====
for i, (train_idx, test_idx) in enumerate(cv.split(X_combined, y_combined)):

    # === 切分資料 ===
    X_train_fold = X_combined[train_idx]
    X_test_fold = X_combined[test_idx]
    y_train_fold = y_combined[train_idx]
    y_test_fold = y_combined[test_idx]

    # === 訓練模型 ===
    model.fit(X_train_fold, y_train_fold)

    # === 預測 ===
    y_train_pred = model.predict(X_train_fold)
    y_test_pred = model.predict(X_test_fold)
```

```
# =====
# Train Confusion Matrix
# =====
cm_train = confusion_matrix(y_train_fold, y_train_pred)
acc_train = accuracy_score(y_train_fold, y_train_pred)

cm_train_list.append(cm_train)
acc_train_list.append(acc_train)

sns.heatmap(
    cm_train,
    annot=True,
    fmt='d',
    cmap='Oranges',
    ax=axes[i, 0]
)
axes[i, 0].set_title(f'Fold {i+1} - Train\nAccuracy={acc_train:.4f}')
axes[i, 0].set_xlabel('Predicted')
axes[i, 0].set_ylabel('True')

# =====
# Test Confusion Matrix
# =====
cm_test = confusion_matrix(y_test_fold, y_test_pred)
acc_test = accuracy_score(y_test_fold, y_test_pred)

cm_test_list.append(cm_test)
acc_test_list.append(acc_test)

sns.heatmap(
    cm_test,
    annot=True,
    fmt='d',
    cmap='Blues',
    ax=axes[i, 1]
)
axes[i, 1].set_title(f'Fold {i+1} - Test\nAccuracy={acc_test:.4f}')
axes[i, 1].set_xlabel('Predicted')
axes[i, 1].set_ylabel('True')
```

```
# =====
# 計算平均混淆矩陣與平均準確率
# =====
mean_cm_train = np.mean(cm_train_list, axis=0)
mean_cm_test = np.mean(cm_test_list, axis=0)

mean_acc_train = np.mean(acc_train_list)
mean_acc_test = np.mean(acc_test_list)

# =====
# 平均 Train Confusion Matrix
# =====
sns.heatmap(
    mean_cm_train,
    annot=True,
    fmt='.2f',
    cmap='Oranges',
    ax=axes[5, 0]
)
axes[5, 0].set_title(
    f'Mean Train (5-Fold CV)\nAccuracy={mean_acc_train:.4f}'
)
axes[5, 0].set_xlabel('Predicted')
axes[5, 0].set_ylabel('True')

# =====
# 平均 Test Confusion Matrix
# =====
sns.heatmap(
    mean_cm_test,
    annot=True,
    fmt='.2f',
    cmap='Blues',
    ax=axes[5, 1]
)
axes[5, 1].set_title(
    f'Mean Test (5-Fold CV)\nAccuracy={mean_acc_test:.4f}'
)
axes[5, 1].set_xlabel('Predicted')
axes[5, 1].set_ylabel('True')

plt.tight_layout()
plt.show()
```


隨機森林-模型建構程式碼

```

# =====
# 12. ROC Curve (五折交叉驗證)
# =====
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

mean_fpr = np.linspace(0, 1, 100)
tprs = []
aucs = []

plt.figure(figsize=(8, 6))

for i, (train_idx, val_idx) in enumerate(cv.split(X_train_res, y_train_res), 1):
    X_tr = X_train_res.iloc[train_idx]
    X_val = X_train_res.iloc[val_idx]
    y_tr = y_train_res[train_idx]
    y_val = y_train_res[val_idx]

    model.fit(X_tr, y_tr)
    y_val_prob = model.predict_proba(X_val)[:, 1]

    fpr, tpr, _ = roc_curve(y_val, y_val_prob)
    roc_auc = auc(fpr, tpr)

    # 畫每一折 ROC
    plt.plot(fpr, tpr, lw=1, alpha=0.4,
             label=f'Fold {i} ROC (AUC = {roc_auc:.4f})')

    # 插值用來算平均 ROC
    interp_tpr = np.interp(mean_fpr, fpr, tpr)
    interp_tpr[0] = 0.0
    tprs.append(interp_tpr)
    aucs.append(roc_auc)

# =====
# 平均 ROC + 標準差
# =====
mean_tpr = np.mean(tprs, axis=0)
mean_tpr[-1] = 1.0
mean_auc = np.mean(aucs)
std_auc = np.std(aucs)

std_tpr = np.std(tprs, axis=0)

plt.plot(
    mean_fpr,
    mean_tpr,
    color='b',
    label=f'Mean ROC (AUC = {mean_auc:.4f} ± {std_auc:.4f})',
    lw=2
)

plt.fill_between(
    mean_fpr,
    mean_tpr - std_tpr,
    mean_tpr + std_tpr,
    color='grey',
    alpha=0.2,
    label='±1 std'
)

# Random guess
plt.plot([0, 1], [0, 1], linestyle=':', color='black', label='Random Guess')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (5-Fold Cross-Validation)')
plt.legend(loc='lower right')
plt.grid()
plt.show()

# =====
# 數值輸出 (老師看數據用)
# =====
print("五折 AUC:", aucs)
print("平均 AUC:", mean_auc)
print("五次標準差:", std_auc)
print(f"平均 ± 標準差: {mean_auc:.4f} ± {std_auc:.4f}")

```

隨機森林-模型建構程式碼

```
# =====
# 13. ROC Curve (五折交叉驗證)
# =====
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt

# ===== 5-fold CV =====
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

mean_fpr = np.linspace(0, 1, 100)

tprs_train, tprs_test = [], []
aucs_train, aucs_test = [], []

for i, (train_idx, test_idx) in enumerate(cv.split(X_train_res, y_train_res), 1):

    X_tr = X_train_res.iloc[train_idx]
    X_te = X_train_res.iloc[test_idx]
    y_tr = y_train_res[train_idx]
    y_te = y_train_res[test_idx]

    # ===== Train model =====
    model.fit(X_tr, y_tr)

    # ===== Train ROC =====
    y_tr_prob = model.predict_proba(X_tr)[:, 1]
    fpr_tr, tpr_tr, _ = roc_curve(y_tr, y_tr_prob)
    auc_tr = auc(fpr_tr, tpr_tr)

    # ===== Test ROC =====
    y_te_prob = model.predict_proba(X_te)[:, 1]
    fpr_te, tpr_te, _ = roc_curve(y_te, y_te_prob)
    auc_te = auc(fpr_te, tpr_te)

    # ===== 每個 fold 畫一張圖 =====
    plt.figure(figsize=(6, 5))
    plt.plot(fpr_tr, tpr_tr, lw=2, label=f'Train ROC (AUC = {auc_tr:.4f})')
    plt.plot(fpr_te, tpr_te, lw=2, label=f'Test ROC (AUC = {auc_te:.4f})')
    plt.plot([0, 1], [0, 1], linestyle=':', color='gray')

    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve - Fold {i}')
    plt.legend(loc='lower right')
    plt.grid()
```

```
# ===== 存起來算平均 =====
tprs_train.append(np.interp(mean_fpr, fpr_tr, tpr_tr))
tprs_test.append(np.interp(mean_fpr, fpr_te, tpr_te))
aucs_train.append(auc_tr)
aucs_test.append(auc_te)

# ===== 平均 ROC =====
mean_tpr_train = np.mean(tprs_train, axis=0)
mean_tpr_test = np.mean(tprs_test, axis=0)

mean_tpr_train[-1] = 1.0
mean_tpr_test[-1] = 1.0

mean_auc_train = np.mean(aucs_train)
std_auc_train = np.std(aucs_train)

mean_auc_test = np.mean(aucs_test)
std_auc_test = np.std(aucs_test)

plt.figure(figsize=(7, 6))

plt.plot(
    mean_fpr,
    mean_tpr_train,
    lw=2,
    label=f'Train Mean ROC (AUC = {mean_auc_train:.4f} ± {std_auc_train:.4f})'
)

plt.plot(
    mean_fpr,
    mean_tpr_test,
    lw=2,
    label=f'Test Mean ROC (AUC = {mean_auc_test:.4f} ± {std_auc_test:.4f})'
)

plt.plot([0, 1], [0, 1], linestyle=':', color='gray')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Mean ROC Curve (5-Fold Cross-Validation)')
plt.legend(loc='lower right')
plt.grid()
plt.show()
```

```
# =====
# 14. 成效報告
# =====

# 重新計算完整的訓練集和測試集預測
# model 是在 step 10 的最後一折交叉驗證中訓練的結果
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# 重新計算完整的訓練集和測試集準確率
acc_train = accuracy_score(y_train, y_train_pred)
acc_test = accuracy_score(y_test, y_test_pred)

print(f'Train Accuracy: {acc_train:.4f}')
print(f'Test Accuracy: {acc_test:.4f}')

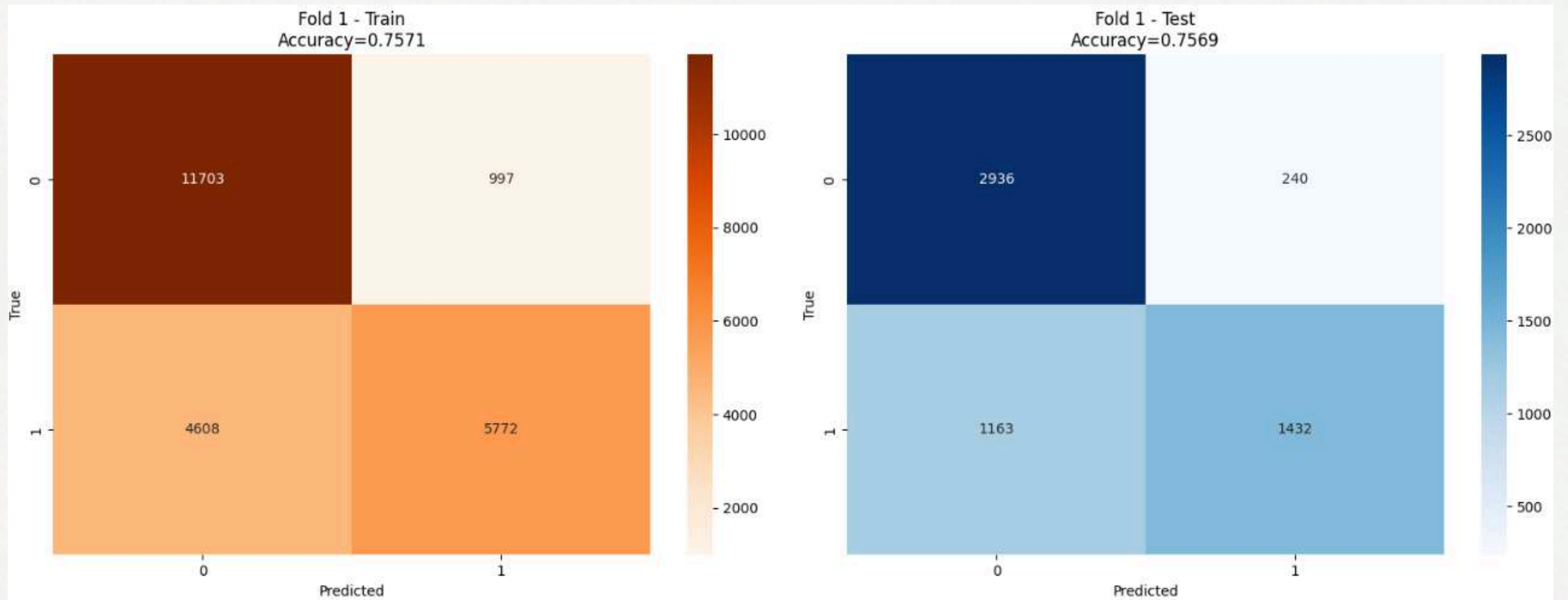
print("\n--- Train Classification Report ---")
print(classification_report(y_train, y_train_pred))

print("\n--- Test Classification Report ---")
print(classification_report(y_test, y_test_pred))
```

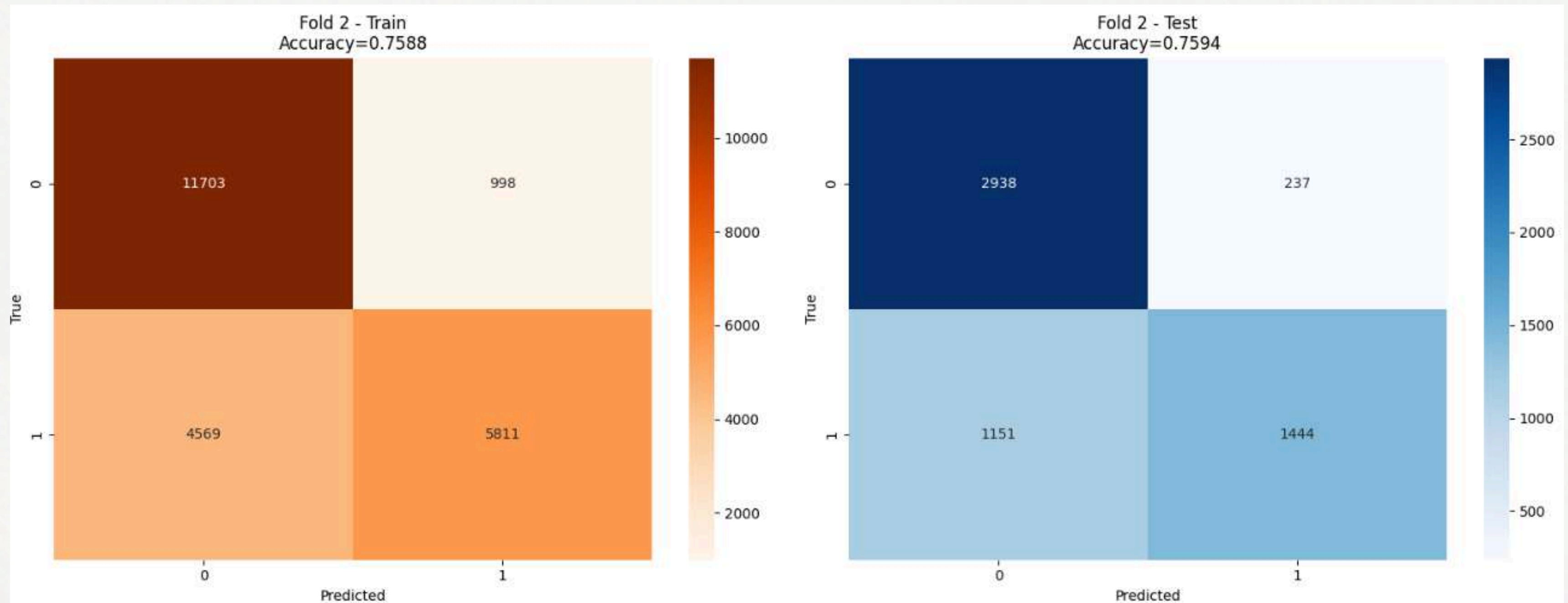
隨機森林-混淆矩陣檢驗模型

	實際為真	實際為假
預測為真	真陽性(TP)	偽陽性(FP) 1
預測為假	偽陰性(FN) 2	真陰性(TN)

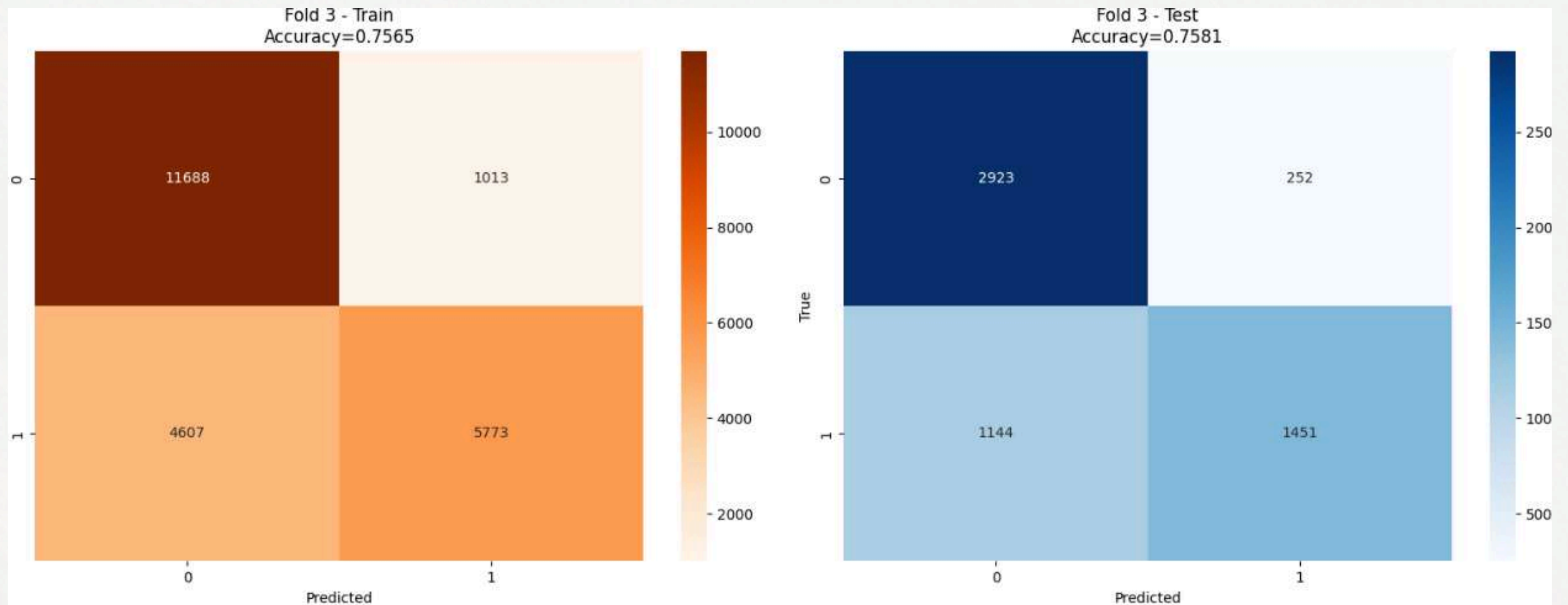
隨機森林-第一折



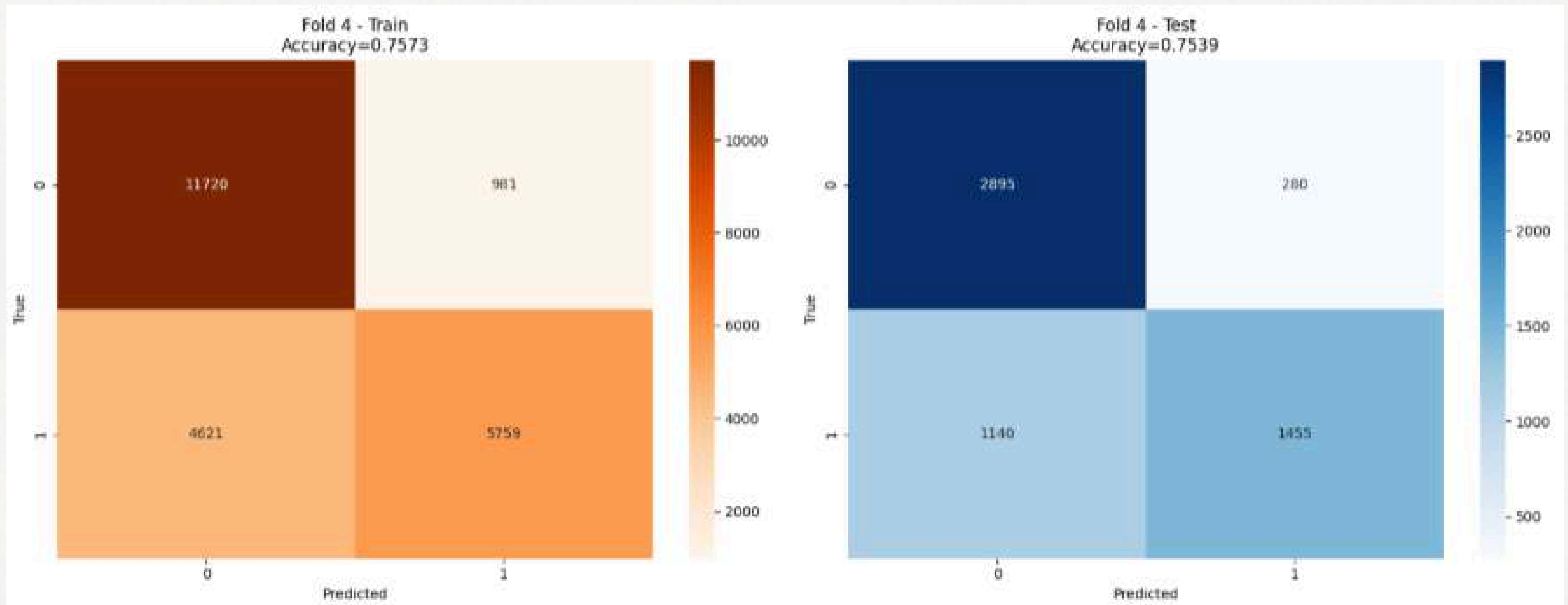
隨機森林-第二折



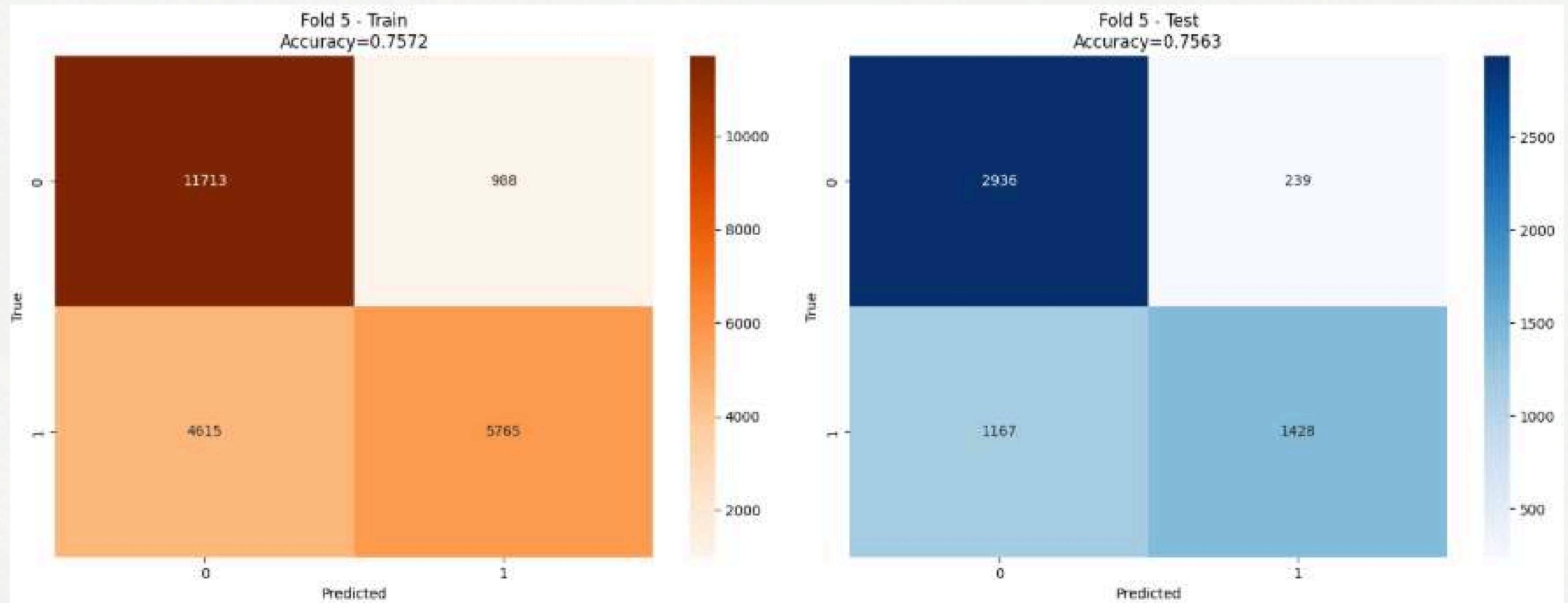
隨機森林-第三折



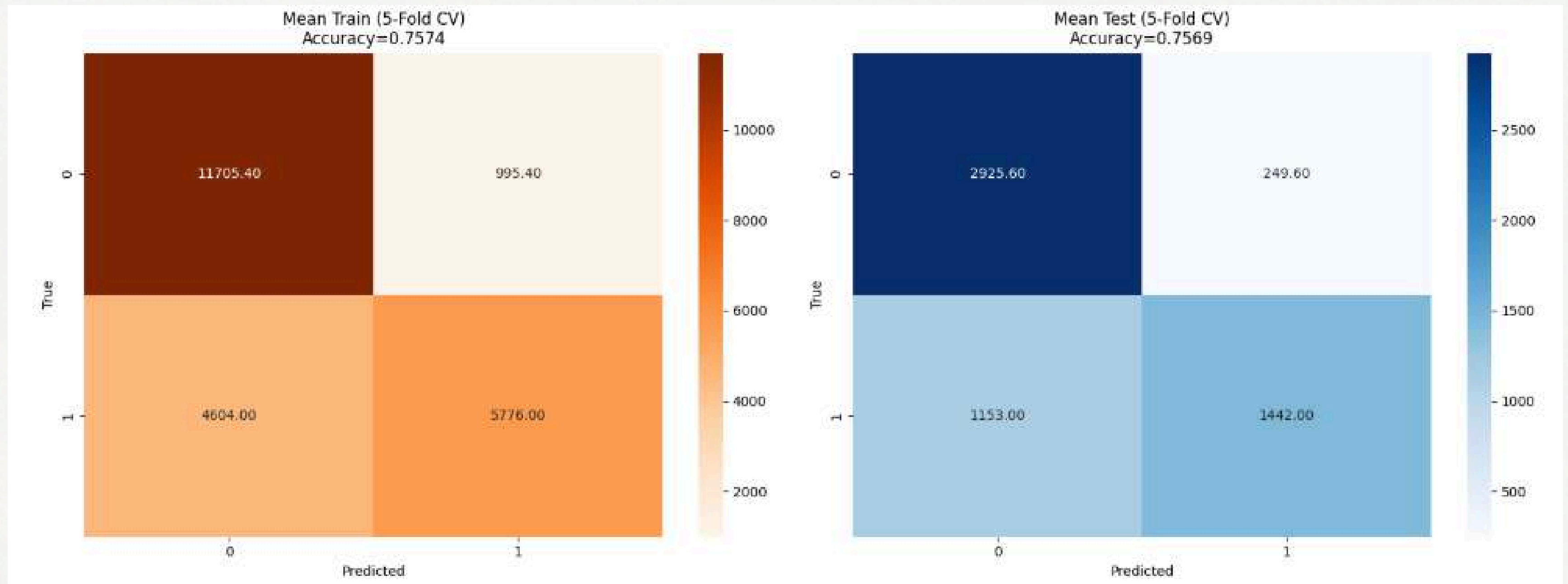
隨機森林-第四折



隨機森林-第五折



隨機森林-訓練集&測試集



五折交叉驗證

五折交叉驗證準確率: [0.78596187 0.78704506 0.78531196 0.78336222 0.7952773]

平均準確率: 0.7873916811091854

五折交叉驗證平均分類報告:

	positive
precision	0.753206
recall	0.855633
f1-score	0.800928
support	2308.000000

五折偏差 (每折-平均): [-0.00142981 -0.00034662 -0.00207972 -0.00402946 0.00788562]

五折標準差: 0.004120903903613393

五次交叉驗證標準差分類報告:

	positive
precision	0.010134
recall	0.018923
f1-score	0.004474
support	0.000000

平均 ± 標準差: 0.7873916811091854 ± 0.004120903903613393

訓練集準確率: 0.7899046793760832

測試集準確率: 0.7535955640270318

$$Accuracy_i = \frac{\text{該 fold 測試集預測正確的樣本數}}{\text{該 fold 測試集樣本數}}$$

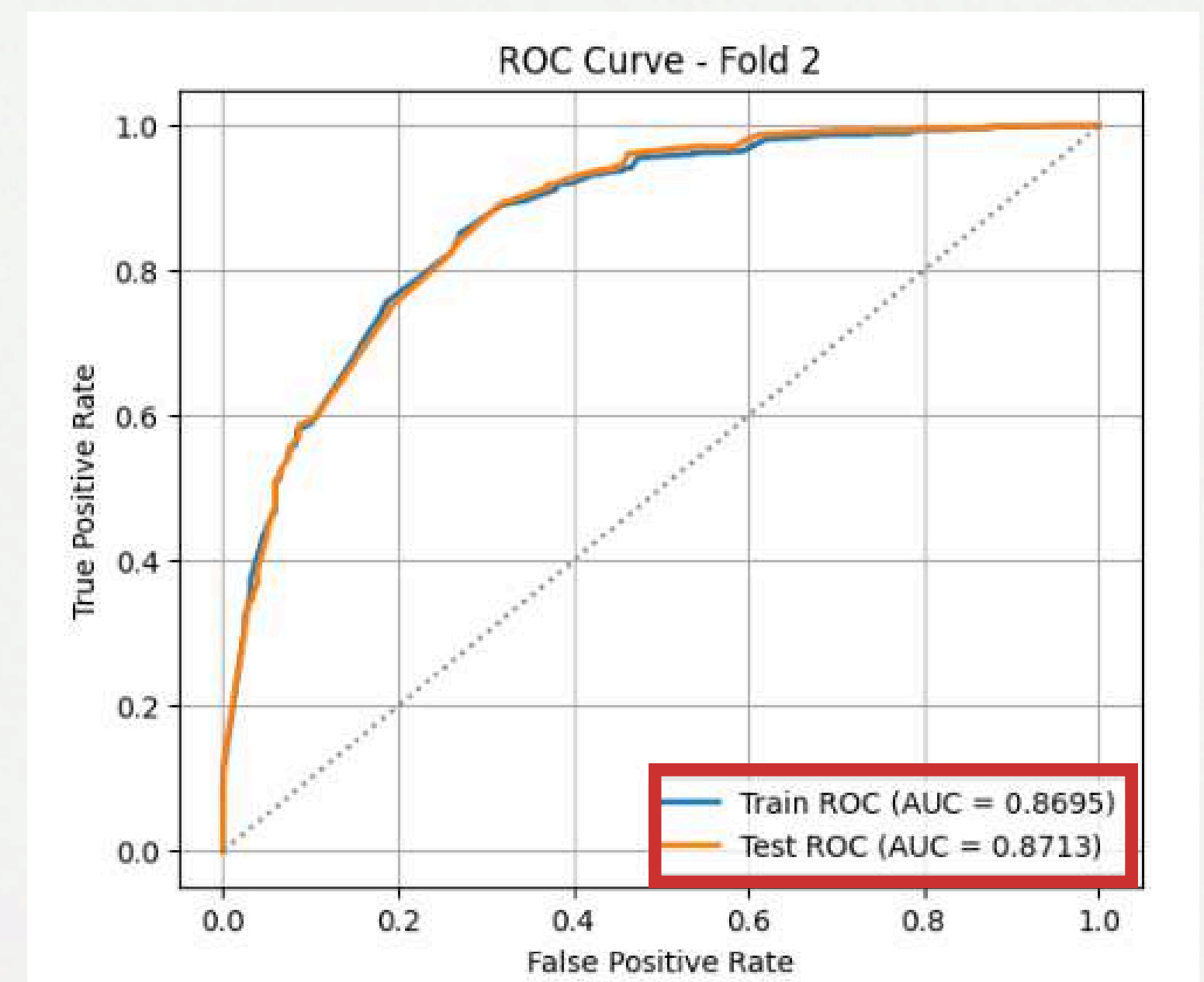
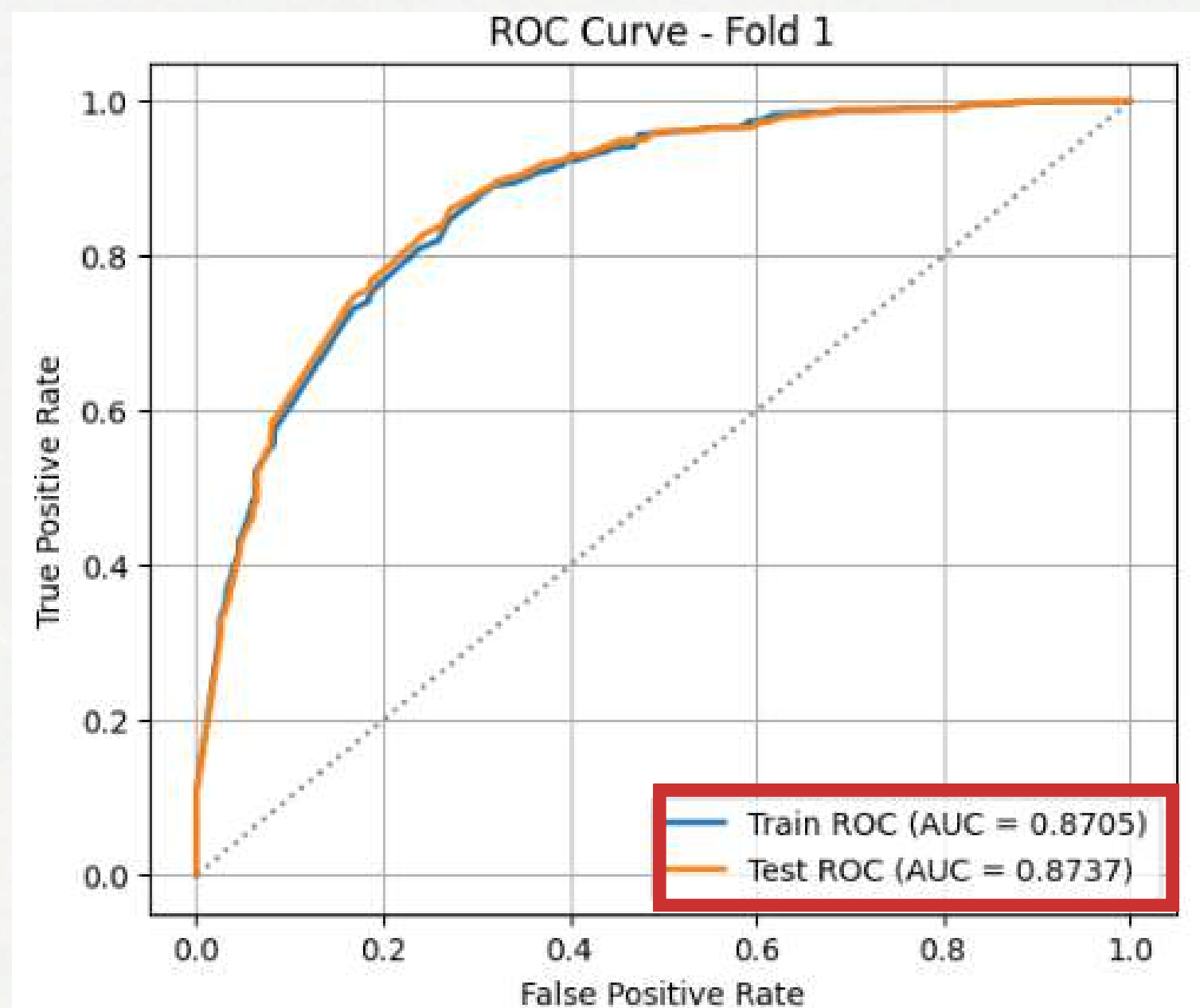
$$\begin{aligned} F1 \text{ Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

隨機森林-ROC曲線

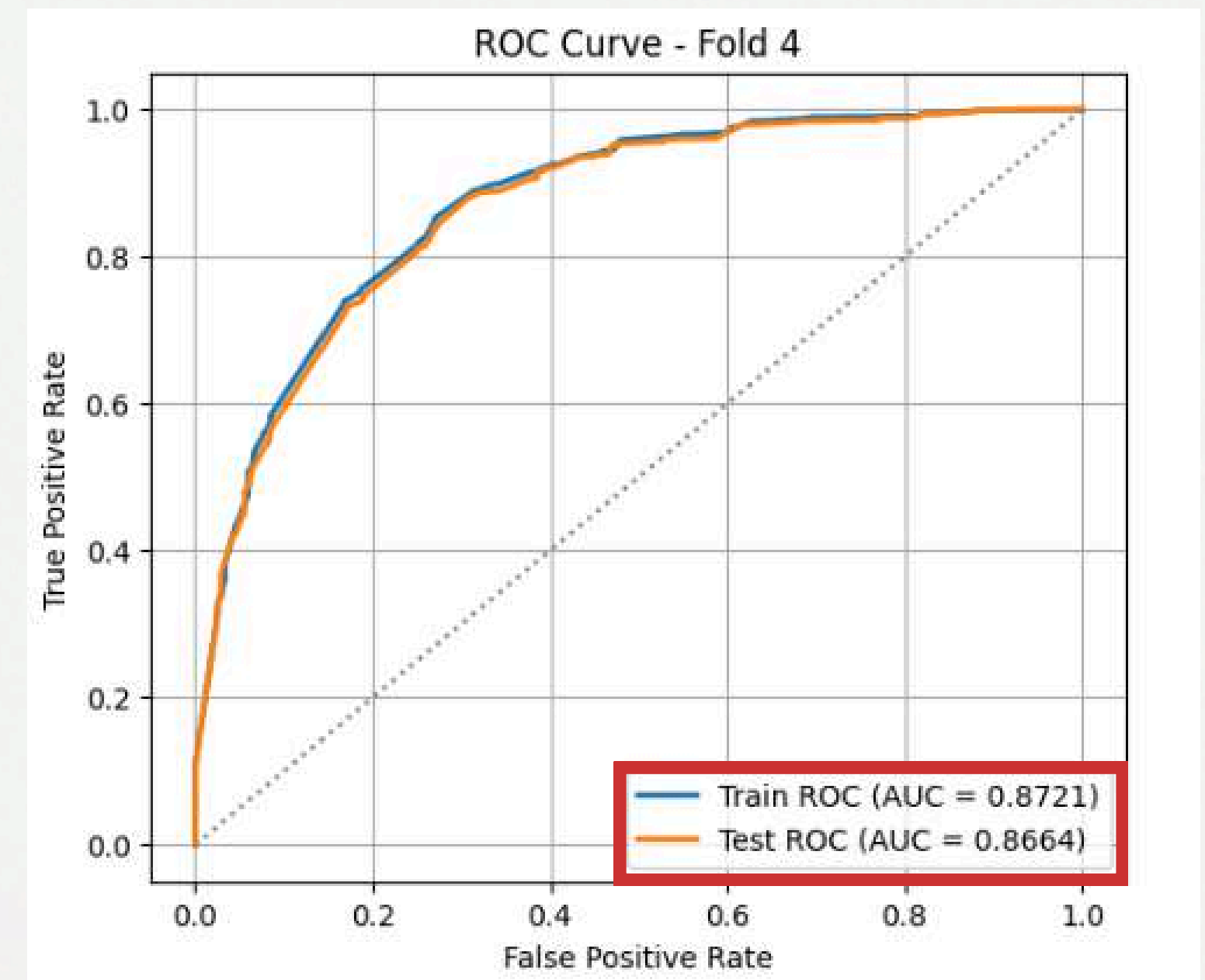
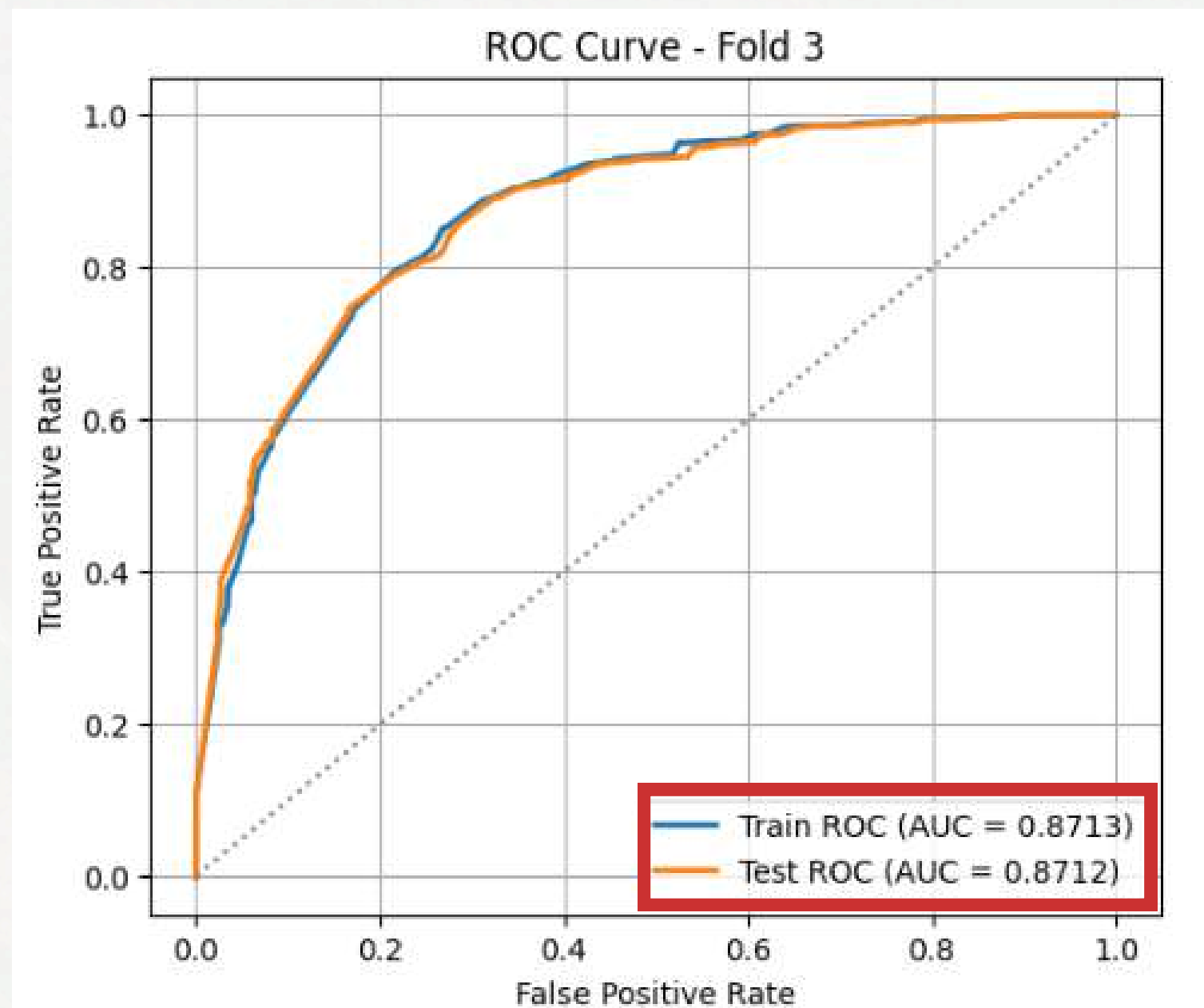
- 定義：AUC=ROC曲線下的面積
- 是一種用來評估分類模型表現的圖形，透過比較真陽性率(TPR)與假陽性率(FPR)，來判斷模型區分正負類的能力。

AUC數值	模型程度
< 0.7	模型弱，需改進
0.7 ~ 0.8	一般
0.8 ~ 0.9	很好
0.9以上	非常優秀

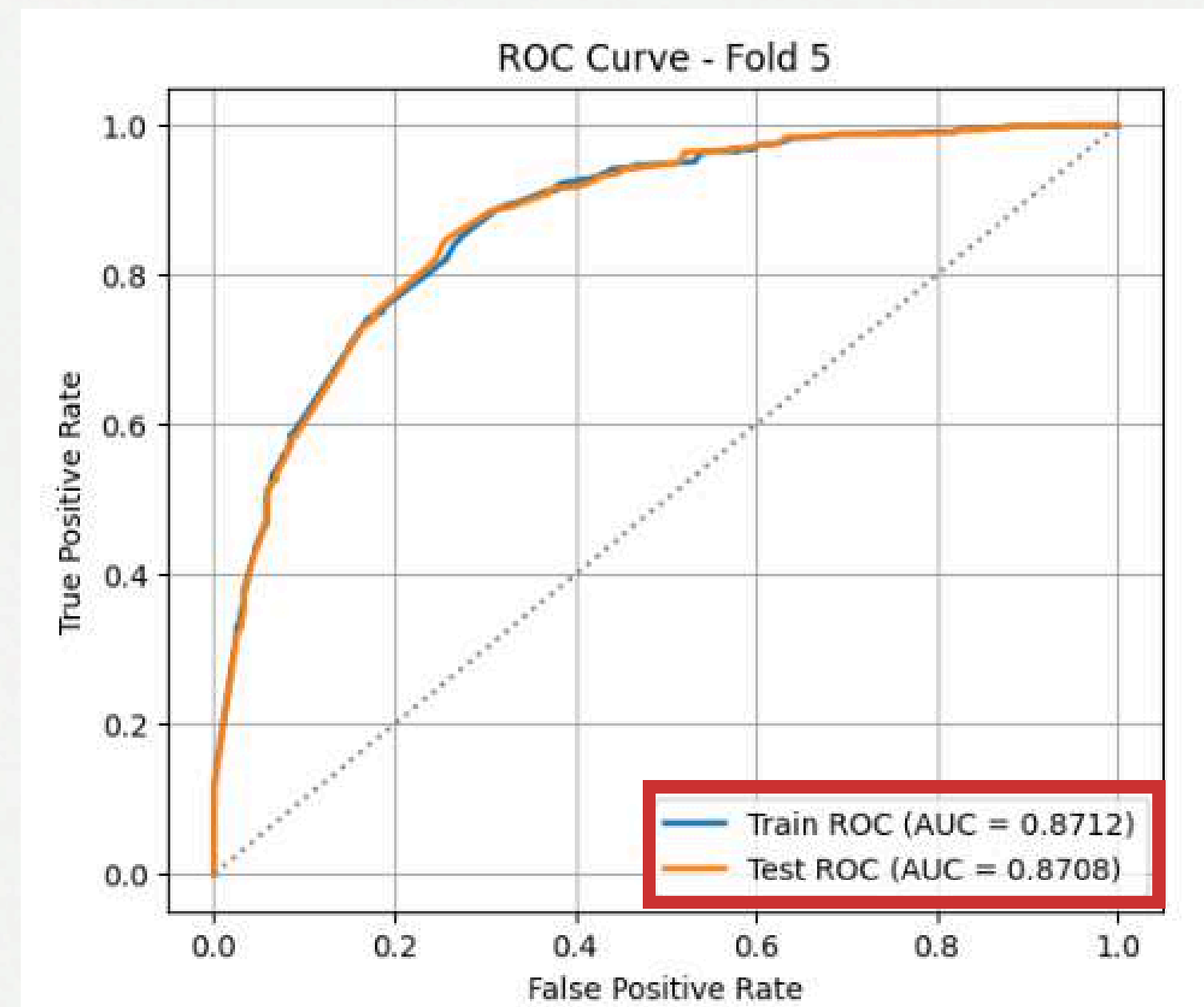
隨機森林-ROC曲線



隨機森林-ROC曲線



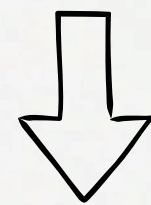
隨機森林-ROC曲線



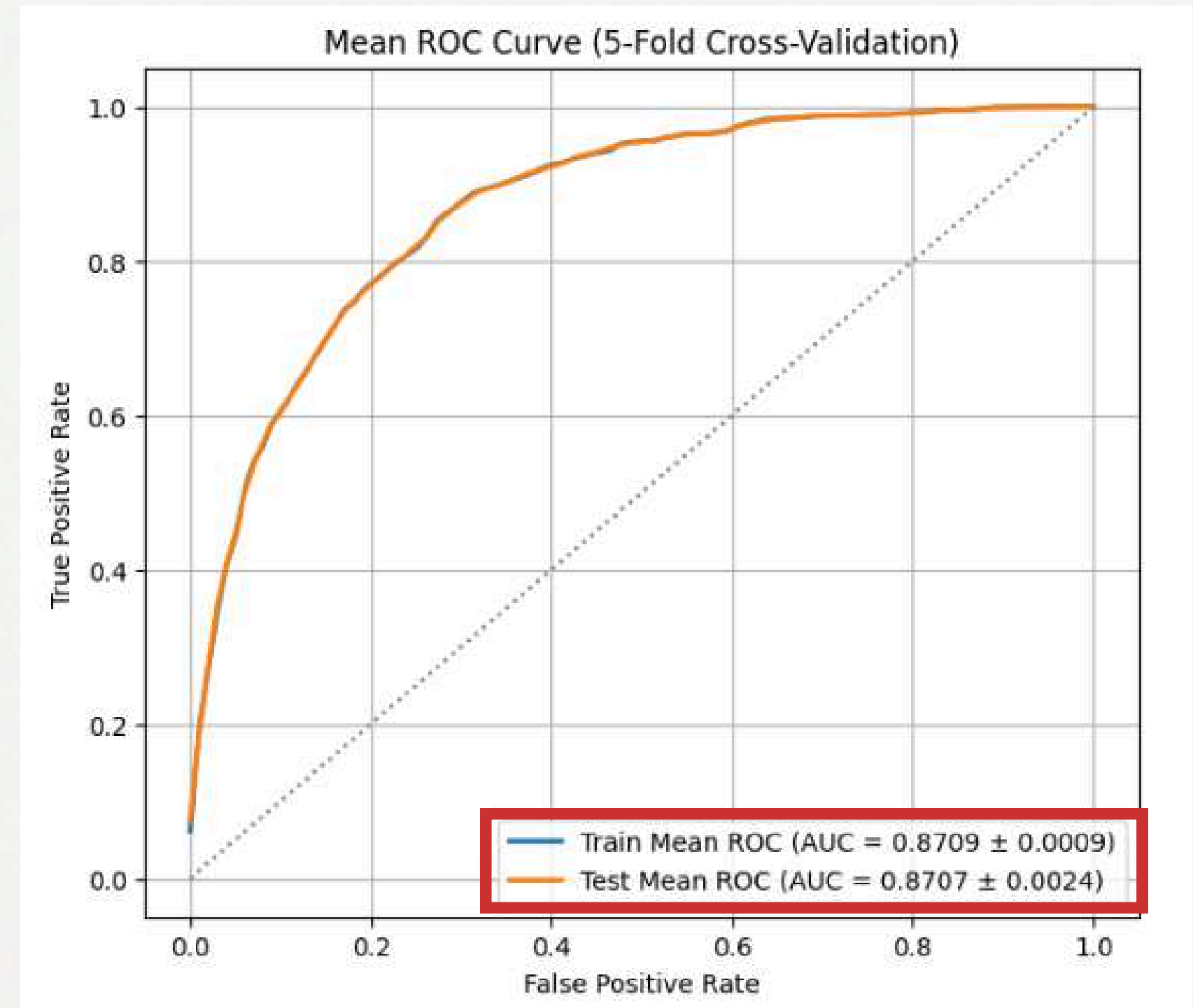
隨機森林-ROC曲線

訓練集 0.8709 ± 0.0009

測試集 為 0.8707 ± 0.0024



模型很好
表示有一定可信度



隨機森林-預測結果

訓練集的準確率為**0.7899**，

此模型中的Recall、F1-score，發現二項指標皆約位於0.79。

測試集的準確率為**0.7536**，

此模型中的Recall、F1-score約介於0.72至0.78之間。

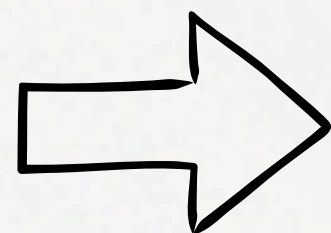
模型經五折交叉驗證之後發現模型平均準確率為

0.7874 ± 0.0041 ，而平均AUC為 **0.87 ± 0.0024** ，代表此模型具

一定程度之可信度，但仍存在部分誤判情形。

羅吉斯回歸

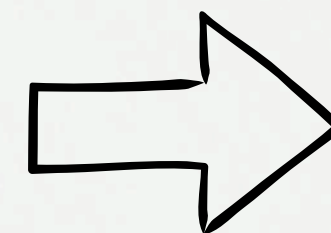
特徵(X)
標籤(Y)



*income $\geq$ 50K發生機率
income $<$ 50K發生機率

勝算比取對數

$$\text{logit}(\text{Odds}) = \ln\left(\frac{p}{1-p}\right) = w_0 + w_1 X$$



Sigmoid函數

$$S(X) = \frac{1}{1 + e^{-X}}$$

≥ 0.5 為 1
 < 0.5 為 0

羅吉斯回歸-模型建構程式碼

```
# =====
# 1 套件
# =====
!pip install pyreadstat
import pandas as pd
import numpy as np
import pyreadstat
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc

# =====
# 2 讀取資料
# =====
train_df, _ = pyreadstat.read_sav("train_balanced.sav")
test_df, _ = pyreadstat.read_sav("test.sav")

feature_cols = [
    "work_cat", "edu_cat", "mar_cat", "occ_cat", "rel_cat",
    "race_cat", "sex_cat", "nat_cat", "edun_cat", "age_cat",
    "capital.gain", "capital.loss", "hours.per.week"
]

X_train = train_df[feature_cols]
y_train = train_df["income_bin"]

X_test = test_df[feature_cols]
y_test = test_df["income_bin"]
```

```
# =====
# 3 模型 Pipeline
# =====
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("logistic", LogisticRegression(
        solver="saga", max_iter=2000, tol=1e-3,
        class_weight="balanced", n_jobs=-1
    ))
])
```


羅吉斯回歸-五折交叉驗證程式碼

```
# =====
# 4 五折交叉驗證
# =====

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

train_acc_list = []
test_acc_list = []
cv_acc_list = []

train_reports = []
test_reports = []

roc_train_list = []
roc_val_list = []
roc_test_list = []

fold = 1
for train_idx, val_idx in cv.split(X_train, y_train):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    pipeline.fit(X_tr, y_tr)

    # 預測
    y_tr_pred = pipeline.predict(X_tr)
    y_val_pred = pipeline.predict(X_val)
    y_test_pred = pipeline.predict(X_test)

    y_tr_prob = pipeline.predict_proba(X_tr)[:, 1]
    y_val_prob = pipeline.predict_proba(X_val)[:, 1]
    y_test_prob = pipeline.predict_proba(X_test)[:, 1]

    # Accuracy
    acc_train = accuracy_score(y_tr, y_tr_pred)
    acc_val = accuracy_score(y_val, y_val_pred)
    acc_test = accuracy_score(y_test, y_test_pred)

    train_acc_list.append(acc_train)
    test_acc_list.append(acc_test)
    cv_acc_list.append(acc_val)

    # 分類報告
    train_reports.append(classification_report(y_tr, y_tr_pred, output_dict=True))
    test_reports.append(classification_report(y_test, y_test_pred, output_dict=True))

    # ROC
    fpr_tr, tpr_tr, _ = roc_curve(y_tr, y_tr_prob)
    fpr_val, tpr_val, _ = roc_curve(y_val, y_val_prob)
    fpr_te, tpr_te, _ = roc_curve(y_test, y_test_prob)

    roc_train_list.append((fpr_tr, tpr_tr))
    roc_val_list.append((fpr_val, tpr_val))
    roc_test_list.append((fpr_te, tpr_te))

    roc_auc_train = auc(fpr_tr, tpr_tr)
    roc_auc_val = auc(fpr_val, tpr_val)
    roc_auc_test = auc(fpr_te, tpr_te)
```

羅吉斯回歸-混淆矩陣程式碼

```
fig, axes = plt.subplots(2,3, figsize=(18,10))

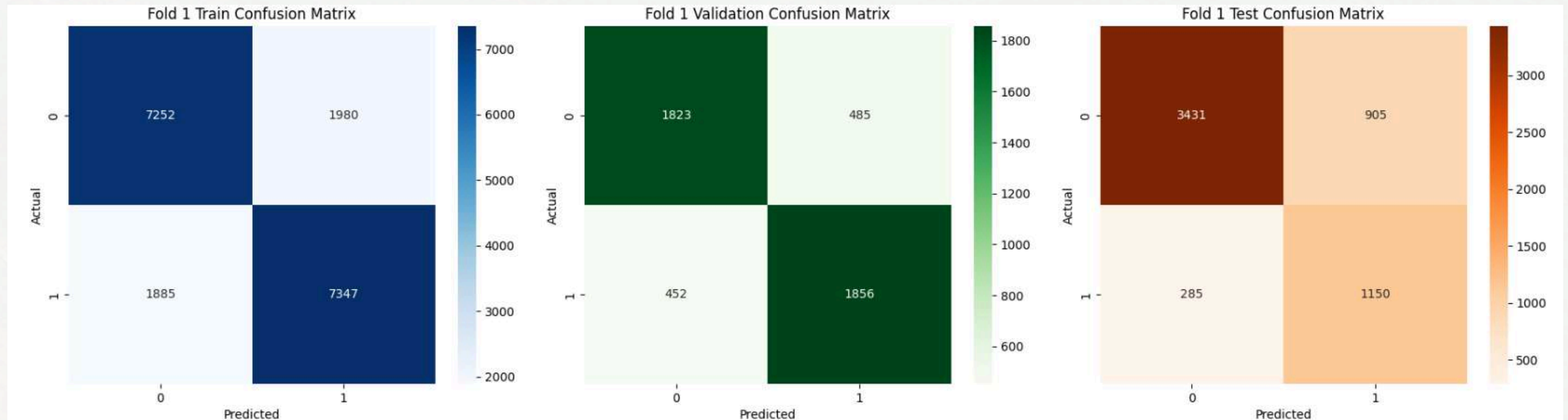
# 混淆矩陣
sns.heatmap(confusion_matrix(y_tr, y_tr_pred), annot=True, fmt='d', ax=axes[0,0], cmap='Blues')
axes[0,0].set_title(f'Fold {fold} Train Confusion Matrix')
axes[0,0].set_xlabel('Predicted')
axes[0,0].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_val, y_val_pred), annot=True, fmt='d', ax=axes[0,1], cmap='Greens')
axes[0,1].set_title(f'Fold {fold} Validation Confusion Matrix')
axes[0,1].set_xlabel('Predicted')
axes[0,1].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_test, y_test_pred), annot=True, fmt='d', ax=axes[0,2], cmap='Oranges')
axes[0,2].set_title(f'Fold {fold} Test Confusion Matrix')
axes[0,2].set_xlabel('Predicted')
axes[0,2].set_ylabel('Actual')
```

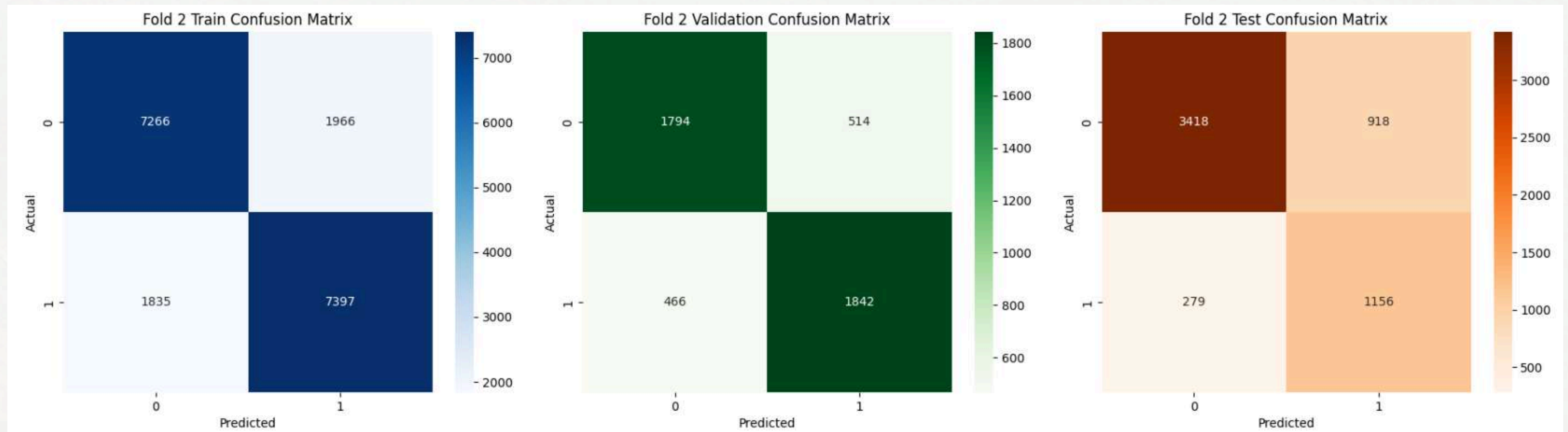
羅吉斯回歸-混淆矩陣Fold1

Train Accuracy: 0.7907, Validation Accuracy: 0.7970, Test Accuracy: 0.7938



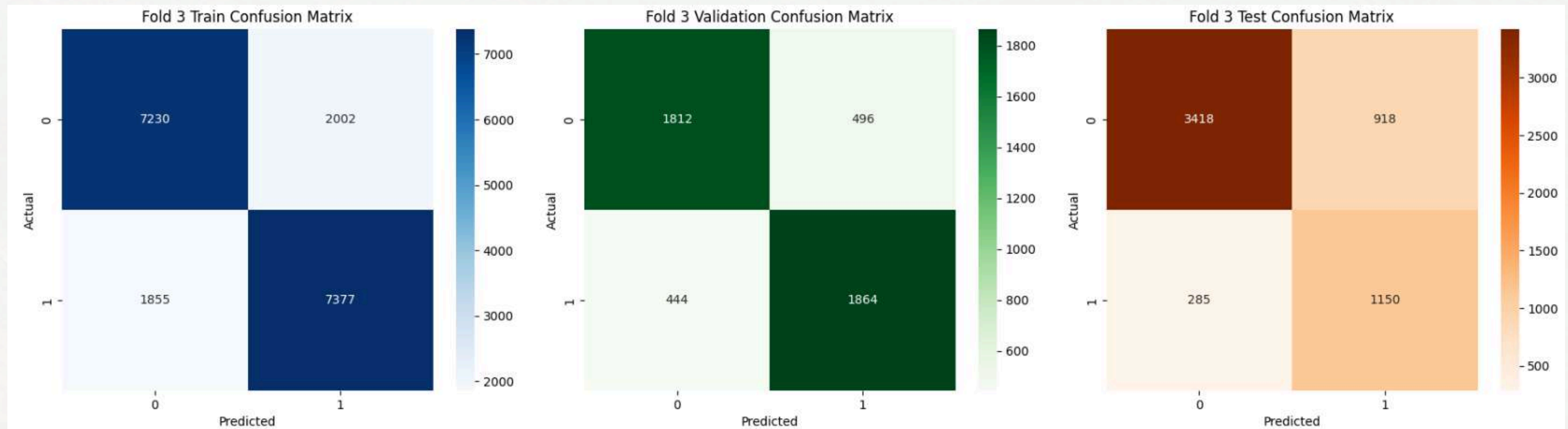
羅吉斯回歸-混淆矩陣Fold2

Train Accuracy: 0.7941, Validation Accuracy: 0.7877, Test Accuracy: 0.7926



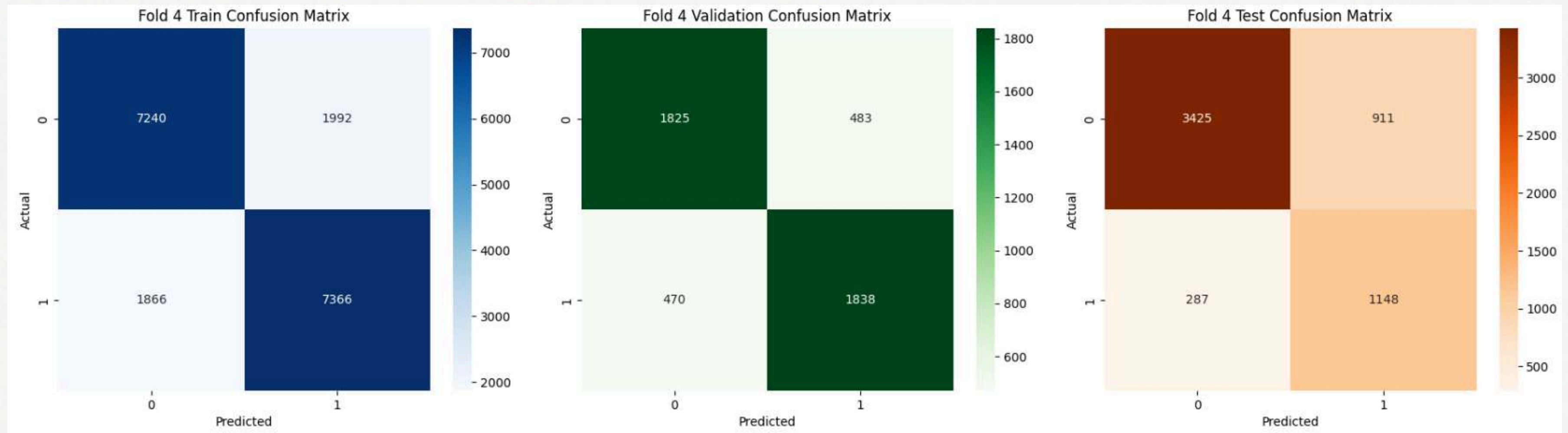
羅吉斯回歸-混淆矩陣Fold3

Train Accuracy: 0.7911, Validation Accuracy: 0.7964, Test Accuracy: 0.7915



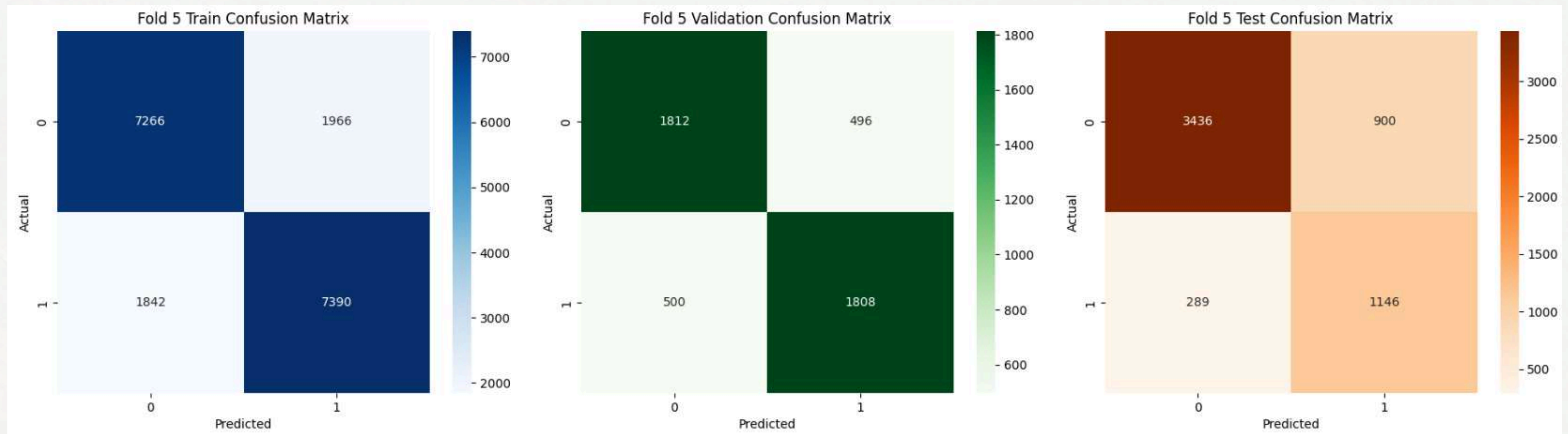
羅吉斯回歸-混淆矩陣Fold4

Train Accuracy: 0.7911, Validation Accuracy: 0.7935, Test Accuracy: 0.7924



羅吉斯回歸-混淆矩陣Fold5

Train Accuracy: 0.7938, Validation Accuracy: 0.7842, Test Accuracy: 0.7940



羅吉斯回歸-分類報告程式碼

```

train_acc_mean = np.mean(train_acc_list)
train_acc_std  = np.std(train_acc_list)
test_acc_mean  = np.mean(test_acc_list)
test_acc_std   = np.std(test_acc_list)
cv_acc_mean    = np.mean(cv_acc_list)
cv_acc_std     = np.std(cv_acc_list)

print("\n==== Average Accuracy =====")
print(f"Train:  {train_acc_mean:.4f} ± {train_acc_std:.4f}")
print(f"Test :  {test_acc_mean:.4f} ± {test_acc_std:.4f}")
print(f"CV      :  {cv_acc_mean:.4f} ± {cv_acc_std:.4f}")

# 偏差 = 每折 - 平均
train_acc_dev = np.array(train_acc_list) - train_acc_mean
test_acc_dev  = np.array(test_acc_list) - test_acc_mean
cv_acc_dev    = np.array(cv_acc_list) - cv_acc_mean

print("\n==== Accuracy Deviation per Fold =====")
for i in range(5):
    print(f"Fold {i+1}: Train={train_acc_dev[i]:+.4f}, Test={test_acc_dev[i]:+.4f}, CV={cv_acc_dev[i]:+.4f}")

```

```

def avg_classification_report(reports):
    keys = [k for k in reports[0].keys() if k not in ['accuracy', 'macro avg', 'weighted avg']]
    avg_report = {}
    std_report = {}
    for k in keys:
        avg_report[k] = {}
        std_report[k] = {}
        for metric in ['precision', 'recall', 'f1-score', 'support']:
            values = [r[k][metric] for r in reports]
            avg_report[k][metric] = np.mean(values)
            std_report[k][metric] = np.std(values)
    return avg_report, std_report

avg_train_report, std_train_report = avg_classification_report(train_reports)
avg_test_report, std_test_report = avg_classification_report(test_reports)

def print_avg_report(avg_report, std_report):
    for k in avg_report.keys():
        print(f"\nClass {k}:")
        for metric in ['precision', 'recall', 'f1-score', 'support']:
            mean = avg_report[k][metric]
            std = std_report[k][metric]
            print(f"{metric}: {mean:.4f} ± {std:.4f}")

print("\n==== Average Classification Report - Train (Mean ± Std) =====")
print_avg_report(avg_train_report, std_train_report)

print("\n==== Average Classification Report - Test (Mean ± Std) =====")
print_avg_report(avg_test_report, std_test_report)

```

羅吉斯回歸-分類報告

==== Average Accuracy =====

Train: 0.7921 ± 0.0015

Test : 0.7929 ± 0.0009

CV : 0.7918 ± 0.0050

==== Accuracy Deviation per Fold =====

Fold 1: Train=-0.0015, Test=+0.0009, CV=+0.0052

Fold 2: Train=+0.0020, Test=-0.0003, CV=-0.0041

Fold 3: Train=-0.0010, Test=-0.0013, CV=+0.0046

Fold 4: Train=-0.0011, Test=-0.0005, CV=+0.0018

Fold 5: Train=+0.0016, Test=+0.0011, CV=-0.0075

==== Average Classification Report - Train (Mean ± Std) =====

Class 0.0:

precision: 0.7961 ± 0.0017

recall: 0.7854 ± 0.0015

f1-score: 0.7907 ± 0.0015

support: 9232.0000 ± 0.0000

Class 1.0:

precision: 0.7883 ± 0.0014

recall: 0.7989 ± 0.0019

f1-score: 0.7935 ± 0.0015

support: 9232.0000 ± 0.0000

==== Average Classification Report - Test (Mean ± Std) =====

Class 0.0:

precision: 0.9232 ± 0.0007

recall: 0.7900 ± 0.0016

f1-score: 0.8514 ± 0.0008

support: 4336.0000 ± 0.0000

Class 1.0:

precision: 0.5582 ± 0.0015

recall: 0.8014 ± 0.0023

f1-score: 0.6580 ± 0.0010

support: 1435.0000 ± 0.0000

羅吉斯回歸-ROC曲線程式碼

```
# ROC 曲線
axes[1,0].plot(fpr_tr, tpr_tr, color='blue', lw=2, label=f'AUC = {roc_auc_train:.3f}')
axes[1,0].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,0].set_title(f'Fold {fold} Train ROC')
axes[1,0].set_xlabel('FPR')
axes[1,0].set_ylabel('TPR')
axes[1,0].legend(loc='lower right')

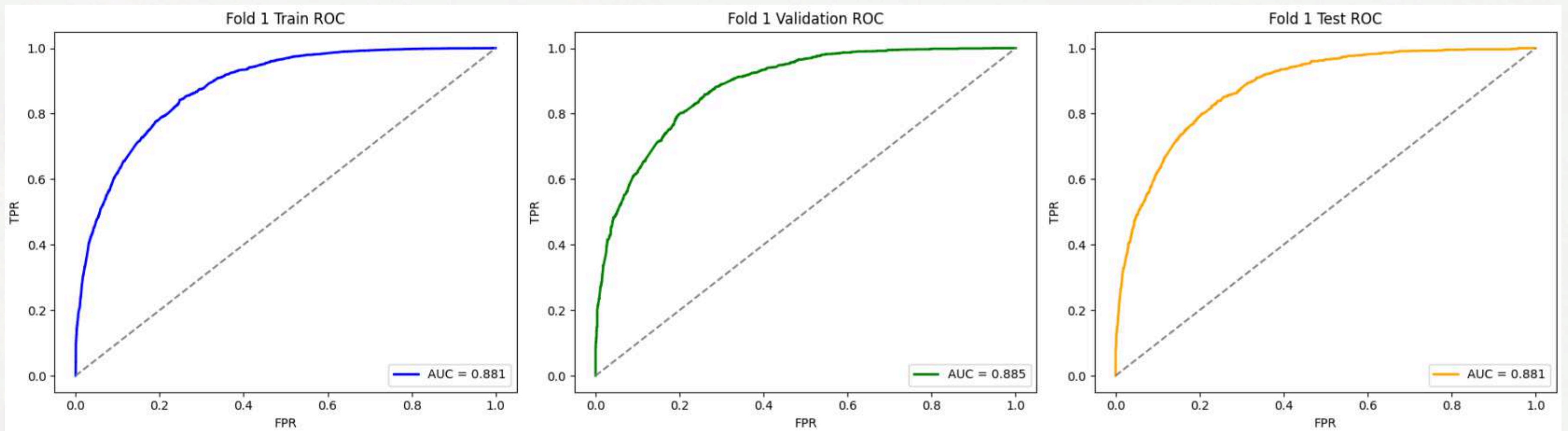
axes[1,1].plot(fpr_val, tpr_val, color='green', lw=2, label=f'AUC = {roc_auc_val:.3f}')
axes[1,1].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,1].set_title(f'Fold {fold} Validation ROC')
axes[1,1].set_xlabel('FPR')
axes[1,1].set_ylabel('TPR')
axes[1,1].legend(loc='lower right')

axes[1,2].plot(fpr_te, tpr_te, color='orange', lw=2, label=f'AUC = {roc_auc_test:.3f}')
axes[1,2].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,2].set_title(f'Fold {fold} Test ROC')
axes[1,2].set_xlabel('FPR')
axes[1,2].set_ylabel('TPR')
axes[1,2].legend(loc='lower right')

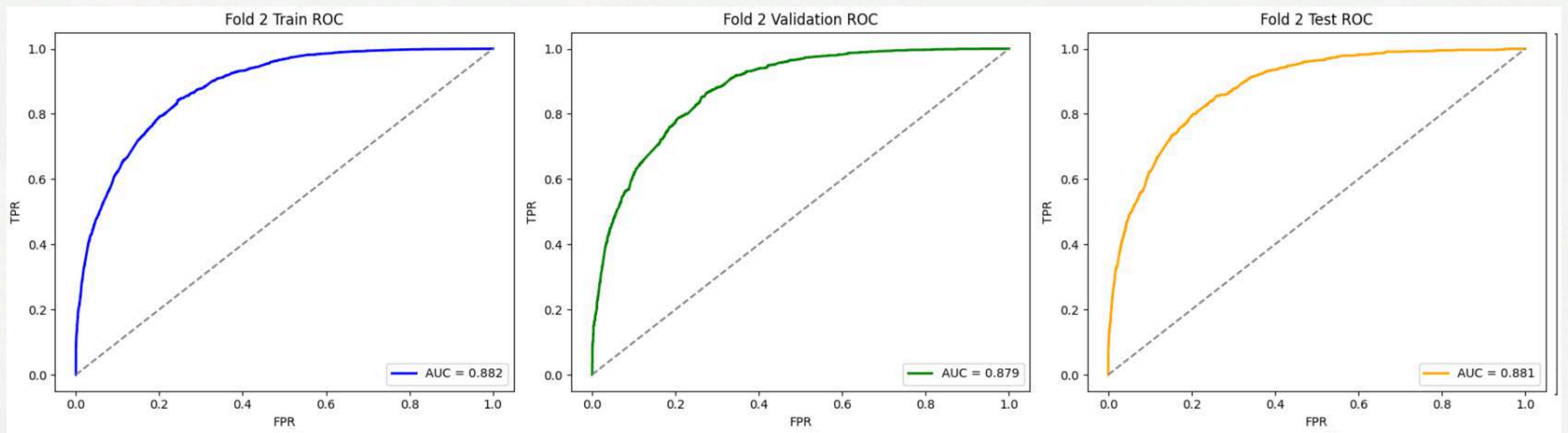
plt.tight_layout()
plt.show()

print(f"\n==== Fold {fold} =====")
print(f"Train Accuracy: {acc_train:.4f}, Validation Accuracy: {acc_val:.4f}, Test Accuracy: {acc_test:.4f}")
fold += 1
```

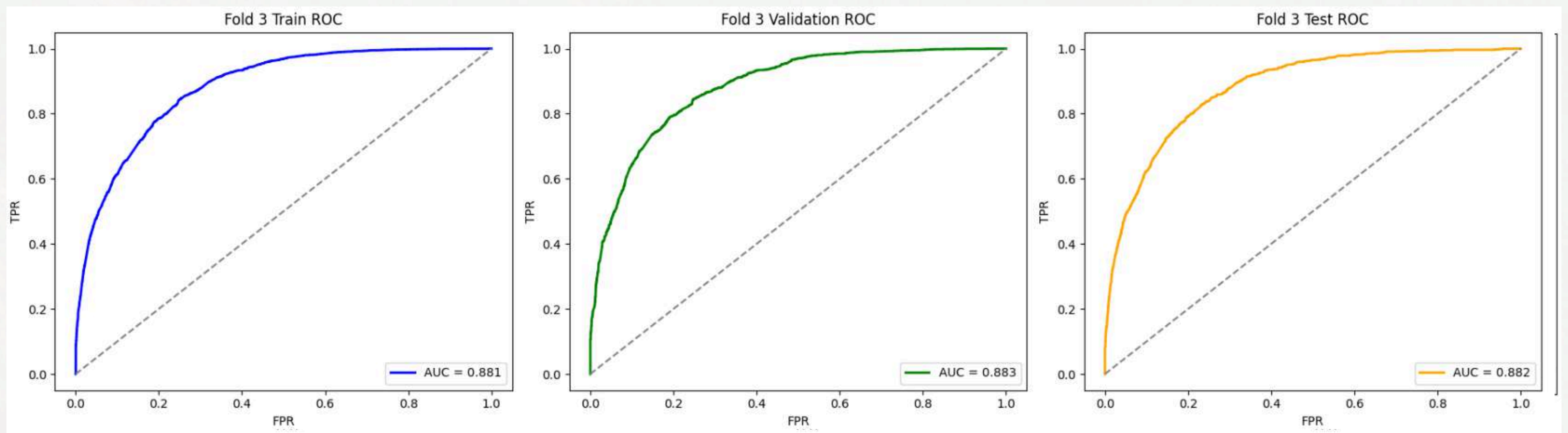
羅吉斯回歸-ROC曲線Fold1



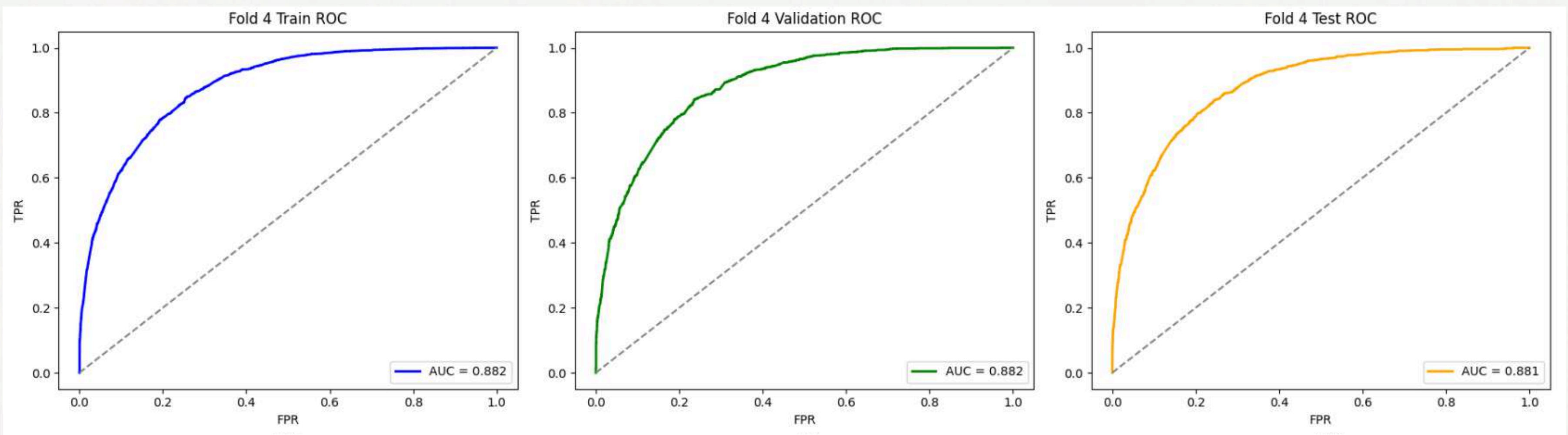
羅吉斯回歸-ROC曲線Fold2



羅吉斯回歸-ROC曲線Fold3

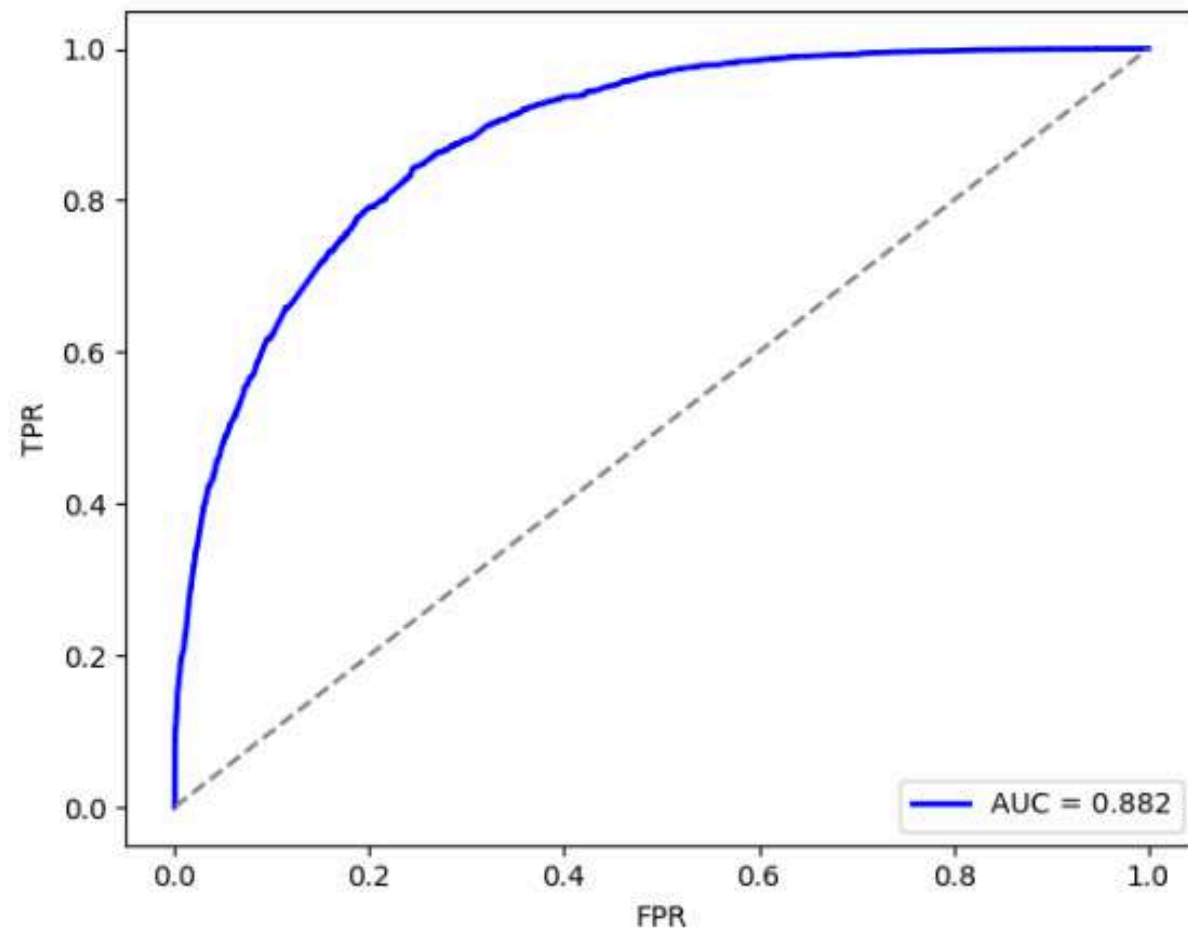


羅吉斯回歸-ROC曲線Fold4

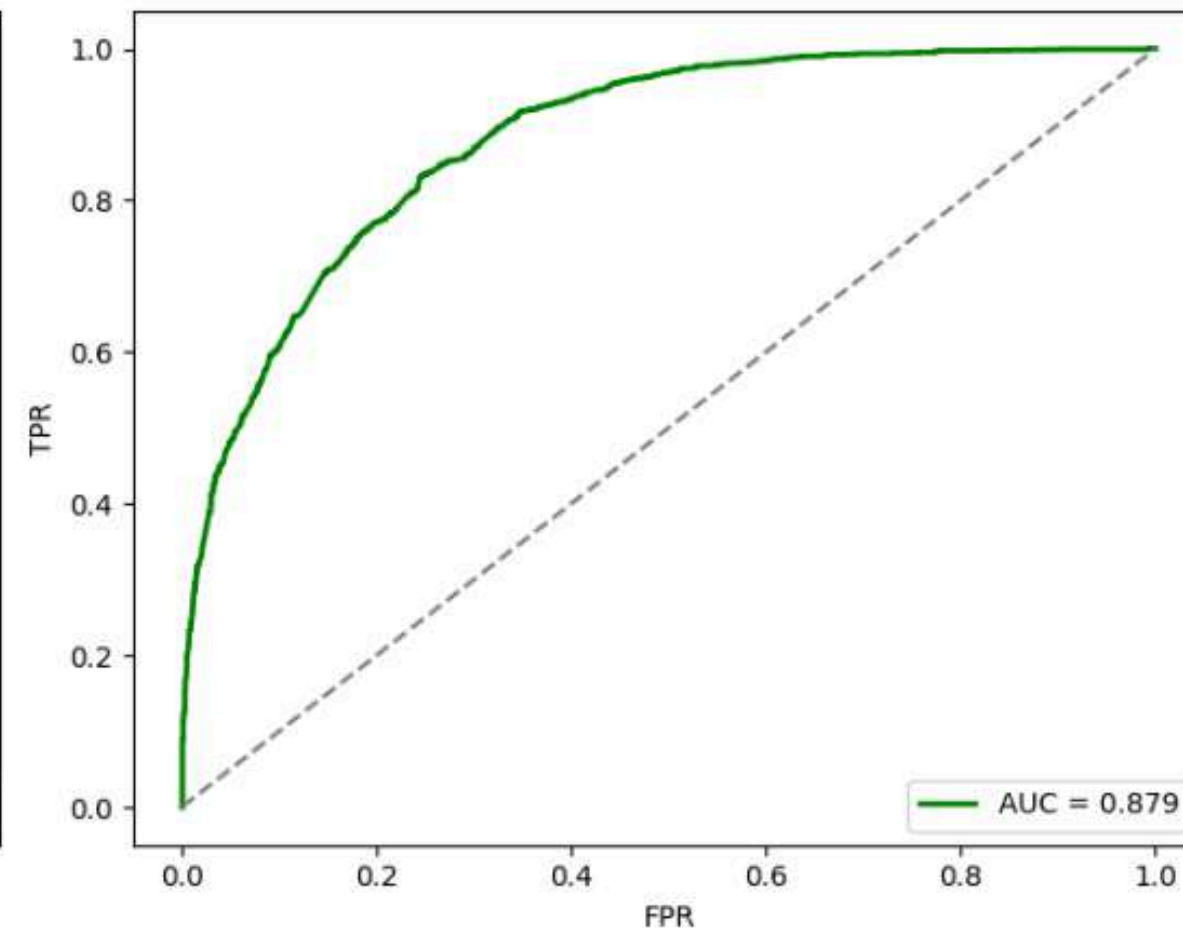


羅吉斯回歸-ROC曲線Fold5

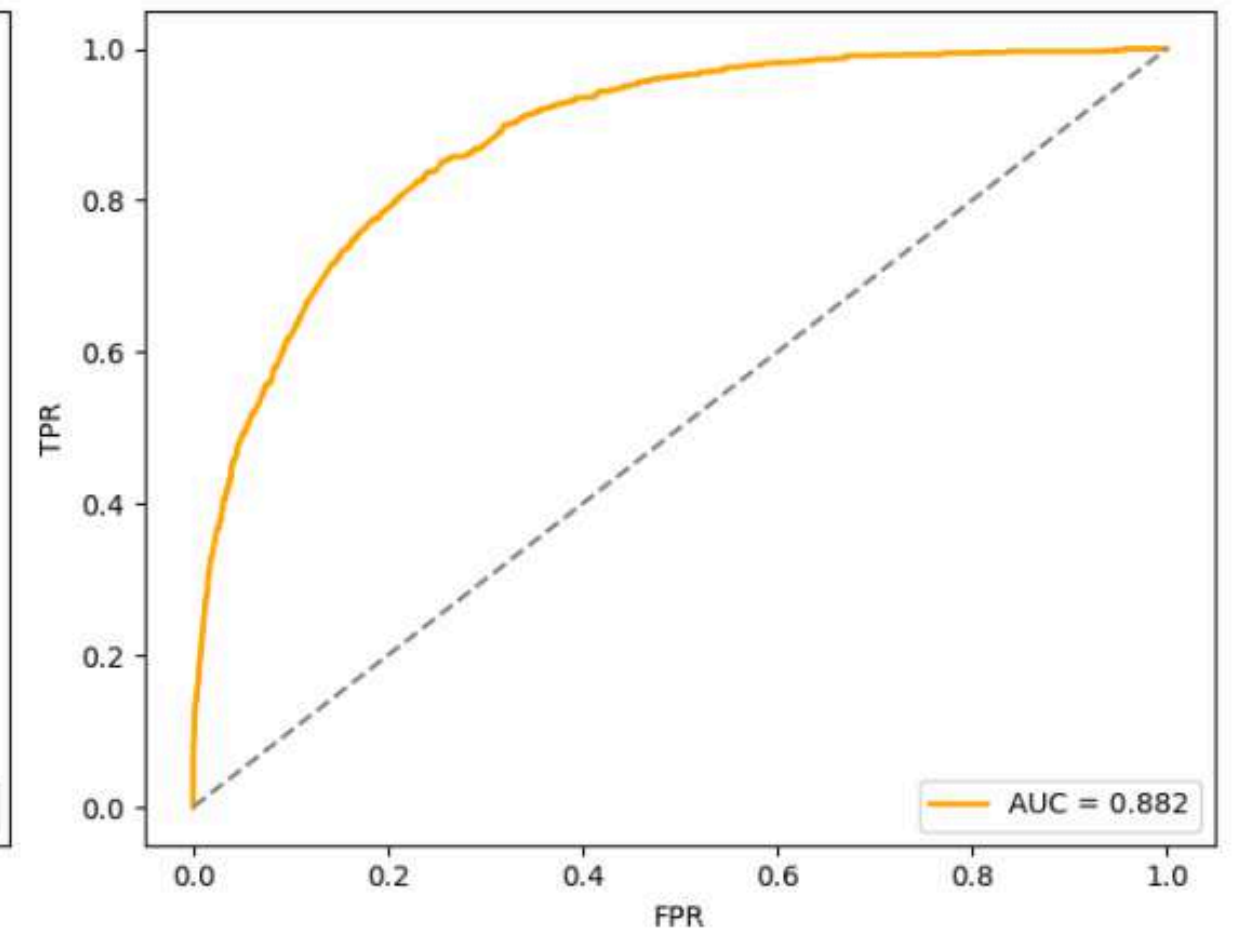
Fold 5 Train ROC



Fold 5 Validation ROC



Fold 5 Test ROC



羅吉斯回歸-平均ROC曲線程式碼

```
# =====
# 7 平均 ROC 曲線 ± 標準差
# =====

mean_fpr = np.linspace(0, 1, 100)
tpr_train_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_train_list])
tpr_val_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_val_list])
tpr_test_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_test_list])

mean_tpr_train = tpr_train_interp.mean(axis=0)
std_tpr_train = tpr_train_interp.std(axis=0)
mean_tpr_val = tpr_val_interp.mean(axis=0)
std_tpr_val = tpr_val_interp.std(axis=0)
mean_tpr_test = tpr_test_interp.mean(axis=0)
std_tpr_test = tpr_test_interp.std(axis=0)

plt.figure(figsize=(18, 5))

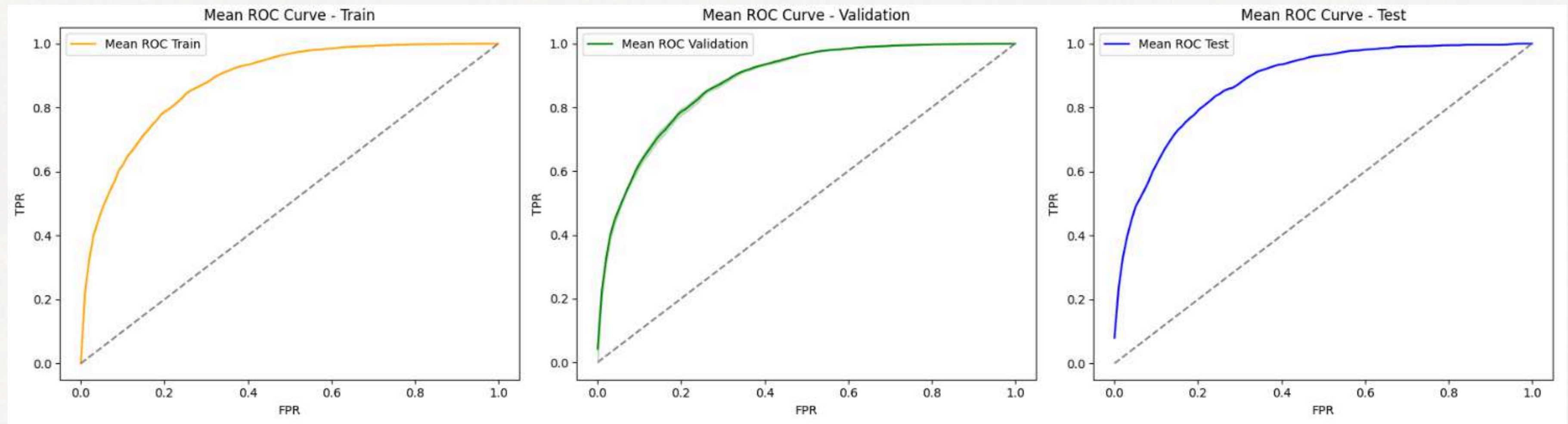
plt.subplot(1, 3, 1)
plt.plot(mean_fpr, mean_tpr_train, color="orange", label="Mean ROC Train")
plt.fill_between(mean_fpr, mean_tpr_train-std_tpr_train, mean_tpr_train+std_tpr_train, color="orange", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Train")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(mean_fpr, mean_tpr_val, color="green", label="Mean ROC Validation")
plt.fill_between(mean_fpr, mean_tpr_val-std_tpr_val, mean_tpr_val+std_tpr_val, color="green", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Validation")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.subplot(1, 3, 3)
plt.plot(mean_fpr, mean_tpr_test, color="blue", label="Mean ROC Test")
plt.fill_between(mean_fpr, mean_tpr_test-std_tpr_test, mean_tpr_test+std_tpr_test, color="blue", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Test")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.tight_layout()
plt.show()
```

羅吉斯回歸-平均ROC曲線



羅吉斯回歸-特徵重要性程式碼

```
# =====  
# 8 特徵重要性 & Odds Ratio  
# =====  
coef_df = pd.DataFrame({  
    'feature': feature_cols,  
    'coefficient': pipeline.named_steps['logistic'].coef_[0]  
})  
coef_df['odds_ratio'] = np.exp(coef_df['coefficient'])  
coef_df = coef_df.sort_values(by='coefficient', key=abs, ascending=False)  
print("\n特徵重要性與勝算比 (Odds Ratio):\n", coef_df)
```


羅吉斯回歸-特徵重要性

特徵重要性與勝算比 (Odds Ratio):

	feature	coefficient	odds_ratio
10	capital.gain	1.085067	2.959639
4	rel_cat	0.816616	2.262828
9	age_cat	0.784244	2.190751
1	edu_cat	0.480105	1.616245
3	occ_cat	0.414385	1.513439
12	hours.per.week	0.355438	1.426806
11	capital.loss	0.310174	1.363663
6	sex_cat	-0.307812	0.735053
8	edun_cat	0.291041	1.337820
5	race_cat	-0.098656	0.906054
7	nat_cat	0.091121	1.095402
2	mar_cat	0.075029	1.077915
0	work_cat	0.027273	1.027648

模型比較

	隨機森林	決策樹	羅吉斯回歸
accuracy	0.78	0.70	0.79
precision	0.75	0.45	0.55
recall	0.86	0.89	0.80
f1-score	0.80	0.59	0.65

準確率：羅吉斯回歸 > 隨機森林 > 決策樹

f1-score：隨機森林 > 羅吉斯回歸 > 決策樹

會議記錄

這個時間大家可以嗎 18:00

12/13 (六)

淇 我可以歐 00:25

7n 可以 00:26

ai 我們有需要先行討論嗎

淇 我都可以欸 但是我星期日不太知道什麼時候回家 14:05

雲 那接下來要決定線上討論的時間 16:44

雲 我是 10/30 (三) 上午 10/31 (四) 上午+晚上有空 16:47

淇 禮拜三下午 禮拜四四點之後 16:47

雲 如果是下周報告的話 這個作業比較急 而且模型修正感覺會需要很多時間 17:10

ai 我三晚上 10.後 週四 4-5.30、9.後 週五 4-5.30、10.後 週六日皆可 17:11

@雲 禮拜四上完課妳有要先上班嗎

已讀 4 19:27

ai 這個時間大家可以嗎

可以

已讀 4 15:38

14:57 4G 36%

涵雲的螢幕畫面

Google Chrome 瀏覽器

已加入 5

吳文秀

淇

涵雲

7n

ai

退出

關閉麥克風

音效設定

派對功能

14:54 5G 68%

資料探勘 (5)

1:49:22

淇

盤古 (雲)

吳文秀

7n

ai

淇, ai

退出

關閉麥克風

關閉擴音

派對功能

參考文獻 & 分工表



參考文獻

1.性別與種族對美國收入影響

Greenman&Xie (2008)-Double Jeopardy?

The Interaction of Gender and Race on Earnings in the U.S.

2.教育年數與工作收入

Polachek(2007)-The Mincer Earnings Function and Its Applications.

Heckman(2005)-The Mincer Equation and Beyond.

3.美國退休年齡

https://www.ccyp.com/CCYPContents?content_id=129848

4.美國工時標準

<https://www.dol.gov/sites/dolgov/files/WHD/legacy/files/whdfsChinese.pdf>

分工表

111029208 吳文秀	112029012 翁鈺淇	112029030 魏麒恩	112029040 劉涵雲	112029043 朱芯僊
資料前處理 隨機森林 決策樹 簡報製作	資料前處理 視覺化分析 隨機森林 簡報製作 協助圖片處理	資料前處理 視覺化分析 簡報製作	資料前處理 視覺化分析 羅吉斯回歸 決策樹 簡報製作 協助書面報告	資料描述 資料前處理 視覺化分析 簡報製作 簡報編排 Q&A 書面報告

Q&A 與建議



Q1：為什麼年齡區段18~25歲是設定為青少年，26-35是青壯年，36-65是壯年？

A1：

統計機構的年齡類別分類例如加拿大統計局將人口分為：

- Youth (15-24歲) 、Adults (25-64歲) 、Seniors (65歲以上)

<https://www.statcan.gc.ca/en/concepts/definitions/age2>

因為我們是設定18歲以下不採納(多不具備工作能力)且65歲以上才退休，所以將65歲包含入壯年區間，而青少年區間設定從18歲開始。

加上區分年齡結構這種分法側重於政策與勞動力分析，有助於比較人口結構與社會資源配置，所以我們採納這樣的方式分配。

Q2：SMOTE 是怎麼放鬆的？如何確定他不是重複抽取同一筆欄位的資料？

A2：

SMOTE 是在兩個少數類別樣本之間插值產生新樣本。

數學上是這樣： $x_{\text{new}} = x_i + \lambda \times (x_{\text{nn}} - x_i)$

- x_i ：某一筆少數類別樣本、 x_{nn} ：它的 k 個最近鄰之一、 λ ：0 到 1 之間的隨機數
- 它會讓新資料點落在兩個真實樣本之間的線段上。我們也可以透過程式碼二次確認。

```
# =====
# 5 訓練集套用 SMOTE 平衡 (維持總數)
# =====
N_train = len(y_train)
target_per_class = N_train // 2 # 每類筆數目標

# 少數類用 SMOTE 補足
minority_class = y_train.value_counts().idxmin()
smote = SMOTE(sampling_strategy=(minority_class: target_per_class), random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

```
# 多數類超過 target → 隨機抽樣
majority_class = y_train.value_counts().idxmax()
X_major = X_train_res[y_train_res==majority_class]
y_major = y_train_res[y_train_res==majority_class]
if len(y_major) > target_per_class:
    X_major_down, y_major_down = resample(
        X_major, y_major, replace=False, n_samples=target_per_class, random_state=42
    )
X_minor = X_train_res[y_train_res!=majority_class]
y_minor = y_train_res[y_train_res!=majority_class]
# 合併
X_train_res = pd.concat([X_major_down, X_minor], axis=0).reset_index(drop=True)
y_train_res = pd.concat([y_major_down, y_minor], axis=0).reset_index(drop=True)

print("平衡後訓練集大小:", len(y_train_res))
print("平衡後各類別數量:\n", y_train_res.value_counts())
```

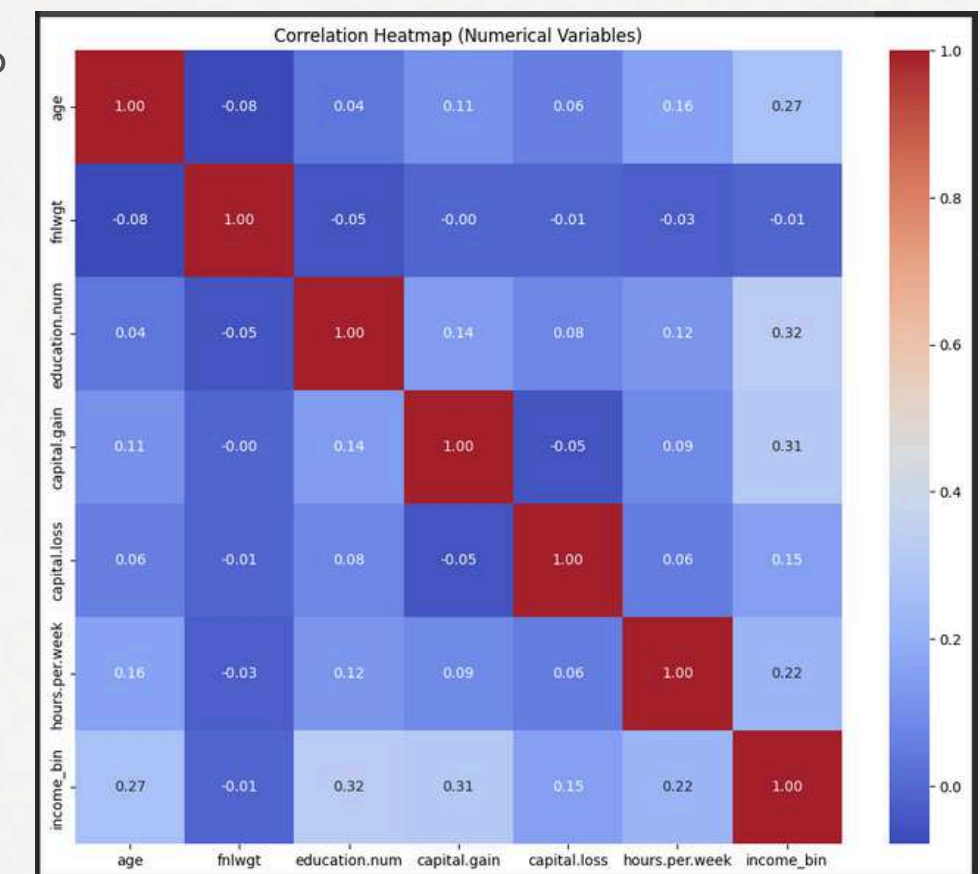
```
# =====
# 6 儲存訓練集與測試集
# =====
train_data = pd.concat([X_train_res.reset_index(drop=True), y_train_res.reset_index(drop=True)])
test_data = pd.concat([X_test.reset_index(drop=True), y_test.reset_index(drop=True)], axis=1)
pyreadstat.write_sav(train_data, "train_data.sav")
pyreadstat.write_sav(test_data, "test_data.sav")
print("檔案已成功產生: train_data.sav & test_data.sav")
```


Q3：熱力圖是用編碼後的資料還是原始資料？

A3：

我們原先是使用編碼後的資料，因為我們有設定區間(如年齡、教育程度等都改為數值)，但後面根據老師說的我們有意識到很大的問題就是：因為類別型變數即使被數值編碼，他的數值大小本身不具備連續或距離意義，直接計算 Pearson 相關係數是不恰當的。

例如：性別：Male=1、Female=2；教育程度："1"表示"低"、"2"表示"中"、"3"表示"中高"、"4"表示"高"。這些數字只是標籤，不是量測值，如果用來算相關係數，可能會產生「假相關」。後續我們補了一張只有連續型資料的熱力圖如下。



有放至p.22熱力圖簡報頁

Q4：隨機森林我們使用哪些特徵？

A4：

我們一開始是將所有特徵變數納入模型，並透過模型計算各變數之特徵重要度，先做初步辨識對高收入具有較高影響力的潛在因素。但後來根據老師的建議說明並考量類別型變數的統計尺度特性，以及避免把解釋力有限的變數無差別納入分析，經指導後對特徵選取方式進行修正。

所以我們最後依據隨機森林特徵重要度結果，選定婚姻狀態、學歷、每週工作時數、資本收益、年齡及職業類別作為主要特徵因素，作為後續模型建構與分析之自變數，以提升模型解釋力與真實性。

Q5：隨機森林為何採用 Macro/Micro/Weighted average 作為模型評估指標？

A5：

我們一開始為了完整呈現模型評估結果，我們直接引用 Macro/Micro/Weighted average 指標，並將其作為模型表現的摘要指標。經老師提醒後，我們理解到這些平均方式主要用於多分類情境或用來比較不同類別權重的影響；在本研究的二分類架構下，這些平均值容易造成資訊重複或解釋混淆，因此需調整，才不會影響模型解釋的準確性。在修改過後我們就沒有正負樣本放在一起看的問題了，詳細修改皆在隨機森林簡報頁。

Q6：隨機森林為何沒有算ACC準確率？

A6：

本研究在初期模型評估時，對 **Accuracy**（準確率）的定義理解不夠精確，誤以為其與交叉驗證（五折驗證）結果直接相關。經老師指正後，我們重新釐清 **Accuracy** 的正確意義，並據此修正模型評估方式後將截圖補上在隨機森林報告頁面。

Accuracy 為混淆矩陣中正確分類樣本所占的比例，其計算方式為混淆矩陣對角線元素（TP 與 TN）之總和，除以全部樣本數，即 $= (TP + TN) / (TP + TN + FP + FN)$

在釐清觀念後，我們在隨機森林模型評估中補充呈現 **Accuracy** 作為整體表現之參考指標，搭配其他指標進行解讀，以避免單一指標造成模型表現的誤判。

Q7：隨機森林為何沒有算標準差？

A7：

因為我們在報告時僅呈現其中一折模型訓練所得到之評估指標（Accuracy、Precision、Recall 與 F1-score），未進一步計算四個指標的五個結果的標準差，因此無法用以衡量模型在不同資料切分下的表現變異程度，更無法完整呈現模型表現的穩定性。

經老師指導與後續檢討後，我們已明確理解：若需呈現評估指標的標準差，應完整列出五折交叉驗證中每一折的評估結果，並據此計算其平均值與標準差，以提升模型評估結果之完整性與可信度。我們已補上在隨機森林簡報頁。

Q8：我們的決策樹是哪一種？

A8：

我們所使用之決策樹演算法為 CART (Classification and Regression Tree)。原因在於我們使用 Python 的 scikit-learn 套件進行隨機森林模型建構，而 scikit-learn 中的 RandomForestClassifier 與 DecisionTreeClassifier 皆以 CART 演算法作為基礎。CART 採用二元分裂架構，並以 Gini impurity (或 Entropy) 作為節點分裂準則，與我們的模型設定與二分類任務相符。

Q9：何謂資本收益？

A9：

資本收益（Capital Gain）是指個人因持有之資產在出售或處分時，價格高於原始取得成本所產生的收益，而非來自勞動所得。常見的資本收益來源包括股票、基金、不動產或其他投資性資產的買賣差價。

在本研究所使用之資料集中，`capital.gain` 變數用以表示個人在特定期間內，因資產交易所獲得的總資本收益金額。此變數屬於非勞動收入的一種，反映個人投資活動與資產累積狀況，與收入水準具有高度關聯性，因此被納入作為影響收入分類的重要特徵之一。

<https://www.irs.gov/taxtopics/tc409>

Q10：為何決策樹明明是二分類卻有超過兩個顏色？

A10：

本研究為二分類問題，決策樹圖中顏色之「種類」代表預測類別，我們只有二分類所以確實也只有藍色跟橘色兩種顏色，至於顏色之「深淺」則是反映節點純度（Gini impurity）。顏色越深表示該節點中某一類別所占比例越高，模型判斷越確定；顏色較淺則表示類別分布較為混雜。所以即使為二分類模型，仍會呈現預測類別的顏色深淺。

scikit-learn 官方文件明確說明：

1. 節點顏色代表「預測類別」
2. 節點顏色的強度（intensity）與「該類別在節點中的比例」有關

<https://scikit-learn.org/stable/modules/tree.html>

.....

Thank
you

.....